

Trường Đại học Công nghệ thông tin
Đại học Quốc gia TP. Hồ Chí Minh

Đề tài luận văn thạc sĩ Khoa học máy tính — Mã số 8480101

**HỆ THỐNG SỬ DỤNG MÔ HÌNH NGÔN NGỮ LỚN ĐIỀU
PHỐI HỢP NHẤT
PHÂN TÍCH TĨNH - ĐỘNG TRONG PHÁT HIỆN RÒ RỈ BỘ
NHỚ C/C++**

***AN LARGE LANGUAGE MODEL - ORCHESTRATED STATIC - DYNAMIC
SYSTEM FOR C/C++ MEMORY LEAK DETECTION***

Học viên thực hiện: Lê Đăng Dũng

Mã số: 230202023 — Khóa 18, Đợt 2

Cán bộ hướng dẫn: TS. Phan Thế Duy

TP. Hồ Chí Minh, 2026

Lời cảm ơn

[Điền sau — todo-16]

Tóm tắt

Rò rỉ bộ nhớ (CWE-401) là lỗi phổ biến trong C/C++: bộ nhớ cấp phát không bao giờ được giải phóng trong khi chương trình vẫn chạy. Phân tích tĩnh (Clang Static Analyzer, Infer) gắn cờ ứng viên nhưng suy luận đường đi và ownership chưa đủ, báo sai cao; phân tích động (Valgrind Memcheck, AddressSanitizer/LeakSanitizer) cho bằng chứng chắc chắn nhưng chỉ trên đường thực thi thật. Cả hai dừng ở cảnh báo: không giải thích nguyên nhân gốc, không đề xuất cách sửa. Luận văn đề xuất CLeak: LLM điều phối điều tra rò rỉ qua bốn tầng discovery (quét vị trí cấp phát), static enrichment (làm giàu bằng chứng tĩnh), investigation agentic (Stage A-D) và judging hybrid (heuristic cho mọi bundle, LLM cho bundle borderline). Công cụ phân tích expose qua MCP; tái lập theo giao thức hai tầng: no_llm tắt định, llm_assisted báo phân phối. Hệ được đánh giá trên Juliet CWE-401 (1658 ca), LAMeD (50 site thực) và MemHint (19 ca, 6 dự án thực) với ablation 9 baseline. B6a đề xuất đạt F1 0.863 ± 0.001 trên toàn corpus so với 0.612 của baseline tĩnh B1; LAMeD bắt 15/50 site (recall 30.0%, std=0); MemHint F1 0.632 ± 0.000 (llm_assisted $\equiv$ no_llm); consensus k=3 đảo chiều so với single-LLM trên mẫu stratified (lật verdict 8.0% so với 2.0%, $p=0.077$). Đóng góp: pipeline hybrid tắt định-trừ-judge, giao thức tái lập hai tầng, làm giàu bằng chứng cho judge, và kết quả âm consensus được kể trung thực.

Abstract

Memory leaks (CWE-401) are a common defect class in C/C++: allocated memory is never released, the program keeps running, and ordinary testing rarely surfaces the problem. Static analysis (Clang Static Analyzer, Infer) flags candidates without executing the program, but its path and ownership reasoning is incomplete and its false-positive rate is high; dynamic analysis (Valgrind Memcheck, AddressSanitizer/LeakSanitizer) provides certain evidence, but only on paths that are actually executed. Both stop at a warning: neither explains the root cause nor proposes a fix.

This thesis presents CLeak, a system in which a large language model (LLM) orchestrates the leak investigation across four stages: discovery (allocation-site scan), static enrichment, agentic investigation (Stage A-D), and hybrid judging (a heuristic for every bundle, an LLM judge for borderline bundles). The analysis tools are exposed over the Model Context Protocol (MCP), and reproducibility follows a two-tier protocol: `no_llm` is deterministic, `llm_assisted` is reported as a distribution. The system is evaluated on Juliet CWE-401 (1,658 cases), LAMeD (50 real sites), and MemHint (19 cases across 6 real projects) with a 9-baseline ablation. The proposed configuration B6a reaches F1 0.863 ± 0.001 on the full corpus versus 0.612 for the static-only baseline B1; on LAMeD the system catches 15 of 50 sites (recall 30.0%, $\text{std}=0$ across 3 runs); on MemHint it reaches F1 0.632 ± 0.000 with `llm_assisted` identical to `no_llm`; consensus $k=3$ reverses against single-LLM on the stratified sample (verdict-flip 8.0% vs 2.0%, $p=0.077$). Contributions include a deterministic-except-judge hybrid pipeline, a two-tier reproducibility protocol, evidence enrichment for the judge, and a negative consensus result reported honestly.

Mục lục

Lời cảm ơn.....	2
Tóm tắt.....	3
Abstract.....	4
Mục lục	5
Danh mục hình vẽ.....	15
Chương 1: Các nghiên cứu và công nghệ liên quan	16
1.1. Memory leak trong C/C++.....	16
1.1.1. Định nghĩa và phân loại.....	16
1.1.2. Patterns phổ biến.....	16
1.1.3. Tác động thực tế	17
1.1.4. Độ phức tạp của bài toán	18
1.2. Phân tích tĩnh cho memory leak	18
1.2.1. Clang Static Analyzer	18
1.2.2. Facebook Infer	18
1.2.3. CodeQL.....	19
1.2.4. Tree-sitter.....	19
1.2.5. NESA — phân tích tĩnh neuro-symbolic.....	19
1.2.6. CodeChecker và khả năng mở rộng.....	19
1.2.7. Abstract interpretation và symbolic execution	20
1.2.8. Kiến trúc quy mô công nghiệp — tiền lệ mà CLeak áp dụng	20
1.3. Phân tích động cho memory leak.....	21
1.3.1. Valgrind Memcheck	21
1.3.2. AddressSanitizer và LeakSanitizer	22
1.3.3. RangeSanitizer.....	22

1.3.4. QMSan	22
1.3.5. CombiSan	22
1.3.6. AirTaint — taint analysis nhanh hơn.....	23
1.3.7. Fuzzers vs Static Analyzers — hai thế giới gần như không giao nhau	23
1.4. LLM cho phát hiện lỗi phần mềm	23
1.4.1. Khả năng hiểu code của LLM	23
1.4.2. LLM kết hợp công cụ: ReAct và tool-augmented reasoning.....	24
1.4.3. IRIS — LLM kết hợp static analysis	24
1.4.4. Prompting như proxy cho static analysis.....	24
1.4.5. VulnLLM-R — LLM chuyên biệt cho vulnerability detection	25
1.4.6. SemTaint — multi-agent taint specification.....	25
1.4.7. MemHint — neuro-symbolic cho memory leak.....	25
1.4.8. LAMeD — LLM annotation cho analyzer cổ điển	25
1.4.9. Revelio — agentic với sanitizer proof.....	26
1.4.10. SAILOR — symbolic execution với LLM.....	26
1.4.11. Hệ thống multi-agent cho software engineering.....	26
1.5. Cơ chế đồng thuận và giảm phương sai LLM	27
1.5.1. Self-consistency decoding	27
1.5.2. LLM-as-judge: calibration và dao động	28
1.5.3. McNemar test cho paired comparison	28
1.6. Benchmarks và datasets	28
1.6.1. Juliet Test Suite (NIST)	28
1.6.2. LAMeD Benchmark	28
1.6.3. DiverseVul.....	29
1.6.4. SV-COMP Memsafety.....	29

1.6.5. Magma	29
1.7. Tổng kết và vị trí nghiên cứu.....	29
Chương 2: Lập kế hoạch, phân tích và thiết kế hệ thống	31
2.1. Yêu cầu và ràng buộc	31
2.1.1. Yêu cầu chức năng.....	31
2.1.2. Yêu cầu phi chức năng.....	31
2.1.3. Ràng buộc kỹ thuật	31
2.2. Phân tích bài toán và lựa chọn kiến trúc	32
2.2.1. Tại sao hybrid — kết hợp static và dynamic	32
2.2.2. Tại sao LLM orchestration thay vì hardcode pipeline.....	32
2.2.3. Tại sao MCP thay vì gRPC/REST.....	32
2.3. Thiết kế tổng quan	33
2.3.1. Pipeline HYBRID	33
2.3.2. Nguyên tắc thiết kế: LLM sở hữu POLICY, engine sở hữu MECHANISM	33
2.3.3. Sơ đồ thành phần	33
2.3.4. Luồng dữ liệu.....	35
2.4. Thiết kế tầng khám phá tĩnh (Discovery)	36
2.4.1. Candidate scan	36
2.4.2. Attribution hàm bằng Tree-sitter	36
2.5. Thiết kế tầng làm giàu bằng chứng tĩnh (Static Enrichment)	36
2.5.1. Function summary	36
2.5.2. Path constraints	36
2.5.3. Interprocedural flow	37
2.5.4. Clang scan-build	37
2.6. Thiết kế tầng điều tra agentic (Investigation)	37

2.6.1. Stage A — Static fan-out sub-agents	37
2.6.2. Stage B — Dynamic worker	37
2.6.3. Stage C — Synthesize	38
2.6.4. Stage D — Hybrid judge	38
2.7. Thiết kế tầng phán xét hybrid (Judging)	38
2.7.1. Heuristic judge — path-sensitive, cho mọi bundle	38
2.7.2. LLM judge — rubric-based, chỉ cho bundle borderline	39
2.7.3. Consensus judge — K-sample voting	39
2.7.4. Escalation logic	40
2.8. Thiết kế giao thức tái lập hai tầng	40
2.8.1. Vấn đề: LLM không tắt định bit-for-bit	40
2.8.2. Tier-1 — no_llm tắt định	40
2.8.3. Tier-2 — llm_assisted báo cáo phân phối	40
2.9. Thiết kế corpus pipeline	41
2.9.1. Chuẩn bị và ingest	41
2.9.2. Validate — 5-gate	41
2.9.3. Lock và gate	41
2.10. Tổng kết chương	41
Chương 3: Hiện thực, triển khai hệ thống	42
3.1. Công nghệ sử dụng và cấu trúc dự án	43
3.1.1. Lựa chọn ngôn ngữ và runtime	43
3.1.2. Kiến trúc monorepo Turborepo	44
3.1.3. Các framework và thư viện chính	45
3.2. Cài đặt static analyzer	46
3.2.1. C-parser service: routing C/C++ và quản lý allocator	46

3.2.2. Đăng ký 11 MCP tools	48
3.2.3. Phân tích path-sensitive: guard-subset reconciliation.....	49
3.2.4. Reachability bảo thủ: collectDeadLines	49
3.2.5. Parameter-ownership leak	50
3.3. Cài đặt dynamic analyzer.....	50
3.3.1. Build orchestration và sanitizer injection	50
3.3.2. Valgrind integration.....	51
3.3.3. llvm-symbolizer.....	51
3.3.4. Đăng ký 9 MCP tools	51
3.4. Cài đặt agent core	52
3.4.1. Vòng lặp tool-calling native	52
3.4.2. Completion nudge mechanism	53
3.4.3. Multi-provider streaming.....	53
3.5. Cài đặt orchestrator (leak-inspector-tui).....	54
3.5.1. Workflow 4-stage	54
3.5.2. LLM judge và escalation logic	56
3.5.3. Consensus judge: K-sample voting	57
3.6. Cài đặt common library (@cleak/common)	57
3.6.1. Heuristic judge: scoring function.....	57
3.6.2. Consensus judge: combineVerdicts	58
3.6.3. Report renderers.....	59
3.7. Cài đặt tầng LLM-generalization (POLICY)	59
3.7.1. Allocator profiler	59
3.7.2. Strategist	60
3.7.3. Judge tuner.....	61

3.8. Triển khai và vận hành	61
3.8.1. Docker Compose	61
3.8.2. LLM gateway.....	61
3.8.3. Configuration hierarchy	62
3.8.4. Corpus pipeline	62
3.9. Tổng kết chương	62
Chương 4: Đánh giá kết quả	64
4.1. Thiết kế thực nghiệm	64
4.1.1. Corpus.....	64
4.1.2. Scoring model	64
4.1.3. Metrics	64
4.1.4. Sampling	65
4.1.5. Fairness rules	65
4.1.6. LLM configuration	65
4.2. Ablation 9-baseline capability (headline)	65
4.2.1. Thiết kế	65
4.2.2. Kết quả full-corpus 1658 ca (headline)	66
4.2.3. Kết quả n=50 stratified (thí nghiệm đầu, giữ làm phụ)	66
4.2.4. Kết quả n=100.....	67
4.2.5. Hòa giải: stratified 0.938 so với full-corpus 0.863	67
4.3. Ma trận 2×2 (LLM orchestration × Dynamic evidence)	68
4.4. Ablation static evidence tools	68
4.5. Đánh giá trên LAMeD — dự án thực	69
4.5.1. Kết quả trên denominator 50 site (bộ run 2026-08-20)	69
4.5.2. Case study: cJSON merge_patch.....	70

4.5.3. Phân tích 6 leak classes cJSON	70
4.6. Ablation consensus judge	71
4.6.1. Thí nghiệm đầu: n=30.....	71
4.6.2. Đo lại trên mẫu stratified n=50: kết quả đảo ngược	72
4.6.3. Vì sao kết quả đầu sai, và kết luận.....	72
4.7. So sánh với baseline bên ngoài.....	73
4.7.1. Juliet n=30 (live re-run, cùng scorer)	73
4.7.2. LAMeD.....	73
4.7.3. So sánh với các hệ thống LLM khác	73
4.8. Xác minh tái lập.....	74
4.8.1. Tier-1	74
4.8.2. Tier-2	74
4.9. Độ chính xác LLM allocator profiling.....	75
4.10. Đánh giá trên corpus MemHint	75
4.10.1. Kết quả.....	76
4.10.2. Đối chiếu với số liệu MemHint tự báo	77
4.10.3. Vì sao không chạy đủ 9 baseline trên corpus này.....	77
4.10.4. Threats to validity	78
4.11. Tổng hợp chương.....	78
Bảng tổng hợp toàn bộ kết quả	78
Điều kiện nào LLM orchestration có lợi?.....	79
Chương 5: Kết luận và hướng phát triển trong tương lai.....	80
5.1. Trả lời câu hỏi nghiên cứu.....	80
RQ1: LLM orchestration có cải thiện hiệu suất phát hiện leak không?	80
RQ2: Kết quả có tái lập được không?.....	81

RQ3: Consensus voting có giảm dao động verdict không?	81
RQ4: Hệ thống hoạt động ra sao trên dự án thực?.....	82
5.2. Đóng góp của luận văn	83
5.3. Bàn luận	84
Hai cách lấy mẫu, hai con số	84
Chi phí của agentic	84
Bản chất non-deterministic của LLM judge	84
Path-sensitive: heuristic CFG chưa đủ chính xác	85
5.4. Hạn chế của nghiên cứu.....	85
5.5. Hướng phát triển trong tương lai	86
Tài liệu tham khảo	88
A. Công cụ phân tích tĩnh	88
B. Công cụ phân tích động	88
C. LLM cho phát hiện lỗi	89
D. Leak C/C++ trực tiếp (baseline chính)	89
E. Agentic / Multi-agent cho SE.....	90
F. LLM Foundations.....	90
G. Benchmarks & Datasets.....	91
H. Giao thức & Tiêu chuẩn.....	92
I. Kiến trúc & hiệu năng phân tích tĩnh quy mô lớn.....	92
Phụ lục A: Bảng kết quả chi tiết	93
A.1. Full confusion matrices — 9-baseline ablation (Juliet n=50, stratified).....	93
A.2. Full confusion matrices — n=100 stratified	93
A.3. 2×2 ablation (Juliet n=30).....	93
A.4. Static evidence-tool ablation (B1, n=50)	94

A.5. LAMeD — bộ run 2026-08-20 trên mẫu đã sửa (50 site)	94
A.6. Consensus ablation — verdict stability	95
A.6.1. Kết quả chôt: n=50 stratified (kết quả đảo chiều).....	95
A.6.2. Thí nghiệm ban đầu n=30 (giữ làm dữ liệu lịch sử có chú thích).....	95
A.7. LLM variance (multi-run, 3 runs).....	95
A.8. MemHint (19 ca / 6 dự án thực).....	96
Phụ lục B: Prompt templates (verbatim)	98
B.1. Static sub-agent — system prompt	98
B.2. Static sub-agent — user message.....	98
B.3. Dynamic worker — system prompt	98
B.4. LLM Judge — system prompt	99
B.5. LLM Judge — user message template	100
B.6. Allocator profiler — system prompt (tóm tắt)	100
B.7. Completion nudges	100
Phụ lục C: Cài đặt và hướng dẫn chạy.....	101
C.1. Yêu cầu hệ thống.....	101
C.2. Cài đặt	101
C.3. Cấu hình LLM.....	101
C.4. Chạy đánh giá.....	102
C.5. Cấu hình nâng cao.....	102
C.6. Cấu trúc output.....	102
Phụ lục D: Bảng tham chiếu tool MCP	103
D.1. Static analyzer — 11 tools	103
D.2. Dynamic analyzer — 9 tools	103
D.3. Tool domain.....	104

D.4. Tool execution policy	104
D.5. MCP protocol.....	104

Danh mục hình vẽ

Hình 2.1: Kiến trúc tổng quan hệ thống CLeak (5 tầng: Source, Orchestrator, Analyzers MCP, Judge, Output)

Hình 2.2: Sequence tổng quan luồng vận hành của hệ thống

Hình 3.1: Topology triển khai hệ thống (orchestrator và hai analyzer MCP)

Hình 3.2: Pipeline HYBRID bốn tầng của orchestrator

Hình 3.3: Vòng lặp tool-calling native của agent-core trong TUI

Hình 3.4: Mô hình dữ liệu LeakBundle và các cấu trúc liên quan

Chương 1: Các nghiên cứu và công nghệ liên quan

Trước khi trình bày thiết kế và kết quả của hệ thống, cần đặt nghiên cứu này vào bối cảnh học thuật rộng hơn. Chương này khảo sát các hướng tiếp cận đã có — từ công cụ phân tích tĩnh và động truyền thống, cho đến các hệ thống dùng mô hình ngôn ngữ lớn (LLM) để phát hiện lỗi — nhằm chỉ ra khoảng trống mà luận văn nhắm lấp đầy.

1.1. Memory leak trong C/C++

1.1.1. Định nghĩa và phân loại

Memory leak xảy ra khi chương trình cấp phát bộ nhớ nhưng không giải phóng sau khi sử dụng xong, khiến vùng nhớ đó không thể tái sử dụng [37]. Trong C/C++, nơi quản lý bộ nhớ là trách nhiệm của lập trình viên, loại lỗi này đặc biệt phổ biến. MITRE phân loại memory leak thuộc CWE-401 ("Missing Release of Memory after Effective Lifetime"), một biến thể con của CWE-772 ("Missing Release of Resource after Effective Lifetime") [37].

Cần phân biệt memory leak với các lỗi liên quan: use-after-free (truy cập bộ nhớ đã giải phóng, CWE-416), double-free (giải phóng hai lần, CWE-415), và buffer overflow (tràn vùng đệm, CWE-120). Dù đều thuộc nhóm memory safety, cơ chế phát hiện và sửa chữa hoàn toàn khác nhau.

1.1.2. Patterns phổ biến

Qua phân tích benchmark Juliet [37] và các CVE thực tế, có thể tóm tắt thành sáu patterns chính:

Missing free đơn giản. Bộ nhớ được cấp phát nhưng không có lệnh giải phóng nào trong hàm. Đây là pattern dễ phát hiện nhất, thường do lập trình viên quên gọi `free()`.

Path-sensitive leak. Bộ nhớ được giải phóng trên một số đường thực thi nhưng không trên đường khác. Ví dụ kinh điển từ trang chính thức CWE-401 [37]:

```
char* getBlock(int fd) {  
    char* buf = (char*) malloc(BLOCK_SIZE);  
    if (!buf) { return NULL; }  
    if (read(fd, buf, BLOCK_SIZE) != BLOCK_SIZE) {
```



```

        return NULL; // buf bị rò rỉ ở đây
    }
    return buf;
}

```

Pattern này đòi hỏi phân tích nhạy cảm đường đi (path-sensitive analysis) — một thách thức lớn đối với cả công cụ tĩnh lẫn động.

Interprocedural leak. Bộ nhớ được cấp phát ở hàm A, truyền cho hàm B, nhưng không hàm nào giải phóng. Việc theo dõi dòng dữ liệu qua biên hàm phức tạp hơn nhiều so với phân tích trong một hàm.

Factory-allocator leak. Các hàm factory như `cJSON_CreateObject()`, `TIFFOpen()` trả về con trỏ đã cấp phát mà không có từ khóa `malloc/alloc` trong tên. Công cụ tĩnh dựa trên pattern matching sẽ bỏ sót nếu không biết tên các hàm này.

Parameter-ownership leak. Hàm nhận con trỏ tham số, giải phóng trên một số đường nhưng không trên đường khác. Ví dụ: hàm `cJSON_merge_patch` giải phóng `target` trên đường thành công nhưng quên trên đường lỗi — leak thực tế đầu tiên mà hệ thống trong luận văn này bắt được.

Overwritten pointer. Con trỏ được gán lại mà không giải phóng vùng nhớ cũ, làm mất tham chiếu đến vùng nhớ đã cấp phát.

1.1.3. Tác động thực tế

Memory leak không chỉ là vấn đề lý thuyết. Các CVE dưới đây cho thấy hậu quả thực tế:

- **CVE-2024-2398** (curl): Rò rỉ bộ nhớ trong xử lý HTTP/2 push headers, ảnh hưởng phiên bản 7.44.0 đến 8.6.0. Kẻ tấn công có thể khai thác để gây từ chối dịch vụ.
- **CVE-2016-0799** (OpenSSL): Leak trong `BN_hex2bn()` và `BN_dec2bn()`, cho phép gây DoS qua tải tài nguyên cạn kiệt.
- **CVE-2023-0461** (Linux kernel): Leak trong garbage collector của `af_unix`, ảnh hưởng hệ thống socket.
- **CVE-2023-32233** (Linux kernel): Use-after-free kết hợp leak trong Netfilter `nf_tables`, dẫn đến leo thang đặc quyền.

Điểm chung: leak trong các dự án lớn thường không phải lỗi logic đơn giản, mà là hệ quả của tương tác phức tạp giữa nhiều hàm, nhiều nhánh điều kiện, và quy tắc sở hữu không rõ ràng.

1.1.4. Độ phức tạp của bài toán

Phát hiện memory leak chính xác đòi hỏi giải quyết ba bài toán con: (1) xác định vùng nhớ nào được cấp phát — khó khi allocator là hàm factory chứ không phải `malloc`; (2) xác định vùng nhớ nào được giải phóng — khó khi free nằm ở hàm khác hoặc trên một nhánh cụ thể; (3) xác định đường đi nào khả thi — khó vì đòi hỏi phân tích điều kiện nhánh. Đây chính là lý do không công cụ đơn lẻ nào đạt recall hoàn hảo mà vẫn giữ FP thấp.

1.2. Phân tích tĩnh cho memory leak

Phân tích tĩnh kiểm tra mã nguồn mà không thực thi chương trình, có khả năng bao phủ mọi đường đi lý thuyết. Tuy nhiên, chính ưu điểm này cũng là điểm yếu: phân tích quá thận trọng sẽ tạo ra nhiều cảnh báo sai (false positive).

1.2.1. Clang Static Analyzer

Clang Static Analyzer [1] sử dụng kỹ thuật symbolic execution trên CFG (Control Flow Graph) để theo dõi trạng thái bộ nhớ. Công cụ này cài đặt các checker chuyên biệt cho từng loại lỗi: `unix.Malloc` cho leak C, `cplusplus.NewDelete` cho leak C++. Ưu điểm lớn nhất là tích hợp sẵn trong toolchain Clang, không cần cài đặt thêm.

Tuy nhiên, Clang SA có giới hạn rõ rệt. Trên benchmark LAMeD [21] gồm 43 leak từ 7 dự án thực, Clang SA phát hiện được 0 leak — `unix.Malloc` không mô hình hoá được factory allocator và interprocedural ownership. Trên Juliet CWE-401 (corpus dễ hơn), Clang đạt F1 khoảng 0.76 với 12 false positive [1]. Con số này cho thấy khoảng cách lớn giữa công cụ tĩnh truyền thống và bài toán thực tế.

1.2.2. Facebook Infer

Infer [2] dựa trên lý thuyết separation logic và kỹ thuật biabduction để suy luận trạng thái bộ nhớ qua biên hàm. Điểm mạnh là khả năng phân tích interprocedural mà không cần phân tích lại toàn bộ caller khi callee thay đổi — rất phù hợp cho CI/CD.

Infer hoạt động tốt trên các pattern đơn giản (null pointer, resource leak) nhưng gặp khó với leak phức tạp đòi hỏi theo dõi nhiều biến cùng lúc. Trên benchmark LAMeD [21], Infer chỉ phát hiện 3 leak trong khi CodeQL tìm được 19.

1.2.3. CodeQL

CodeQL [3] dùng Datalog để truy vấn quan hệ giữa các phần tử code. Đối với CWE-401, các query tiêu chuẩn theo dõi dòng dữ liệu từ `malloc/new` đến điểm sử dụng cuối cùng, kiểm tra xem có đường nào dẫn đến `free()` hay không.

CodeQL mạnh ở tính linh hoạt — người dùng có thể viết query tùy chỉnh — nhưng đòi hỏi chuyên môn cao. Trên LAMeD benchmark, CodeQL tìm được 5–10 leak (khi có annotation từ LLM [21]) nhưng tạo ra 139–653 warnings, tỉ lệ FP rất cao.

1.2.4. Tree-sitter

Tree-sitter [4] là bộ phân tích cú pháp tăng trưởng (incremental parser) được thiết kế cho các editor và công cụ phân tích code. Khác với Clang (cần compile đầy đủ), Tree-sitter parse trực tiếp mã nguồn thành AST, nhanh và nhẹ hơn nhiều.

Trong luận văn này, Tree-sitter đóng vai trò backbone cho phân tích tĩnh: xác định phạm vi hàm, phân loại kiểu cấp phát, xây dựng CFG cơ bản. Lựa chọn này cho phép phân tích code C/C++ mà không cần compile — quan trọng khi làm việc với các dự án chưa biết cách build.

1.2.5. NESA — phân tích tĩnh neuro-symbolic

NESA [5] (Neuro-Symbolic Static Analysis) kết hợp LLM với phân tích tĩnh dựa trên Datalog hạn chế. Hệ thống chia bài toán thành các bài toán con đơn giản hơn, dùng LLM để suy luận từng bước, giảm hallucination qua prompting tăng dần (lazy and incremental prompting).

Kết quả trên TaintBench: precision 66.27%, recall 78.57%, F1 0.72 — vượt phương pháp công nghiệp 0.20 điểm F1. Đặc biệt, NESA phát hiện 13 leak thực đã được nhà phát triển sửa. Đây là minh chứng sớm cho hướng neuro-symbolic trong phân tích memory safety.

1.2.6. CodeChecker và khả năng mở rộng

Horvath và cộng sự [6] nghiên cứu cách mở rộng Clang Static Analyzer cho codebase lớn qua framework CodeChecker. Các kỹ thuật bao gồm: phân tích tăng trưởng (chỉ phân tích phần code thay đổi), ức chế FP tự động, tích hợp CI/CD, và khai thác kết quả qua pattern discovery. Nghiên cứu cho thấy các kỹ thuật này tương tác và bổ trợ lẫn nhau, tạo ra hệ thống lớn hơn tổng các phần.

1.2.7. Abstract interpretation và symbolic execution

Bên cạnh các công cụ cụ thể, cần nhắc đến hai nền tảng lý thuyết đứng sau phân tích tĩnh: abstract interpretation và symbolic execution.

Abstract interpretation — khởi nguồn từ công trình của Cousot và Cousot (1977) — cho phép suy luận tính chất chương trình bằng cách "trừu tượng hoá" giá trị biến thành các domain đơn giản hơn (ví dụ: interval, sign, parity). Frama-C/Eva và Astrée là hai analyzer tiêu biểu dùng kỹ thuật này cho C; Goblint chuyên về phân tích concurrency. Ưu điểm: soundness có thể chứng minh (mọi bug trong abstract domain đều có bug trong concrete domain). Nhược điểm: over-approximation tạo FP — lý do chính khiến abstract interpretation hiếm khi dùng trực tiếp cho leak detection mà thường phục vụ cho chứng minh absence of bug.

Symbolic execution (King, 1976; Cadar et al., KLEE 2008) chạy chương trình với giá trị biểu tượng thay vì concrete, khám phá nhiều đường đi bằng cách fork tại mỗi branch. Clang SA [1] chính là một partial symbolic executor trên CFG. Điểm mạnh: precision cao hơn abstract interpretation. Điểm yếu: path explosion — số đường tăng exponential với số branch. KLEE và các công cụ kế nhiệm (Manticore, Angr) dùng heuristic search và constraint solving (SMT) để kiểm soát explosion.

Trong ngữ cảnh memory leak, abstract interpretation thường quá thận trọng (mọi cấp phát đều có thể leak → FP cao), còn symbolic execution tốn kém để chạy trên codebase lớn. Cách tiếp cận trong luận văn nằm ở giữa: dùng Tree-sitter [4] parse AST nhẹ (không symbolic), kết hợp guard-subset reconciliation để đạt một phần path-sensitivity mà không cần SMT — đổi precision lấy scalability.

1.2.8. Kiến trúc quy mô công nghiệp — tiền lệ mà CLeak áp dụng

Ngoài năng lực phát hiện leak, kiến trúc engine của CLeak được thiết kế theo đúng các nguyên tắc mà nền tảng phân tích quy mô công nghiệp đã dùng để đạt tốc độ, độc lập với việc chọn ngôn ngữ lập trình:

- **Compositional summary (Infer).** Việc cache kết quả parse theo hash nội dung file (Chương 3) thay vì tính lại mỗi lần gọi phản ánh đúng nguyên lý compositional analysis của Infer: mỗi hàm chỉ được tóm tắt một lần, caller tái sử dụng summary thay vì suy luận lại từ đầu — đây là lý do Infer scale được tới hàng triệu dòng code tại Meta [46].

- **Shardable, stateless services (Tricorder).** Việc chỉ 5/22 MCP tool được cấp trực tiếp cho LLM sub-agent, và các tool này nhận nội dung source inline thay vì yêu cầu filesystem dùng chung, giữ cho cả hai service phân tích ở trạng thái stateless và triển khai độc lập — cùng nguyên lý kiến trúc microservice có thể chia nhỏ (shardable) mà Google Tricorder dùng để chạy hàng trăm analyzer trên hàng trăm nghìn thay đổi code mỗi ngày [45]. Nguyên lý này cũng lý giải cho việc engine tắt định chạy trước, LLM chỉ được gọi cho các case biên (Chương 4) — một biến thể của kỷ luật kiểm soát tỉ lệ false-positive liên tục mà Tricorder áp dụng.
- **So khớp trực tiếp trên AST, không qua database trung gian (Semgrep).** Semgrep đạt tốc độ bằng cách match pattern trực tiếp trên AST đã parse, không xây database trung gian như CodeQL [44] — CLeak dùng cùng nguyên lý này thay vì cách tiếp cận database-query của CodeQL.
- **Tái sử dụng kết quả build qua các lần chạy lặp lại (CodeQL incremental).** Ở tầng dynamic-analyzer, việc build lại project cho cùng một commit (phổ biến trên corpus như LAMeD: 41 case nhưng chỉ 7 repo khác nhau) được tránh nhờ compiler cache — cùng nguyên lý incremental reuse mà CodeQL đã áp dụng trong production để không phân tích lại phần code không đổi [47].

1.3. Phân tích động cho memory leak

Phân tích động thực thi chương trình và quan sát hành vi thực tế. Ưu điểm là FP thấp (phát hiện leak trên đường thực sự chạy), nhưng nhược điểm là coverage phụ thuộc vào test case.

1.3.1. Valgrind Memcheck

Valgrind [7] sử dụng kỹ thuật dynamic binary instrumentation, chèn mã kiểm tra vào binary khi thực thi. Memcheck — tool mặc định — theo dõi mọi lần cấp phát và giải phóng, báo cáo leak khi chương trình kết thúc.

Valgrind đáng tin cậy nhưng rất chậm (10–50× overhead). Trên Linux, đây là tiêu chuẩn de facto cho kiểm tra memory leak. Tuy nhiên, Valgrind chỉ phát hiện leak trên đường thực thi thực sự — nếu test case không chạy đến đường lỗi, leak trên đường đó sẽ bị bỏ sót.

1.3.2. AddressSanitizer và LeakSanitizer

ASan [8] chèn mã kiểm tra tại compile-time, sử dụng "redzones" xung quanh vùng cấp phát để phát hiện buffer overflow, use-after-free, và double-free. LSan tích hợp trong ASan, kiểm tra leak tại thời điểm chương trình kết thúc.

ASan nhanh hơn Valgrind đáng kể (khoảng $2\times$ slowdown) nhưng cần compile lại với flag `-fsanitize=address`. LSan đặc biệt hữu ích vì báo cáo chính xác vùng nhớ nào bị rò rỉ và stack trace dẫn đến lệnh cấp phát.

Một lưu ý kỹ thuật quan trọng: ASan/LSan dự trữ khoảng 20 TB địa chỉ ảo, nên cần bỏ giới hạn `ulimit -v`. Không có `llvm-symbolizer`, frame leak sẽ không có `file:line` và không thể tương quan với candidate tĩnh.

1.3.3. RangeSanitizer

Gorter và Giuffrida [9] đề xuất RSan, một sanitizer mới sử dụng redzone kết hợp với per-object metadata và pointer tagging. RSan đạt overhead trung bình 44% trên SPEC CPU2017 — nhanh gấp đôi ASan — đồng thời cải thiện throughput fuzzing lên đến 70% khi kết hợp với AFL++.

Đây là bước tiến đáng kể: sanitizer nhanh hơn đồng nghĩa với khả năng chạy nhiều test case hơn trong cùng thời gian, cải thiện coverage.

1.3.4. QMSan

Marini và cộng sự [10] phát triển QMSan để phát hiện lỗi sử dụng bộ nhớ chưa khởi tạo (use-of-uninitialized-memory). Hệ thống kết hợp QEMU dynamic binary translation với cơ chế học lọc FP qua các lần fuzzing. Kết quả: phát hiện 44 lỗi mới trong 10 dự án open-source và 5 phần mềm thương mại, với overhead $1.51\times$, zero FP và zero FN.

Điểm đáng chú ý là QMSan không cần compile lại toàn bộ thư viện — một lợi thế thực tế khi phân tích dự án lớn với dependency phức tạp.

1.3.5. CombiSan

Marini và cộng sự [11] tiếp tục đề xuất CombiSan, thống nhất nhiều sanitizer (ASan, MSan, UBSan) thành một runtime duy nhất. Hệ thống duy trì "violation map" để đạt độ chính xác tương đương chạy từng sanitizer riêng lẻ, nhưng chỉ cần một lần fuzzing pass thay vì nhiều lần.

1.3.6. AirTaint — taint analysis nhanh hơn

Sang và cộng sự [12] đề xuất AirTaint, kết hợp basic-block-level taint rules với assembly-code-level instrumentation. Hệ thống đạt tốc độ nhanh hơn 931 lần so với libdft, 5.97 lần so với SelectiveTaint, và 328.3 lần so với TaintRabbit. AirTaint phát hiện tất cả 14 CVE trong 9 ứng dụng thực.

1.3.7. Fuzzers vs Static Analyzers — hai thế giới gần như không giao nhau

Hassler và cộng sự [13] thực hiện nghiên cứu so sánh thực nghiệm giữa 5 static analyzer và 13 fuzzer trên hơn 100 lỗ hổng C/C++ đã biết. Phát hiện quan trọng: fuzzers tìm được tập bug gần giống nhau (nhiều tool cùng tìm 1 bug), trong khi static analyzer tìm được tập bug khác biệt hơn. Đáng chú ý, tập bug mà fuzzers tìm được và tập bug mà static analyzer tìm được **gần như không giao nhau**.

Kết luận này biện minh trực tiếp cho kiến trúc hybrid trong luận văn: kết hợp static + dynamic để bao phủ nhiều loại leak hơn bất kỳ phương pháp đơn lẻ nào.

1.4. LLM cho phát hiện lỗi phần mềm

Sự xuất hiện của các mô hình ngôn ngữ lớn (LLM) như GPT-4, Claude, và các model open-source đã mở ra hướng tiếp cận mới cho phát hiện lỗi: thay vì dựa hoàn toàn vào rule cứng, LLM có thể "đọc" code, hiểu ngữ nghĩa, và suy luận về khả năng xảy ra lỗi.

1.4.1. Khả năng hiểu code của LLM

Khare và cộng sự [15] đánh giá 16 pre-trained LLM trên 5.000 mẫu code từ 5 dataset khác nhau, bao gồm cả Java và C/C++, trải rộng 25 loại lỗ hổng. Kết quả trung bình: accuracy 62.8%, F1 0.71. LLM hoạt động tốt hơn trên các lỗ hổng chỉ cần phân tích trong một hàm (intra-procedural), kém hơn trên lỗ hổng cần theo dõi qua nhiều hàm.

Một phát hiện đáng chú ý: kỹ thuật prompting "step-by-step analysis" cải thiện F1 lên đến 0.18 điểm. Điều này gợi ý rằng cách đặt câu hỏi cho LLM quan trọng không kém năng lực của model.

1.4.2. LLM kết hợp công cụ: ReAct và tool-augmented reasoning

Trước khi bàn đến các hệ thống cụ thể, cần hiểu paradigm đứng sau: làm thế nào LLM sử dụng công cụ bên ngoài?

Yao và cộng sự [28] đề xuất ReAct (Reasoning + Acting), trong đó LLM xen kẽ reasoning traces (suy luận về bước tiếp theo) và actions (gọi công cụ bên ngoài như search API, code interpreter). Khác với chain-of-thought thuần (chỉ reasoning), ReAct cho phép model "hành động" — đọc kết quả thực từ môi trường rồi điều chỉnh suy luận. Trên HotpotQA và Fever, ReAct vượt imitation learning và RL 10–34% absolute success rate.

ReAct là nền tảng cho mọi hệ thống agentic trong phần sau: RepoAudit [25], FuzzingBrain V2 [26], SAILOR [24] đều interleaving reasoning và tool-calling. Trong luận văn, orchestrator dùng native tool-calling (tool_use/tool_result) thay vì ReAct text parsing — nhưng nguyên tắc "suy luận → hành động → quan sát → suy luận lại" giống hệt.

1.4.3. IRIS — LLM kết hợp static analysis

Li và cộng sự [16] xây dựng IRIS, hệ thống kết hợp LLM với static analysis để phát hiện lỗ hổng bảo mật trên toàn repository. LLM suy luận taint specification (nguồn và đích của dòng dữ liệu bất an toàn) mà không cần người viết specification thủ công.

Trên CWE-Bench-Java (120 lỗ hổng đã xác minh), CodeQL phát hiện 27 lỗ hổng; IRIS với GPT-4 phát hiện 55 — tăng 28 lỗ hổng. Đặc biệt, IRIS tìm được 4 lỗ hổng mới mà không công cụ nào khác phát hiện. Đây là minh chứng mạnh mẽ cho hướng LLM-augmented static analysis.

1.4.4. Prompting như proxy cho static analysis

Ceka và cộng sự [17] đặt câu hỏi táo bạo: liệu LLM prompting có thể thay thế static analysis không? Họ đề xuất kỹ thuật kết hợp hướng dẫn vulnerability bằng ngôn ngữ tự nhiên với chain-of-thought reasoning sử dụng mẫu đối chiếu (contrastive samples).

Kết quả: security-aware prompting vượt baseline static analysis, cải thiện accuracy lên 31.6%, F1 lên 71.7%, và giảm false negative rate 37.6%. Nghiên cứu này

cho thấy prompting chiến lược có thể khai thác khả năng reasoning của LLM mà không cần fine-tuning.

1.4.5. VulnLLM-R — LLM chuyên biệt cho vulnerability detection

Nie và cộng sự [18] huấn luyện model 7B chuyên biệt cho vulnerability detection, nhấn mạnh reasoning về trạng thái chương trình. Quy trình bao gồm chọn lọc dữ liệu, tạo dữ liệu reasoning, và tối ưu hoá test-time.

VulnLLM-R vượt cả CodeQL và AFL++ trên các dự án thực, phát hiện zero-day trong các repository đang hoạt động. Điều đáng chú ý là model nhỏ (7B parameter) nhưng được huấn luyện đúng cách có thể cạnh tranh với model lớn hơn nhiều.

1.4.6. SemTaint — multi-agent taint specification

Ghebremichael và cộng sự [19] xây dựng SemTaint, hệ thống multi-agent kết hợp LLM với static analysis để trích xuất taint specification. Hệ thống sử dụng static analysis để tính call graph, sau đó giao cho LLM phân loại source, sink, và xác định các call edge không giải quyết được.

Tích hợp với CodeQL, SemTaint phát hiện 106 trong 162 lỗ hổng mà CodeQL không thể phát hiện, và tìm được 4 lỗ hổng mới trong các npm package phổ biến. Dù nghiên cứu tập trung vào JavaScript, kỹ thuật này có thể áp dụng cho C/C++.

1.4.7. MemHint — neuro-symbolic cho memory leak

Huang và cộng sự [20] xây dựng MemHint, hệ thống kết hợp LLM với Z3 SMT solver để phát hiện memory leak trong C/C++. Pipeline gồm ba bước: (1) LLM phân loại hàm thành allocator, deallocator, hoặc neither; (2) Z3 xác minh tính khả thi của leak path dựa trên CFG; (3) LLM xác nhận kết quả cuối cùng.

Trên 7 dự án thực (3.4M+ SLOC), MemHint tự báo 52–54 leak tìm thấy (49 đã được xác nhận và sửa, kèm 4 CVE); cùng thiết lập đó, CodeQL tìm 19 và Infer tìm 3. Đây là kết quả ấn tượng, nhưng MemHint chưa được peer-review (arXiv preprint).

1.4.8. LAMeD — LLM annotation cho analyzer cổ điển

Shemetova và cộng sự [21] đề xuất hướng tiếp cận khác: thay vì dùng LLM trực tiếp phát hiện lỗi, dùng LLM để tạo annotation cho analyzer cổ điển. LLM sinh

metadata về hàm nào là allocator, hàm nào là deallocator, sau đó feed vào CodeQL, Infer, hoặc Coody.

Đây là baseline peer-reviewed duy nhất (EASE 2025, CORE-A) cho leak C/C++. Trên benchmark cJSON, LAMeD đạt $P=0.933$, $R=0.583$ (28 TP, 2 FP, 20 FN). Kết quả cho thấy đánh đổi kinh điển: recall tăng thì FP cũng tăng.

1.4.9. Revelio — agentic với sanitizer proof

Hou và cộng sự [23] xây dựng Revelio, hệ thống agentic phát hiện memory safety vulnerability ở quy mô repository. Điểm độc đáo: Revelio chỉ báo cáo lỗi hổng có thể tái hiện bằng sanitizer — giảm hallucination bằng cách yêu cầu "bằng chứng thực thi" (executable proof-of-vulnerability).

Trên 7 dự án production (đã fuzz 5–8 năm), Revelio phát hiện 19 lỗi hổng mới, tổng chi phí \$300. Hệ thống vượt các frontier coding agent ở cùng chi phí token.

1.4.10. SAILOR — symbolic execution với LLM

Shafiuzzaman và cộng sự [24] kết hợp static analysis với LLM để tự động xây dựng harness cho symbolic execution. Ba giai đoạn: static analysis → LLM orchestration (với iterative refinement) → concrete replay xác nhận.

Trên 10 dự án C/C++ (6.8M dòng code), SAILOR phát hiện 379 lỗi hổng memory safety mới, xác nhận 421 crash. Baseline mạnh nhất (Claude Code với agentic vulnerability detection) chỉ tìm được 12. Khi bỏ static analysis, số lỗi hổng giảm 12.2 lần; khi bỏ iterative LLM synthesis, giảm xuống 0.

1.4.11. Hệ thống multi-agent cho software engineering

Bên cạnh các hệ thống đơn agent, một hướng nghiên cứu khác là phối hợp nhiều agent cùng làm việc. Guo và cộng sự [33] tổng hợp toàn cảnh: từ ChatDev (mô phỏng công ty phần mềm với agent CEO/CTO/programmer/tester), MetaGPT (SOP encoded vào prompt, assembly-line paradigm), đến Mixture-of-Agents (layered architecture, mỗi layer tổng hợp output từ layer trước).

Trong lĩnh vực vulnerability discovery, hai hệ thống nổi bật:

RepoAudit [25] (Guo et al., ICML 2025 poster) là agentic code auditor theo dõi data-flow qua nhiều file trong repository. Agent tự khám phá codebase, sinh audit hypothesis, rồi validate finding. Trên 15 dự án, RepoAudit đạt precision 78.43% (40

TP/11 FP) với chi phí \$2.54/dự án. Điểm đáng học hỏi: validator module kiểm tra path-condition SAT để giảm FP — tương tự tăng judge trong luận văn.

FuzzingBrain V2 [26] (Sheng et al., 2026) là multi-agent system trên MCP gần nhất với kiến trúc luận văn: static analysis (Fuzz Introspector) + dynamic (libFuzzer + ASan) + Claude Opus/Sonnet/Haiku multi-agent. Trên AIXCC 2025 Final (40 vuln/12 dự án), đạt 90% detection rate và phát hiện 29 zero-day. Khác biệt then chốt: xác minh bằng crash sanitizer — leak chỉ là incidental (5 ca). Luận văn mở rộng sang non-crash leak (bằng chứng LSan/Valgrind, không cần crash).

ATLANTIS [27] (Team Atlanta, vô địch AIXCC 2025) là hệ thống CRS (Cyber Reasoning System) kết hợp multi-language fuzzer với LLM-based vulnerability analysis. Buttercup (Trail of Bits, hạng 2) dùng dynamic + multi-agent + MCP, open-source AGPL-3. Cả hai xác minh bằng crash → tương phản với lớp non-crash leak mà luận văn nhắm đến.

Hai hệ thống non-agentic cũng đáng chú ý: SWE-agent [34] (Yang et al., 2024) đạt 12.5% pass@1 trên SWE-bench nhờ thiết kế Agent-Computer Interface (ACI) tốt; Agentless [35] (Xia et al., 2024) đạt 32% trên SWE-bench với pipeline đơn giản 3 bước (localize → repair → validate) — không cần agent phức tạp mà vẫn vượt nhiều hệ thống agentic. Bài học: thiết kế pipeline rõ ràng quan trọng hơn mức độ "tự chủ" của agent.

1.5. Cơ chế đồng thuận và giảm phương sai LLM

1.5.1. Self-consistency decoding

Wang và cộng sự [29] đề xuất phương pháp self-consistency: thay vì lấy một đường reasoning (greedy decoding), lấy mẫu nhiều đường reasoning độc lập rồi chọn câu trả lời nhất quán nhất (majority vote). Trên GSM8K, phương pháp này cải thiện CoT +17.9%; trên SVAMP +11.0%.

Ý tưởng cốt lõi: bài toán phức tạp có thể giải bằng nhiều cách khác nhau, và các cách đúng thường hội tụ về cùng đáp án. Đây là nền tảng lý thuyết cho consensus judge trong luận văn.

1.5.2. LLM-as-judge: calibration và dao động

Zheng và cộng sự [36] chỉ ra rằng GPT-4-as-judge đạt hơn 80% đồng thuận với chuyên gia con người — tương đương tỉ lệ đồng thuận giữa hai người. Tuy nhiên, nghiên cứu cũng phát hiện các bias: position bias (thiên vị vị trí), verbosity bias (thiên vị câu dài hơn), và self-enhancement bias (model tự đánh giá mình cao hơn).

Trong ngữ cảnh memory leak, vấn đề lớn hơn là dao động run-to-run: cùng một prompt, cùng model, cùng temperature=0, nhưng verdict có thể khác nhau giữa các lần chạy do provider-side batching. Đây chính là vấn đề mà consensus judge trong luận văn nhắm giải quyết.

1.5.3. McNemar test cho paired comparison

Khi so sánh hai hệ thống trên cùng corpus, McNemar test là phương pháp thống kê phù hợp — kiểm tra xem tỉ lệ "A đúng, B sai" có khác biệt đáng kể so với "B đúng, A sai" hay không. Đây là test paired, mạnh hơn so sánh aggregate F1 đơn lẻ.

1.6. Benchmarks và datasets

1.6.1. Juliet Test Suite (NIST)

Juliet Test Suite v1.3 [37] là bộ test tiêu chuẩn của NIST cho C/C++, bao gồm 118 CWE. Riêng CWE-401 chứa khoảng 1.820 test cases (kết hợp biến thể bad và good). Mỗi file test chứa hàm `bad()` (leak) và một hoặc nhiều hàm `good1()`, `good2()` (không leak).

Mã hoá tên file phân loại biến thể: loại cấp phát (`malloc`, `calloc`, `realloc`, `new`, `strdup`), kiểu dữ liệu (`char`, `int`, `struct`, `wchar_t`), và số lượng phần tử. Đây là corpus tiêu chuẩn cho đánh giá static analysis tool.

Hạn chế lớn nhất: code Juliet là synthetic, pattern có công thức, không phản ánh độ phức tạp của dự án thực. Dù vậy, Juliet vẫn hữu ích cho ablation study vì ground-truth rõ ràng.

1.6.2. LAMeD Benchmark

LAMeD benchmark [38] (Zenodo 15089703) gồm 36 entry từ 7 dự án open-source thực: `curl` (14), `libtiff` (6), `libsolv` (6), `cjson` (6), `libxml2` (4), `libssh2` (3),

rabbitmq-c (2). Mỗi entry chứa link đến phiên bản lỗi, commit sửa lỗi, và file bị ảnh hưởng.

Đây là benchmark leak C/C++ thực tế duy nhất có ground-truth function-level. Hạn chế: chỉ có positive (leak), không có negative (code sạch), nên chỉ đánh giá được recall và FP count, không tính được specificity hay accuracy.

1.6.3. DiverseVul

DiverseVul [39] là dataset vulnerability lớn nhất cho C/C++ tại thời điểm công bố: 330.492 hàm (18.945 vulnerable), 150 CWE, từ 295+ dự án. Dữ liệu được crawl từ security issue websites, trích xuất vulnerability-fixing commits.

Nghiên cứu đánh giá 11 model thuộc 4 họ kiến trúc và kết luận: "deep learning vẫn chưa sẵn sàng cho vulnerability detection, do FP cao và F1 thấp." Tuy nhiên, LLM được đánh giá là "hướng nghiên cứu đầy hứa hẹn."

1.6.4. SV-COMP Memsafety

SV-COMP [29] là cuộc thi thường niên đánh giá công cụ verification trên C, bao gồm category memsafety với các thuộc tính: không leak, không buffer overflow, không null deref, không use-after-free. Chương trình có cả biến thể an toàn và không an toàn với kết quả mong đợi.

1.6.5. Magma

Magma [30] là benchmark fuzzing sử dụng ground-truth, chèn bug thật (bao gồm leak) vào chương trình thật (libpng, libtiff, SQLite, OpenSSL). Bug trigger từ CVE đã biết, cho phép đánh giá chính xác khả năng phát hiện của công cụ.

1.7. Tổng kết và vị trí nghiên cứu

Bảng sau tổng hợp các hệ thống liên quan và vị trí của luận văn:

Hệ thống	Static	Dynamic	Agentic	Judge	Leak focus	Peer-review
Clang SA [1]	✓	✗	✗	✗	✓	✓
Infer [2]	✓	✗	✗	✗	✓	✓
CodeQL [3]	✓	✗	✗	✗	✓	✓
MemHint [20]	✓	✗	✗	✓	✓	✗
LAMeD [21]	✓	✗	✗	●	✓	✓
Revelio [23]	✓	✓	✓	✓	●	✗
SAILOR [24]	✓	✓	✓	✗	●	✗
IRIS [15]	✓	✗	✗	✗	✗	✗

Hệ thống	Static	Dynamic	Agentic	Judge	Leak focus	Peer-review
Đề tài	✓	✓	✓	✓	✓	—

Khoảng trống rõ ràng: **chưa có hệ thống nào kết hợp LLM orchestration + static + dynamic chuyên cho memory leak C/C++**. MemHint và LAMeD chỉ dùng static; Revelio và SAILOR nhắm crash/vulnerability nói chung, không chuyên leak. Đồ thị Venn giữa "static + dynamic" và "memory leak focus" vẫn còn trống — đó chính là vị trí của luận văn này.

Xem danh sách tham khảo đầy đủ trong [paper/references/bibliography.md](#).

Chương 2: Lập kế hoạch, phân tích và thiết kế hệ thống

Khi bắt đầu dự án, chúng tôi đối mặt một câu hỏi cơ bản: nếu đã có Clang Static Analyzer, Valgrind, và CodeQL — mỗi công cụ đều đã được kiểm chứng qua nhiều năm — thì tại sao cần xây dựng thêm một hệ thống mới? Câu trả lời nằm ở chỗ không công cụ đơn lẻ nào giải quyết được bài toán đầy đủ: phân tích tĩnh cho nhiều false positive, phân tích động cho coverage thấp, và cả hai đều thiếu khả năng suy luận ngữ nghĩa cấp cao mà LLM có thể cung cấp. Chương này trình bày cách chúng tôi thiết kế hệ thống để kết hợp thế mạnh của từng phương pháp, đồng thời giảm thiểu nhược điểm cố hữu của mỗi phương pháp.

2.1. Yêu cầu và ràng buộc

2.1.1. Yêu cầu chức năng

Hệ thống phải đáp ứng bốn yêu cầu cốt lõi: (1) phát hiện memory leak trong code C/C++ với recall cao mà FP thấp; (2) phân loại mức độ nghiêm trọng (confirmed, likely, uncertain, false positive); (3) cung cấp giải thích và gợi ý sửa chữa cho mỗi finding; (4) hoạt động trên cả corpus synthetic (Juliet) lẫn dự án thực (curl, cJSON, libtiff) mà không cần thay đổi code.

2.1.2. Yêu cầu phi chức năng

Tái lập được. Cùng một input phải cho cùng một output — ít nhất ở chế độ deterministic. Điều này đặc biệt quan trọng cho luận văn: một kết quả không thể tái lập không có giá trị học thuật.

Mở rộng được. Thêm project mới phải là việc cấu hình, không phải viết code mới. Hệ thống phải tự khám phá API cấp phát/giải phóng của project thay vì hardcoded.

Chi phí hợp lý. LLM tốn token. Hệ thống cần chiến lược để giữ chi phí thấp mà không hy sinh chất lượng — ví dụ chỉ dùng LLM cho bundle "borderline" thay vì mọi bundle.

2.1.3. Ràng buộc kỹ thuật

MCP (Model Context Protocol) [6] là giao thức giao tiếp giữa orchestrator và analyzer — lựa chọn này gắn liền với thiết kế tool-calling native. Docker là môi

trường chạy analyzer (Valgrind chỉ hoạt động trên Linux). Multi-provider LLM là bắt buộc: hệ thống không nên phụ thuộc vào một nhà cung cấp model duy nhất.

2.2. Phân tích bài toán và lựa chọn kiến trúc

2.2.1. Tại sao hybrid — kết hợp static và dynamic

Phân tích tĩnh (Clang SA, CodeQL) bao phủ mọi đường đi lý thuyết nhưng không biết đường nào thực sự chạy được → FP cao. Phân tích động (Valgrind, LSan) chỉ phát hiện leak trên đường thực sự chạy → FN cao. Hassler và cộng sự [7] chỉ ra rằng tập bug mà fuzzers tìm được và tập bug mà static analyzer tìm được gần như không giao nhau.

Kết luận: cần cả hai. Nhưng "kết hợp" không có nghĩa là chạy cả hai rồi nối kết quả — cần một tầng hợp nhất thông minh, biết khi nào bằng chứng tĩnh đủ mạnh, khi nào cần xác nhận bằng bằng chứng động.

2.2.2. Tại sao LLM orchestration thay vì hardcoded pipeline

Mỗi dự án C/C++ có cách quản lý bộ nhớ riêng: libcurl dùng `curl_easy_cleanup()`, cJSON dùng `cJSON_Delete()`, libTIFF dùng `TIFFClose()`. Một pipeline hardcoded sẽ cần liệt kê tất cả các tên này cho mỗi project — không khả thi ở quy mô lớn.

LLM có thể đọc header/source và khám phá ra các API này, cùng với quy tắc ownership (ai sở hữu con trỏ sau khi hàm trả về?). Đây là tầng "POLICY" — quyết định theo-từng-project — trong khi tầng "MECHANISM" (parse AST, tính CFG, ghép cặp alloc→free, scoring) vẫn tất định.

2.2.3. Tại sao MCP thay vì gRPC/REST

MCP [6] cung cấp hai lợi thế: (1) tool description đi kèm schema, model thấy ngay tool nào khả dụng mà không cần document riêng; (2) giao thức chuẩn, analyzer có thể thay thế mà không ảnh hưởng orchestrator. Thực tế, dự án từng dùng gRPC nhưng đã loại bỏ sau khi chuyển sang TUI-only — MCP đủ cho mọi nhu cầu hiện tại.

2.3. Thiết kế tổng quan

2.3.1. Pipeline HYBRID

Hệ thống là một pipeline gồm bảy giai đoạn, chạy theo thứ tự nhưng một số giai đoạn có thể song song: (xem Hình 2.1)

```
profiling (LLM, optional)
  ↓
strategy (LLM, optional)
  ↓
discovery (deterministic)
  ↓
static enrichment (deterministic, optional)
  ↓
investigation (agentic – Stage A + B song song → C → D)
  ↓
judging (hybrid)
  ↓
reporting
```

Hai giai đoạn đầu (profiling, strategy) là tầng POLICY host-side, chạy trước khi pipeline chính bắt đầu. Chúng khám phá allocator set và chọn chiến lược phân tích cho project. Trong benchmark, tầng này bị bỏ qua (manifest cung cấp allocator đồng cứng) → eval không có LLM non-deterministic.

2.3.2. Nguyên tắc thiết kế: LLM sở hữu POLICY, engine sở hữu MECHANISM

Đây là nguyên tắc cốt lõi. Mọi quyết định thay đổi theo project (tên allocator, quy tắc ownership, chiến lược phân tích) do LLM khám phá. Mọi cơ chế phân tích (parse tree-sitter, tính CFG, ghép cặp alloc→free, scoring) do code tắt định thực thi. Output của LLM luôn được verify (grep tên trong source) và cache theo repo+commit.

Ranh giới MUST-STAY-CODE: parse tree-sitter, CFG construction, alloc→free pairing, scoring math, consensus logic, grep-verify. Những thứ này không bao giờ được LLM hoá.

Trong engine, kết quả parse được cache theo hash nội dung file thay vì tính lại ở mỗi lần gọi — content-addressed memoization là kỹ thuật chuẩn để tránh lặp lại công việc trong một lõi phân tích tắt định, cùng nguyên lý compositional summary mà Facebook Infer dùng để scale [46].

2.3.3. Sơ đồ thành phần

```
flowchart TB
    subgraph Client ["Client / Orchestrator"]
```

```

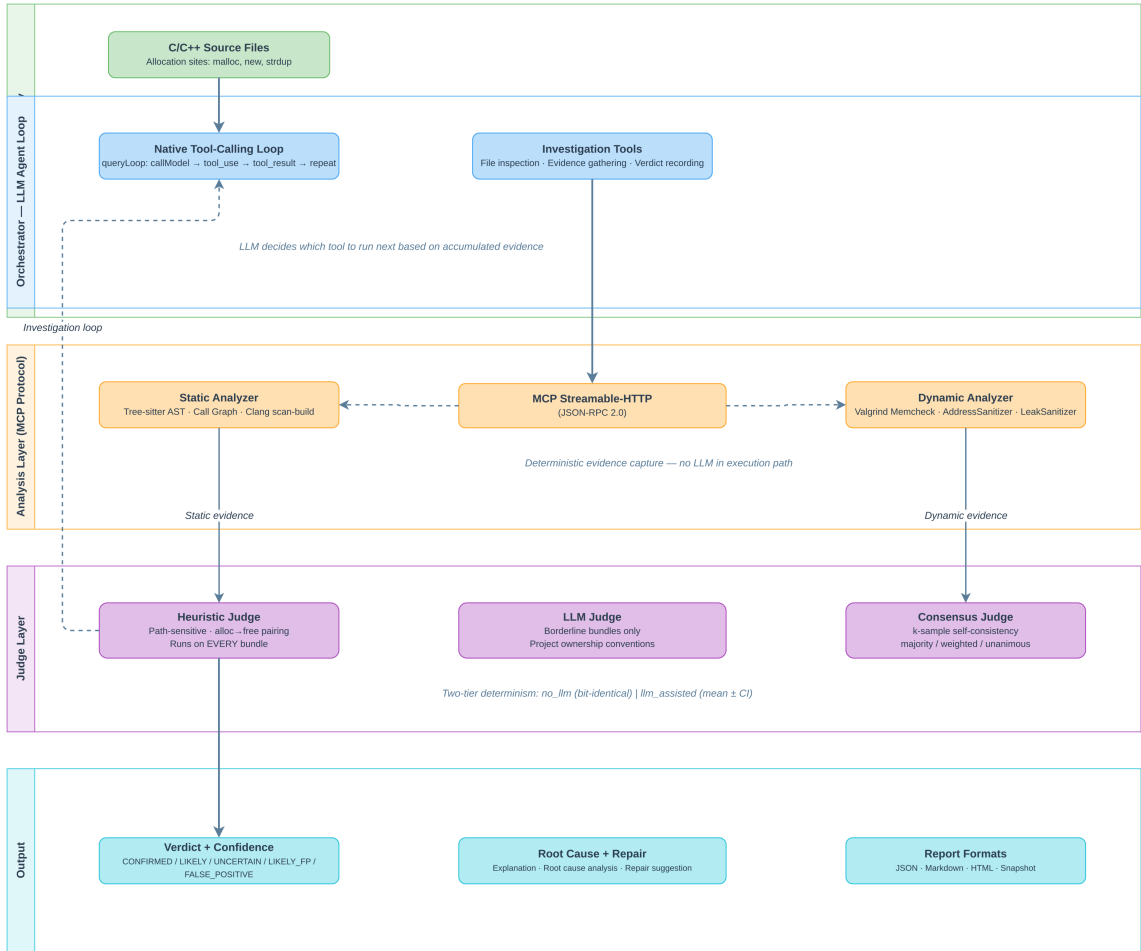
    TUI["leak-inspector-tui<br/>(Ink CLI, headless)"]
end

subgraph Analyzers["Analyzers (mcpvul-net)"]
    SA["static-analyzer<br/>MCP :50061"]
    DA["dynamic-analyzer<br/>MCP :50062"]
end

subgraph External["External"]
    LLM["LLM gateway :20128<br/>(mimo / OpenAI / Anthropic)"]
end

TUI -- "MCP Streamable-HTTP /mcp" --> SA
TUI -- "MCP Streamable-HTTP /mcp" --> DA
TUI -- "HTTP SSE (streaming)" --> LLM

```



Hình 2.1: Kiến trúc tổng quan hệ thống CLeak (5 tầng: Source, Orchestrator, Analyzers MCP, Judge, Output)

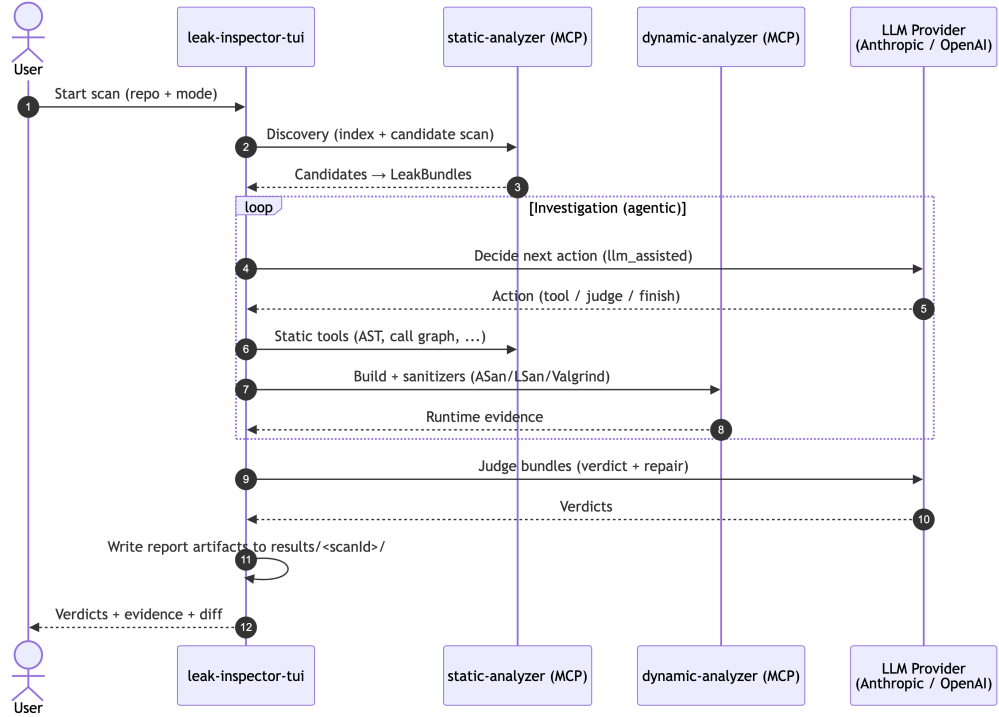
Hai analyzer nối qua Docker bridge network mcpvul-net. Orchestrator gọi analyzer qua MCP, gọi LLM qua HTTP SSE. Mỗi lời gọi tool là một HTTP POST độc lập (stateless JSON mode).

2.3.4. Luồng dữ liệu

```

flowchart LR
    C["LeakCandidate<br/>(allocation site)"] --> B["LeakBundle"]
    EV["Evidence<br/>(asan/lsan/valgrind/scan-build/heuristic)"] --> B
    B --> V["VerdictResult<br/>verdict · confidence · rootCause ·  
repairDiff"]
    V --> R["ScanReport<br/>(json/md/html/snapshot)"]
  
```

Dữ liệu đi qua bốn cấu trúc chính: (xem Hình 2.2)



Hình 2.2: Sequence tổng quan luồng vận hành của hệ thống

- **LeakCandidate**: vị trí cấp phát được phát hiện (file, line, function, allocation_type).
- **LeakBundle**: candidate + evidence tĩnh + evidence động + verdict. Đây là đơn vị xử lý chính.
- **VerdictResult**: verdict (confirmed/likely/uncertain/likely_FP/FP) + confidence + explanation + rootCause + repairDiff.
- **ScanReport**: tổng hợp mọi VerdictResult, output dạng JSON/Md/HTML/snapshot.

2.4. Thiết kế tầng khám phá tĩnh (Discovery)

2.4.1. Candidate scan

Hệ thống scan mỗi file C/C++ tìm các vị trí cấp phát. Không chỉ tìm `malloc/calloc/realloc/new` (libc), mà còn tìm factory allocator theo tên project — ví dụ `cJSON_New_Item()` trong `cJSON`, `TIFFmalloc()` trong `libTIFF`. Danh sách này được cung cấp bởi tầng LLM allocator profiler hoặc manifest đông cứng (trong benchmark).

Ngoài ra, hệ thống phát hiện "parameter-ownership leak": tham số con trỏ được giải phóng trên một số đường nhưng không trên đường khác. Các hàm nhận con trỏ và quản lý nó (như `cJSON_merge_patch` nhận `target`) được tạo candidate synthetically.

2.4.2. Attribution hàm bằng Tree-sitter

Khi một dòng cấp phát được tìm thấy, cần xác định nó thuộc hàm nào. Tree-sitter [4] cung cấp range-based AST traversal: tìm node function definition chứa dòng đó. Routing C/C++ dựa trên đuôi file: `.c/.h` dùng `tree-sitter-c`, `.cc/.cpp/.cxx/.hpp` dùng `tree-sitter-cpp`.

2.5. Thiết kế tầng làm giàu bằng chứng tĩnh (Static Enrichment)

2.5.1. Function summary

Mỗi candidate được phân tích bởi `functionSummary`: đếm lệnh cấp phát và giải phóng trong hàm bao, liệt kê exit paths (return), và xác định biến nào được giải phóng trên đường nào. Kết quả là `AllocFreePair` với status `paired` (giải phóng trên mọi đường), `unpaired` (không giải phóng đâu cả), hoặc `conditional` (giải phóng trên một số đường).

2.5.2. Path constraints

`pathConstraints` phân tích guard conditions (if/switch) xung quanh candidate. Guard-subset reconciliation: nếu lệnh free nằm trên nhánh return khác, nó không reconcile exit hiện tại — tức free trên đường thành công không "giải oan" cho đường lỗi.

Đây là cơ chế path-sensitive mà không cần SMT solver. Kết quả: `feasibleLeakPaths` — danh sách đường đi khả thi mà leak có thể xảy ra, kèm `unreconciledAllocations` — biến cấp phát mà không có free nào trên đường đó.

2.5.3. Interprocedural flow

`interproceduralFlow` mở rộng phân tích qua biên hàm. Variable-level cross-frame matching: nếu hàm A cấp phát biến `x`, gọi hàm B, và `x` không được giải phóng ở bất kỳ exit nào `reachable` → leak cross-function. Kết quả fold thành `feasibleLeakPath`.

Trên Juliet (leak intra-function), tool này $\Delta=0$. Trên LAMeD (dự án thực), nó bắt thêm 1 leak (`cjson merge_patch`), FP=0. Gain nhỏ nhưng sạch — cơ chế hoạt động end-to-end trên dự án thật.

2.5.4. Clang scan-build

`scanBuild` chạy Clang SA ở project level (không phải per-TU như `clang --analyze`). Kết quả là corroborative evidence: nếu scan-build cũng flag cùng biến, confidence tăng. Nhưng scan-build không thay đổi verdict trên Juliet (heuristic đã đủ mạnh).

2.6. Thiết kế tầng điều tra agentic (Investigation)

2.6.1. Stage A — Static fan-out sub-agents

Candidates được nhóm theo file affinity (4 candidate/nhóm), mỗi nhóm sinh một sub-agent. Sub-agent nhận 5 tool tĩnh (`candidateScan`, `astScan`, `functionSummary`, `pathConstraints`, `ownershipConventions`) + `read_file` + `done_static`. Tối đa 3 sub-agent chạy đồng thời.

Tool partitioning: sub-agent tĩnh CHỈ nhận tool tĩnh — model không thể nhảy sang tool động. Điều này đảm bảo mỗi agent tập trung vào nhiệm vụ cụ thể.

2.6.2. Stage B — Dynamic worker

Stage B chạy song song với Stage A. Có hai nhánh:

Nhánh tắt định (không LLM): Khi case có `buildCommand`, hệ thống chạy công thức cố định `buildTarget(buildCommand) → lsanRun(binary)`. Không có LLM trong vòng chạy → coverage tắt định → kết quả tái lập. Đây là đóng góp C3 của luận văn.

Nhánh LLM: Khi không có `buildCommand`, một dynamic worker (LLM sub-agent) tự đọc Makefile, chọn lệnh build, chạy sanitizer. Nếu build thất bại hai lần, worker dừng.

Bằng chứng dynamic được capture tự động qua `withDynamicEvidenceCapture`: mọi finding từ sanitizer được ghi vào store, không qua "discretion" của LLM. `reconcileDynamicEvidence` gộp finding vào bundle tương quan nhất.

2.6.3. Stage C — Synthesize

Stage C hợp nhất static context (từ Stage A) và dynamic evidence (từ Stage B) vào mỗi bundle. Coverage status: `exercised_clean` (chạy sạch), `exercised_leak` (phát hiện leak), `not_exercised` (không chạy được), `dynamic_off`. Đây là deterministic merge, không có LLM.

2.6.4. Stage D — Hybrid judge

Xem Section 2.7 bên dưới.

2.7. Thiết kế tầng phán xét hybrid (Judging)

2.7.1. Heuristic judge — path-sensitive, cho mọi bundle

Mọi bundle đều đi qua heuristic judge trước. Hàm scoring tính tổng điểm từ các tín hiệu:

Tín hiệu	Điểm
Runtime leak tương quan (LINKED)	+0.4 – 0.5
Structural "high" thiếu free	+0.5
Alloc→free chưa cặp (unpaired)	+0.25
Feasible leak path	+0.2
Ownership không chuyển ra ngoài	+0.15
Ownership chuyển ra ngoài	−0.1 / −0.25
Early-return	+0.1

Threshold: `clamped ≥ 0.7 → confirmed_leak`, `≥ 0.4 → likely_leak`, còn lại → `uncertain`.

Hai công precision ghi đề: (1) freed-by-callee hoặc dynamic chạy sạch → `likely_false_positive` (0.8); (2) verdict "flagged" thiếu tín hiệu mạnh (`correlatedRuntimeLeak` || `structuralHigh` || `unpaired`) → hạ xuống `uncertain`.

2.7.2. LLM judge — rubric-based, chỉ cho bundle borderline

Không phải mọi bundle đều cần LLM judge. Chỉ những bundle "borderline" mới được leo thang: verdict `likely_leak/uncertain`, hoặc confidence trong khoảng [0.35, 0.7].

Ngoài ra, `shouldEscalate` kiểm tra mâu thuẫn: static nói leak nhưng dynamic chạy sạch, hoặc ngược lại. Đây là lúc consensus cần nhất.

LLM judge nhận: source snippet (toàn bộ hàm bao, đã xoá comment), static context, dynamic evidence, và ownership conventions. Output: JSON với 5 nhãn (`confirmed_leak/likely_leak/uncertain/likely_false_positive/false_positive`), confidence, explanation.

Rubric ưu tiên evidence theo thứ tự: (1) runtime leak LINKED → `confirmed` ≥ 0.9; (2) ownership transferred → `likely_false_positive`; (3) freed on all paths → `false_positive`; (4) path-sensitive leak (conditional + feasible path) → `confirmed`.

2.7.3. Consensus judge — K-sample voting

Đây là đóng góp C1 của luận văn. Khi `CONSENSUS_N` > 1 (ví dụ K=3), hệ thống lấy K mẫu verdict LLM độc lập với temperature lấy mẫu > 0 (mặc định 0.7) để tạo diversity.

`combineVerdicts` gộp K nhãn thành 1 cờ flag theo luật:

- **Majority:** `flagging * 2 > n`.
- **Weighted:** `flagWeight / total > 0.5`. Phiếu nghị bằng chứng dynamic bị nhân ×0.3 (giảm trọng số).
- **Unanimous-to-flag:** `flagging === n`.

Nhãn cuối: verdict modal trong cụm đồng thuận. Hoà → chọn nhãn ít nghiêm trọng hơn.

Precision-override veto: exculpation mạnh của heuristic (FP/likely_FP, conf ≥ 0.75) phủ quyết cờ flag — nhưng chỉ gỡ flag, không *thêm*. Dynamic confirmed không bị veto.

2.7.4. Escalation logic

`shouldEscalate` leo thang lên consensus khi:

1. Bundle borderline (theo `isBorderline`).
2. Static nói leak nhưng verdict heuristic là FP/likely_FP.
3. Verdict là leak nhưng dynamic chạy sạch (`exercised_clean`).
4. Không có verdict leak nhưng dynamic có leak tương quan (`exercised_leak + LINKED`).

2.8. Thiết kế giao thức tái lập hai tầng

2.8.1. Vấn đề: LLM không tắt định bit-for-bit

Ngay cả ở `temperature=0`, LLM có thể cho kết quả khác nhau giữa các lần chạy do provider-side batching, floating-point scheduling. Báo cáo một con số đơn lẻ sẽ overstate reproducibility.

2.8.2. Tier-1 — `no_llm` tắt định

Toàn bộ đường phi-LLM (heuristic judge + recipe dynamic ghim + capture tắt định + scoring) cho kết quả chấm điểm y hệt giữa hai lần chạy. Ép bằng `gate-determinism-gate.sh` chạy hai lần → `assert-determinism.ts` so sánh scoring-relevant subset.

Gate từ chối hai kiểu "đậu giả": (a) self-compare — timestamp trùng làm hai lần chạy ghi cùng thư mục; (b) degenerate run — analyzer lỗi, mọi ca error.

2.8.3. Tier-2 — `llm_assisted` báo cáo phân phối

Vì không thể đạt bit-for-bit với LLM judge, số liệu được báo cáo dạng `mean ± std` qua nhiều lần chạy. `verdict-stability.ts` đo case-level stability, flip rate, modal agreement. Điều này phơi bày dao động mà aggregate "may mắn" có thể che giấu.

2.9. Thiết kế corpus pipeline

2.9.1. Chuẩn bị và ingest

Juliet: NIST v1.3 zip, hash `ada9d7e1...` `ingest.ts` copy file verbatim (kể cả `.h` local), nhóm C++ multi-file variant (tránh phá build), derive flaw/clean label từ source (quy ước bad/good).

LAMeD: Zenodo 15089703, clone repo tại bug commit. `cjson` pin `12c4bf1986`, 6 project còn lại từ `DiverseVul`.

2.9.2. Validate — 5-gate

`validate-corpus.ts` chạy năm gate: (a) zod schema, (b) structural (case dir + local `#include`), (c) compile (`clang -fsyntax-only` per TU), (d) label (overlap detection), (e) content-hash. HARD failure → quarantine; exit non-zero (CI-able).

2.9.3. Lock và gate

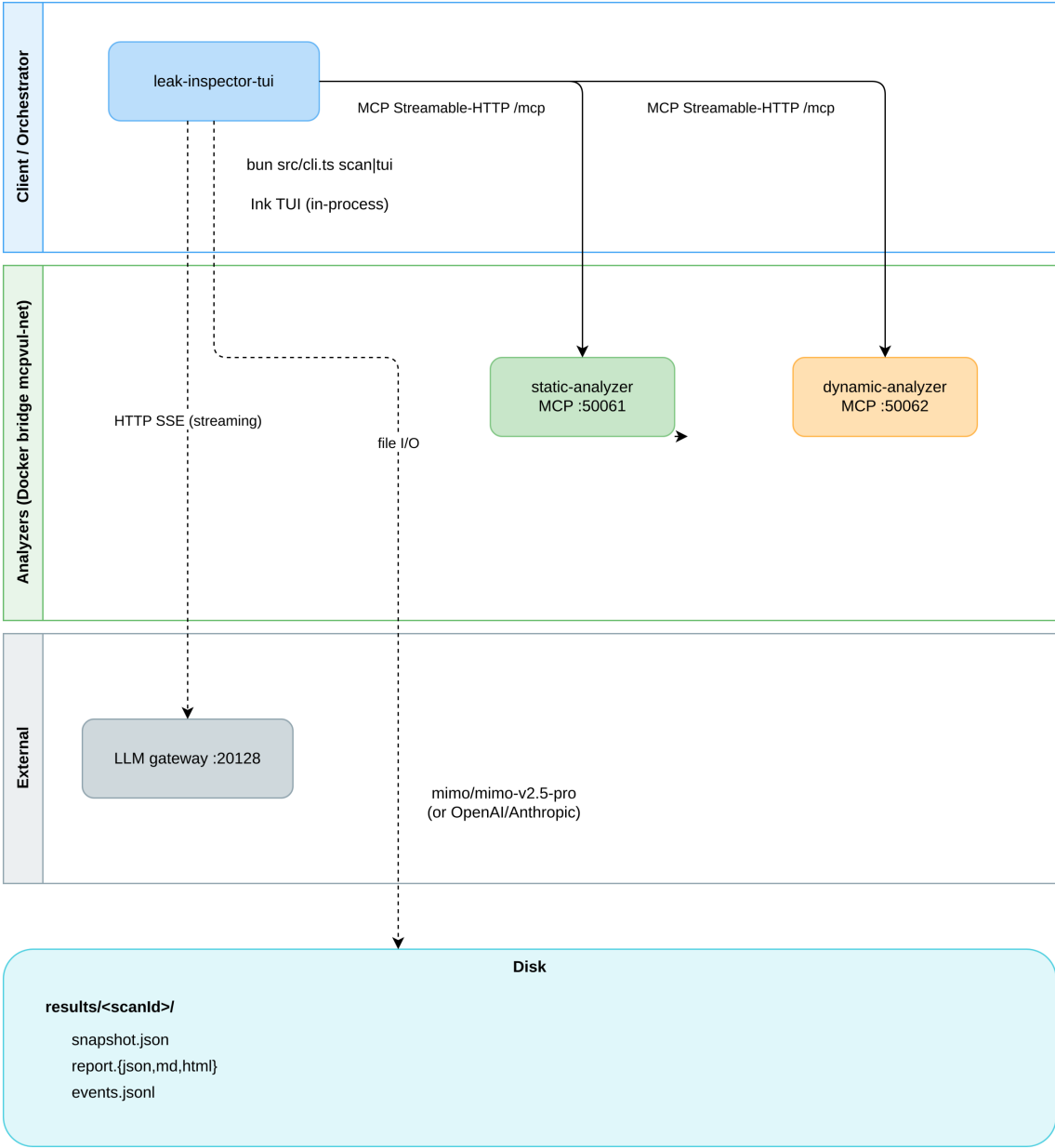
`--write-lock emit <corpus>.lock.json`: source hash, ingest commit, clang version, content-hash, validation summary. Juliet: `f578c3ee...`, 1658 cases, 0 quarantined. `evalHarness.runEval` từ chối corpus không có lockfile hoặc hash drift.

2.10. Tổng kết chương

Chương này trình bày thiết kế hệ thống bảy tầng, xoay quanh nguyên tắc "LLM owns POLICY, engine owns MECHANISM." Điểm khác biệt chính so với các hệ thống đã có: (1) kết hợp static + dynamic chuyên cho memory leak (không phải vulnerability nói chung); (2) consensus judge giảm dao động verdict; (3) tắt định hoá dynamic để đảm bảo reproducibility; (4) pipeline mở rộng được qua LLM discovery, không cần hardcoded per-project. Chương tiếp theo sẽ trình bày chi tiết cách hiện thực hoá thiết kế này.

Chương 3: Hiện thực, triển khai hệ thống

Một hệ thống phân tích memory leak dù được thiết kế tốt đến đâu, nếu chỉ tồn tại trên sơ đồ khối, cũng chưa thể kiểm chứng được. Chương này trình bày quá trình hiện thực hoá kiến trúc HYBRID pipeline đã đề cập ở Chương 2 — từ lựa chọn công cụ, cấu trúc thư mục, cho đến từng hàm cụ thể xử lý parsing AST, phối hợp sub-agent, và tổng hợp phán xét. Thay vì liệt kê công nghệ một cách khô khan, chúng tôi bắt đầu bằng cách giải thích tại sao từng lựa chọn được đưa ra. (xem Hình 3.1)



Hình 3.1: Topology triển khai hệ thống (orchestrator và hai analyzer MCP)

3.1. Công nghệ sử dụng và cấu trúc dự án

3.1.1. Lựa chọn ngôn ngữ và runtime

Hệ thống được viết hoàn toàn bằng TypeScript, chạy trên nền Node.js, quản lý qua pnpm. Đây là quyết định có phần đi ngược số đông — phần lớn công cụ phân tích code C/C++ (Clang SA [1], Infer, CodeQL) đều viết bằng C++ hoặc OCaml. Tuy nhiên, mục tiêu của luận văn không phải xây thêm một analyzer từ đầu, mà xây một **orchestrator** điều phối nhiều analyzer có sẵn qua giao thức MCP [42]. Với bài toán này, TypeScript có mấy lợi thế cụ thể:

Thứ nhất, MCP SDK chính thức (@modelcontextprotocol/sdk) chỉ có bản TypeScript/JavaScript. Dùng ngôn ngữ khác nghĩa là tự implement lại giao thức từ zero — một khoản nợ kỹ thuật không đáng có. Thứ hai, Tree-sitter [4] — thư viện parse AST mà static analyzer dựa vào — có binding Node.js trưởng thành, hỗ trợ cả grammar C (tree-sitter-c) và C++ (tree-sitter-cpp) với tốc độ parse hàng triệu dòng/giây. Thứ ba, hệ thống cần streaming SSE cho LLM response, HTTP server cho MCP endpoint, và terminal UI — tất cả đều là thế mạnh của hệ sinh thái JavaScript/TypeScript.

Runtime Bun được thử nghiệm ban đầu vì tốc độ khởi động nhanh hơn Node.js trong giai đoạn phát triển sớm. Tuy nhiên, khi vận hành CI/CD (GitHub Actions, Ubuntu Linux), hệ thống gặp một lỗi tương thích nghiêm trọng: gói tree-sitter — native addon biên dịch qua node-gyp — không phát hành sẵn bản build cho linux-x64; khi chạy dưới runtime Bun, cơ chế resolve module của gói này rẽ nhánh thẳng tới một đường dẫn prebuilds/ cố định, bỏ qua fallback node-gyp-build vốn biết tìm bản build biên dịch tại chỗ (build/Release/). Hệ quả: mọi lệnh parse AST trên CI thất bại âm thầm (lỗi bị nuốt bởi một khối try/catch phòng thủ), trả về kết quả rỗng mà không có log lỗi rõ ràng — một lớp bug khó phát hiện trong chính pipeline đánh giá tự động của luận văn. Sau khi xác định nguyên nhân gốc, hệ thống được chuyển sang Node.js làm runtime (dev, test, script) và pnpm làm package manager, đồng thời thay bun:test bằng Vitest. Quyết định này ưu tiên tính tương thích và độ ổn định của pipeline CI/CD hơn tốc độ khởi động thô — vốn không phải

nút thắt cổ chai thực sự trong một hệ thống mà độ trễ chủ yếu đến từ lời gọi LLM, không phải từ runtime JavaScript.

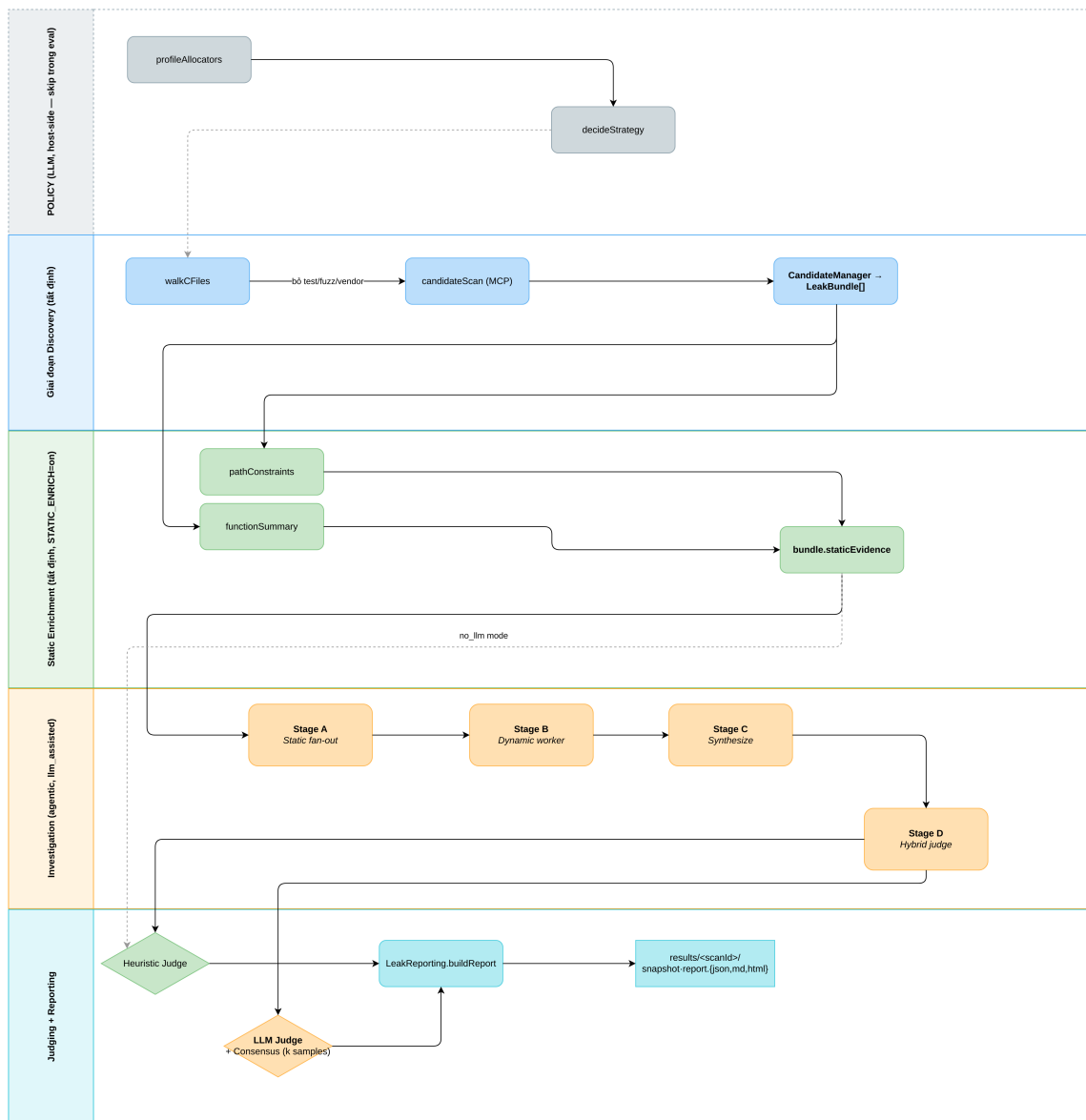
3.1.2. Kiến trúc monorepo Turborepo

Toàn bộ workspace được tổ chức theo mô hình monorepo với Turborepo quản lý dependency graph và task pipeline. Cấu trúc thư mục như sau:

```
Thesis/
├── apps/
│   ├── static-analyzer/      ← NestJS MCP service, port 50061
│   ├── dynamic-analyzer/    ← NestJS MCP service, port 50062
│   └── leak-inspector-tui/   ← Ink TUI, orchestrator chính
├── packages/
│   └── common/               ← @cleak/common: types, Zod schemas, judges,
renderers
│   └── agent-core/           ← @cleak/agent-core: tool-calling loop,
providers, MCP client
├── docker-compose.yml
├── turbo.json
└── package.json
```

`turbo.json` định nghĩa pipeline build gồm 5 task: `build` (output `dist/**`, phụ thuộc `^build`), `dev` (persistent, không cache), `typecheck`, `lint`, và `test` (phụ thuộc `build`). Task `build` sử dụng `tsup` cho việc bundle — mỗi `app/package` có `tsup.config.ts` riêng, xuất ESM module.

Hai NestJS apps (`static-analyzer`, `dynamic-analyzer`) dùng framework NestJS cho dependency injection và lifecycle management, nhưng điểm mấu chốt là chúng **không** expose REST controller hay gRPC như thông thường. Thay vào đó, `main.ts` của mỗi app khởi tạo DI context rồi gọi `createStaticMcpServer()` / `createDynamicMcpServer()` để serve MCP/HTTP trực tiếp. Quyết định loại bỏ gRPC (và toàn bộ `proto/` directory) được đưa ra sau khi xác nhận rằng không còn consumer nào cho gRPC sau khi hệ thống web bị xóa khỏi master. (xem Hình 3.2)



Hình 3.2: Pipeline HYBRID bốn tầng của orchestrator

3.1.3. Các framework và thư viện chính

Bảng 3.1 tổng hợp các thư viện cốt lõi và vai trò cụ thể của chúng trong hệ thống.

Thư viện	Phiên bản	Vai trò trong hệ thống
@modelcontextprotocol/sdk	≥1.0	MCP server (analyzer) + client (TUI) — giao thức chuẩn để orchestrator gọi analyzer
@nestjs/core, @nestjs/common	≥10	DI container + lifecycle cho static-analyzer và dynamic-analyzer
tree-sitter, tree-sitter-c, tree-sitter-cpp	≥0.21	Parse AST C/C++ — backbone của toàn bộ static analysis
ink (React for CLI)	≥4	Terminal UI framework — render kết quả realtime trong terminal
zod	≥3	Schema validation cho MCP

Thư viện	Phiên bản	Vai trò trong hệ thống
		tool input, config file, allocator profile, corpus manifest
tsx / tsup	latest	Dev server (tsx) + production bundle (tsup)

Điều đáng chú ý là hệ thống **không** dùng bất kỳ framework AI/agent nào (LangChain, CrewAI, AutoGen). Vòng lặp tool-calling được implement thủ công trong `packages/agent-core/src/loop.ts` — một async generator khoảng 280 dòng, xử lý tool dispatch, permission resolution, context compaction, và completion guard. Lý do: các framework agent thêm một lớp abstraction không cần thiết khi orchestrator chỉ cần native function-calling (`tool_use/tool_result`) mà mọi LLM provider lớn đều hỗ trợ.

3.2. Cài đặt static analyzer

3.2.1. C-parser service: routing C/C++ và quản lý allocator

Static analyzer bắt đầu từ `CParserService` — một NestJS injectable service đóng vai trò parser trung tâm. Mọi tool phân tích (`candidateScan`, `astScan`, `functionSummary`, `pathConstraints`, `callGraph`, `interproceduralFlow`) đều gọi `cParser.parse()` để nhận kết quả AST trước khi thực hiện phân tích chuyên biệt.

Vấn đề đầu tiên cần giải quyết là routing ngôn ngữ. `tree-sitter-c` và `tree-sitter-cpp` là hai grammar riêng biệt — dùng grammar sai sẽ dẫn đến parse error hoặc AST sai (ví dụ: `tree-sitter-c` không hiểu `new/delete`, `template`, hay toán tử `::`). Service xác định grammar dựa trên phần mở rộng file:

```
static isCppPath(filePath?: string): boolean {
  return /\.(cc|cpp|cxx|c\+\+|hpp|hxx|hh|ipp|tcc|inl)$/i.test(filePath || '');
}
```

Hai parser (C và C++) được lazy-initialize và reuse qua toàn bộ vòng đời service — mỗi parser chỉ tạo một lần, lần đầu cần dùng. Kết quả parse được cache theo content hash trong một Map bounded theo **dung lượng byte** (`STATIC_PARSER_CACHE_MAX_MB`, mặc định 256MB, sàn 16MB — không còn giới hạn theo số entry cố định như thiết kế ban đầu) để tránh parse lại cùng một file khi nhiều tool gọi liên tiếp trong CÙNG một process.

Tầng cache thứ hai, bổ sung sau khi phát hiện vấn đề hiệu năng thực tế trên sweep LAMeD: một baseline sweep chạy $9 \text{ config} \times$ tối đa 3 lần lặp lại quét CÙNG một project checkout không đổi rất nhiều lần, nhưng cache trong-process bị mất mỗi khi process khởi động lại — cache này được persist ra đĩa dưới `RUNS_DIR/ast-cache` (key = SHA1 nội dung file, giống hệt key trong-process), nên một process mới tái sử dụng kết quả parse của process trước thay vì parse lại từ đầu. Kết quả `ParseResult/FunctionInfo` chỉ gồm kiểu dữ liệu JSON-an-toàn (string/number/boolean/array/plain object, không có Set/Map), nên round-trip qua đĩa không mất dữ liệu. Đo trực tiếp trên một case LAMeD thật (`libxml2`, 143 file, 43 candidate, mode `no_llm`): quét lại cùng case với cache đĩa đã "ấm" (process mới, cache đĩa còn nguyên) mất 3.0s / đỉnh RSS 266MB, so với 54.0s / đỉnh RSS 1.03GB khi cache đĩa rỗng — giảm ~18 lần thời gian và ~74% bộ nhớ đỉnh cho đúng pattern quét-lại mà một sweep thực hiện lặp đi lặp lại. Cả hai thay đổi đều thuần cơ chế (không đụng logic judge hay nội dung evidence), nên không kỳ vọng thay đổi bất kỳ số liệu F1/precision/recall nào đã báo cáo ở Chương 4 — nhưng đây là một giả định dựa trên ngữ nghĩa của cache, chưa phải một phép đo độc lập xác nhận lại toàn bộ sweep.

Một thiết kế quan trọng khác là cơ chế threading allocator set. Hardcoded set mặc định chỉ gồm các hàm libc cơ bản: `malloc`, `calloc`, `realloc`, `strdup`, `free`, và các biến thể kernel (`kmalloc`, `kfree`). Nhưng các dự án thực tế sử dụng

factory	allocator	riêng	—	cJSON	có
<code>cJSON_CreateObject()</code>	<code>cJSON_Delete()</code> ,			<code>libtiff</code>	có
<code>_TIFFmalloc()</code>	<code>_TIFFfree()</code> .				

Nếu chỉ dựa vào set mặc định, hệ thống sẽ miss hoàn toàn các allocation site này.

Giải pháp: mỗi lần gọi `parse()`, caller có thể truyền `extraAllocators` và `extraDeallocators` — danh sách tên hàm cụ thể cho dự án đó. Các tên này được validate bằng regex (`/^[A-Za-z_]\w*$/`) trước khi merge vào set hiện tại. Danh sách extra này được phát hiện bởi tầng LLM-generalization (mục 3.7.1) và threading xuyên suốt **tất cả** 7 static tool — từ `candidateScan` (phát hiện allocation site) đến

functionSummary (cấp hoá alloc→free) đến pathConstraints (theo dõi biến chưa free trên exit path).

3.2.2. Đăng ký 11 MCP tools

Static analyzer expose 11 tools qua MCP/HTTP, được đăng ký trong `createStaticMcpServer()`. Mỗi tool được khai báo với Zod inputSchema — MCP SDK sử dụng Zod schema trực tiếp làm JSON Schema cho tool description, nên client (TUI) biết chính xác kiểu dữ liệu cần gửi.

Bảng 3.2 liệt kê đầy đủ 11 tools và chức năng:

Tool	Chức năng	Input chính
indexFiles	Quét đệ quy tất cả file C/C++ từ root path	rootPath, fileLimit?, excludePatterns?
candidateScan	Lexical scan tìm allocation site (malloc, new, factory)	filePath, content?, extraAllocators?, extraDeallocators?
astScan	Phân tích cấu trúc AST cho memory leak pattern	filePath, content?
callGraph	Trích xuất đồ thị gọi hàm	rootPath, files[], extraAllocators?, extraDeallocators?
functionSummary	Tóm tắt hàm: alloc/free balance, exit path	filePath, content?, functionName, extraAllocators?, extraDeallocators?
pathConstraints	Phân tích ràng buộc đường đi quanh allocation	filePath, content?, lineNumber, extraAllocators?, extraDeallocators?
interproceduralFlow	Truy vết alloc/free liên hàm	rootPath, functionName, files[], extraAllocators?, extraDeallocators?
ownershipSummary	Tóm tắt quy ước ownership qua nhiều file	files[], rootPath
ownershipConventions	Phát hiện quy ước transfer ownership trong file	content?, filePath
scanBuildRun	Chạy Clang Static Analyzer (scan-build)	projectPath, buildCommand, timeoutSec?
scanBuildGetReport	Lấy kết quả scan-build	runId

Chức năng `ok()` wrapper chuyển mọi kết quả service thành format MCP chuẩn: `{ content: [{ type: 'text', text: JSON.stringify(result) }], structuredContent: result }`. Cách tiếp cận này giữ cho MCP server mỏng — toàn bộ logic phân tích nằm trong các service, server chỉ đóng vai trò routing.

3.2.3. Phân tích path-sensitive: guard-subset reconciliation

Đây là phần phức tạp nhất của static analyzer, và cũng là phần trải qua nhiều lần refactor nhất. Bài toán cơ bản: cho một allocation site, xác định những exit path nào của hàm sẽ **mất** con trỏ đã cấp phát mà không giải phóng.

`PathConstraintsService.analyze()` nhận vào `filePath`, `content`, và `lineNumber` (dòng allocation). Bước đầu tiên là xác định hàm chứa allocation — service tìm hàm innermost bằng tree-sitter range (`fn.lineNumber ≤ lineNumber ≤ fn.endLine`), sort theo kích thước range tăng dần để chọn hàm lồng nhau đúng nhất.

Từ kết quả parse, service trích xuất tất cả exit paths của hàm. Mỗi exit path được gán nhãn `leakRisk` dựa trên phân tích guard-subset reconciliation — một kỹ thuật tự phát triển, lấy cảm hứng từ cách Clang SA [1] xử lý path condition nhưng đơn giản hơn đáng kể:

Nguyên tắc cơ bản: Nếu một biến `ptr` được cấp phát trong hàm, và trên một exit path nào đó, `ptr` bị free bởi một lệnh free nằm dưới **cùng điều kiện guard** (hoặc guard con) với allocation, thì cặp `alloc→free` được coi là "hoà giải" (reconciled) trên path đó. Ngược lại, nếu exit path có allocation nhưng không có free tương ứng, hoặc free nằm dưới guard khác, biến đó là "unreconciled" — một dấu hiệu leak.

Ví dụ cụ thể: trong hàm `process()` của `cJSON`, `merge_patch` được cấp phát ở dòng 50, và `cJSON_Delete(merge_patch)` chỉ xuất hiện ở nhánh else (dòng 65). Nhánh if (dòng 58) return NULL mà không giải phóng — đây là exit path unreconciled, leak risk = high.

Mỗi exit path còn được gán thêm mảng `guards` — danh sách các điều kiện bao quanh (kể cả polarity: bị phủ định hay không). Thông tin này cho phép heuristic judge phân biệt giữa "free nằm trong cùng nhánh với alloc" (an toàn) và "free nằm ở nhánh khác" (có thể leak).

3.2.4. Reachability bảo thủ: collectDeadLines

Một vấn đề thực tế khi phân tích CFG: không phải mọi exit path đều thật sự reachable. Nếu một hàm có `return` ở dòng 10 và `return` ở dòng 20, thì dòng 20

chỉ reachable nếu không có unconditional terminator (goto, exit, abort) giữa dòng 10 và 20.

`collectDeadLines()` trong `CParserService` xử lý bài toán này bằng cách duyệt tuần tự các statement trong hàm. Khi gặp unconditional terminator (`return`, `goto`, `exit()`, `abort()`), tất cả các statement **sau** nó cho đến label đích tiếp theo (nếu là `goto`) hoặc closing brace (nếu là `return`) đều bị đánh dấu là dead code. Exit paths bắt đầu từ dead code bị loại khỏi danh sách reachable.

Cách tiếp cận này cố tình bảo thủ — nó chỉ detect unconditional terminator, không suy luận sâu về điều kiện luôn đúng (ví dụ `if (true)`). Lý do: mục tiêu là giảm FP (false positive), không phải tối đa hoá recall. Một exit path bị đánh sai là unreachable (thực tế là reachable) chỉ gây ra miss leak — nhưng một exit path unreachable bị giữ lại sẽ tạo FP.

3.2.5. Parameter-ownership leak

Một biến thể leak mà các công cụ truyền thống thường bỏ sót: hàm nhận một con trỏ tham số, giải phóng nó trên **một số** path nhưng không giải phóng trên path khác. `cJSON` là ví dụ điển hình — `cJSONUtils_MergePatch(caseSensitive, target, patch)` gọi `cJSON_Delete(target)` trên nhánh error nhưng không gọi trên nhánh thành công (vì `target` được modify in-place và caller vẫn sở hữu).

Hệ thống phát hiện pattern này bằng cách kiểm tra: (a) một tham số kiểu con trỏ xuất hiện trong lệnh `free` nào đó của hàm, và (b) có exit path nào reachable mà không có `free` tương ứng. Nếu cả hai điều kiện đúng, một synthetic candidate được tạo với `allocation_type = 'parameter_ownership'` — cho phép heuristic judge và LLM judge đánh giá nó như một allocation site bình thường.

3.3. Cài đặt dynamic analyzer

3.3.1. Build orchestration và sanitizer injection

Dynamic analyzer đảm nhận việc biên dịch dự án với sanitizer flags, chạy binary đã instrument, và parse kết quả. Quá trình build được orchestrate bởi tool `buildTarget` — nhận `projectPath` và `buildCommand` từ caller, inject thêm sanitizer flags, rồi thực thi trong `child_process`.

Hai sanitizer chính được hỗ trợ:

- **LeakSanitizer (LSan):** flag `-fsanitize=leak -g -O0`. LSan là sanitizer chuyên dụng cho leak detection, chạy tại exit point của chương trình và báo cáo các block chưa giải phóng. Ưu điểm: không abort mid-run (khác ASan), output format dễ parse.
- **AddressSanitizer (ASan):** flag `-fsanitize=address`. ASan detect broader class lỗi (buffer overflow, use-after-free) nhưng leak detection chỉ là tính năng phụ. Dùng khi cần coverage cho cả memory error lẫn leak.

Một chi tiết kỹ thuật quan trọng: ASan và LSan reserve khoảng 20TB virtual address space cho shadow memory. Trên môi trường có `ulimit -v` giới hạn, quá trình này sẽ crash ngay lập tức với lỗi `mmap`. Dynamic analyzer xử lý bằng cách set `unlimitedAddressSpace = true` trước khi spawn child process — tương đương `ulimit -v unlimited` trong shell.

3.3.2. Valgrind integration

Bên cạnh compiler-based sanitizer, hệ thống còn tích hợp Valgrind Memcheck [7] — một binary instrumentation tool chạy trên Linux. Valgrind không cần recompile (khác sanitizer), nhưng chậm hơn khoảng 10–50×.

Việc tích hợp tương đối đơn giản: `spawn valgrind --tool=memcheck -leak-check=full --show-leak-kinds=all <binary>` trong `child_process`, capture `stderr` (nơi Valgrind ghi output), rồi parse kết quả thành normalized report. Report parser xử lý format chuẩn của Valgrind: mỗi leak block gồm header (definitely lost: X bytes), stack trace, và allocation context.

3.3.3. llvm-symbolizer

Một dependency bắt buộc nhưng dễ bị bỏ qua: sanitizer output chỉ chứa địa chỉ hex (`0x4a2b3c`) nếu không có symbolizer. Dynamic analyzer yêu cầu `llvm-symbolizer` có sẵn trong `PATH` — tool này chuyển đổi địa chỉ thành `file:line:function` mà LLM judge cần để liên kết finding với source code.

3.3.4. Đăng ký 9 MCP tools

Bảng 3.3 liệt kê 9 tools của dynamic analyzer:

Tool	Chức năng
buildTarget	Build dự án với sanitizer-instrumented compiler
valgrindMemcheck	Chạy Valgrind Memcheck
valgrindGetReport	Lấy normalized Valgrind report
valgrindListFindings	Query Valgrind findings với filter
valgrindCompareRuns	So sánh hai Valgrind runs
asanRun	Chạy binary dưới AddressSanitizer
lsanRun	Chạy binary dưới LeakSanitizer
runBinary	Chạy binary không instrument
listRuns	Liệt kê các dynamic analysis runs đã lưu

Cấu trúc đăng ký tương tự static analyzer: Zod `inputSchema`, `ok()` wrapper, delegation tới service methods. Điểm khác biệt là dynamic analyzer có thêm `RUNS_DIR` volume mount (mặc định `/runs` trong Docker) để lưu artifact giữa các run.

3.4. Cài đặt agent core

3.4.1. Vòng lặp tool-calling native

`packages/agent-core` là trái tim của hệ thống — nơi LLM và tools giao tiếp với nhau. Module chính là `queryLoop()` trong `loop.ts`, một async generator implement vòng lặp: hỏi model → nhận text + `tool_use` → dispatch tools → thread `tool_result` → lặp lại.

Mỗi iteration (turn) gồm 5 bước:

- 1. Compaction check:** Ước tính kích thước prompt (dựa trên `lastInputTokens` từ model response, hoặc heuristic 4 chars/token). Nếu vượt `thresholdTokens` (mặc định 100,000), các `tool_result` cũ ngoài `keepRecentTurns` turns gần nhất được prune in-place — thay bằng `placeholder [elided: N chars of stale tool output pruned to save context]`. Cách này giữ nguyên message structure (không phá `tool_use↔tool_result` pairing) trong khi giảm rõ rệt kích thước prompt. Trong thực nghiệm, compaction thường tiết kiệm 30–50% token khi sub-agent đã chạy 8+ turns.

- 2. Model call:** Gọi `deps.callModel()` với system prompt, messages, và tool definitions. Response được streaming (SSE) — chunk đầu tiên trigger `onModelActivity('receive')` để TUI hiển thị indicator.

3. Permission resolution: Mỗi `tool_use` request đi qua

`tool.checkPermissions()`. Static tools luôn auto-allow (read-only);

dynamic tools có thể yêu cầu user approval trong interactive mode.

4. Tool dispatch: Các tool call được phân thành hai nhóm — read-only + concurrency-safe chạy song song (giới hạn bởi concurrency, mặc định 10), còn lại chạy tuần tự. `mapWithLimit()` quản lý pool worker.

5. Terminal check: Nếu tool vừa chạy nằm trong `terminalTools` set (ví dụ `done_static, done_dynamic`), vòng lặp kết thúc với reason `'finalized'`.

3.4.2. Completion nudge mechanism

Một vấn đề thực tế: LLM đôi khi dừng sớm trước khi hoàn thành công việc — ví dụ sub-agent chỉ gather evidence cho 2 trong 4 candidates rồi tuyên bố "done". Cơ chế `checkCompletion` giải quyết vấn đề này.

Khi model trả về response không có `tool_use` (tức muốn dừng), loop gọi `checkCompletion()` — hàm do caller cung cấp. Nếu có công việc chưa xong, hàm trả về chuỗi nudge (ví dụ: "You stopped, but 2 candidate(s) have NO static evidence yet: id-1, id-2. Run functionSummary/pathConstraints for them, then call `done_static`"). Nudge này được inject làm user message, buộc model tiếp tục.

Budget nudge tối đa là `maxStopNudges` (mặc định 3). Counter reset mỗi khi model gọi tool thành công — nghĩa là nudge chỉ áp dụng cho các turn mà model "nghỉ ngơi" mà không làm gì. Cơ chế này ngăn model loop vô hạn trong khi vẫn cho phép nó recover từ stop sớm.

3.4.3. Multi-provider streaming

Hệ thống hỗ trợ 4 loại provider, được chọn qua config `provider`:

- **local:** OpenAI-compatible endpoint (mặc định

`http://localhost:20128/v1, model mimo/mimo-v2.5-pro`). Đây là cấu hình của thí nghiệm đầu (mục 4.2.3, 2026-06); các sweep chót của luận văn

chạy trên `openai-compatible` với model `deepseek-v4-flash` (mục 4.2.2, Phụ lục C.3).

- **openai:** OpenAI API trực tiếp.
- **anthropic:** Anthropic Messages API — cần adapter riêng vì format khác OpenAI.
- **openai-compatible:** Bất kỳ endpoint nào tuân thủ OpenAI Chat Completions API.
`callOpenAiChat()` trong `providers/openaiChat.ts` xử lý cả `local` và `openai-compatible` — hai loại này chỉ khác nhau ở `baseUrl` và `auth`.
Request gửi `stream: true, stream_options: { include_usage: true }` để nhận token usage realtime. Native function-calling được dùng (body có `tools` và `tool_choice: 'auto'`), không phải ReAct text parsing.

Idle timeout (mặc định 75 giây) reset theo mỗi chunk nhận được — không phải tổng thời gian chạy. Cách này cho phép một request chạy lâu (ví dụ model đang suy nghĩ về code phức tạp) mà không bị timeout, trong khi vẫn detect được connection chết (không nhận chunk nào trong 75 giây).

3.5. Cài đặt orchestrator (leak-inspector-tui)

3.5.1. Workflow 4-stage

Orchestrator là ứng dụng chính — nơi tất cả thành phần ghép lại với nhau. File `workflowInvestigation.ts` implement pipeline 4-stage đã thiết kế ở Chương 2. (xem Hình 3.3)

Stage A — Static evidence (fan-out bounded):

Giai đoạn này spawn nhiều sub-agent chạy song song, mỗi agent负责 một nhóm candidates. Việc nhóm candidates theo file affinity (`groupByFileAffinity()`) đảm bảo các allocation site trong cùng một file luôn thuộc cùng một sub-agent — cho phép agent quan sát interprocedural pattern (allocator và freeing sink trong cùng file).

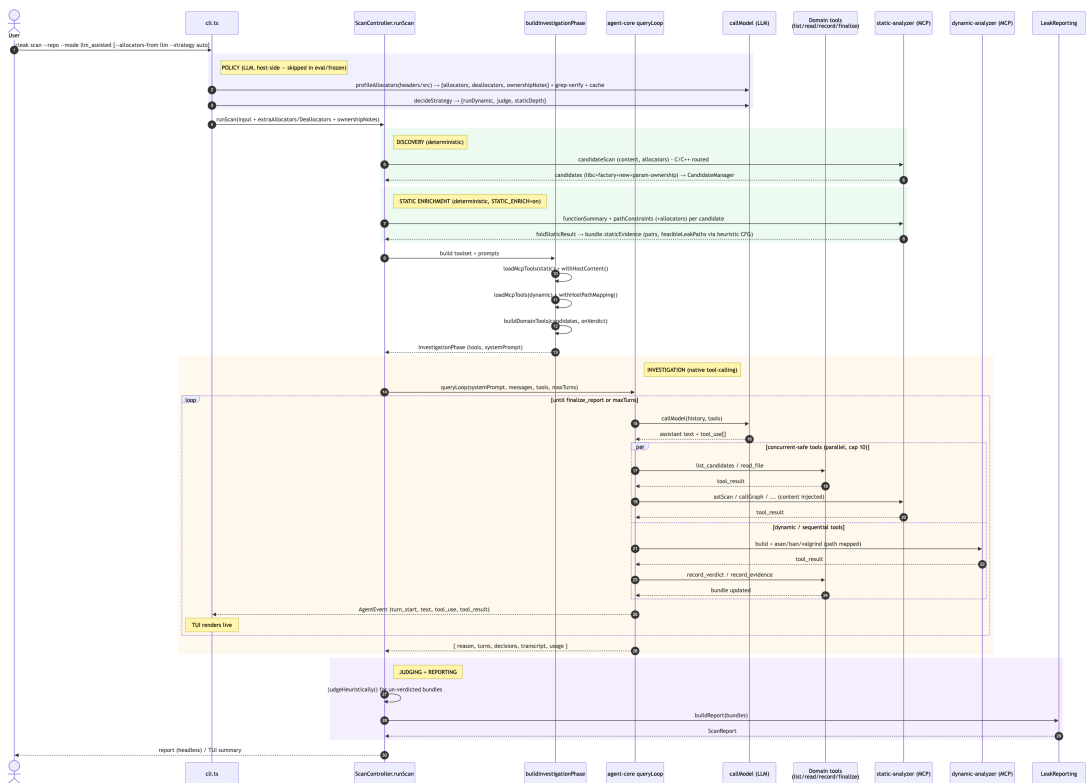
Mỗi sub-agent nhận: (a) system prompt hướng dẫn gather evidence bằng 5 static tools, (b) user message liệt kê candidates của nó, (c) tool set gồm `functionSummary`, `pathConstraints`, `astScan`, `ownershipConventions`, `read_file`, và `done_static`. Agent chạy trong

queryLoop riêng, context hoàn toàn cách ly — output được fold vào staticStore (Map<string, Record<string, any>>) qua wrapper withStaticContextCapture().

Concurrency mặc định cho Stage A: 3 sub-agent đồng thời (staticConcurrency), mỗi nhóm tối đa 4 candidates (staticGroupSize). Các con số này dựa trên thực nghiệm: nhiều hơn 3 sub-agent gây áp lực lên LLM gateway (đặc biệt với local model), và group size 4 đủ nhỏ để context mỗi agent không vượt quá 8,000–10,000 tokens.

Khi toolSelect = false (ablation baseline không có agentic tool selection), Stage A bị bỏ qua hoàn toàn — static evidence đến từ deterministic enrichment stage chạy trước đó.

Stage B — Dynamic evidence (chạy đồng thời với A):



Hình 3.3: Vòng lặp tool-calling native của agent-core trong TUI

Stage B chạy concurrently với Stage A (dùng Promise.all). Có hai path:

Deterministic path: Khi buildCommand đã biết, hệ thống chạy runDeterministicDynamic() — gọi buildTarget rồi lsanRun theo thứ

tự cố định, không có LLM nào trong loop. Kết quả sanitizer được capture tự động bởi `wrapper withDynamicEvidenceCapture()`.

LLM fallback: Khi build system không rõ, một LLM worker nhận `dynamicWorkerSystemPrompt` — hướng dẫn đọc Makefile/CMakeLists, build với sanitizer flags, rồi chạy. Worker cũng bị giới hạn: build tối đa 1 lần, mỗi dynamic tool tối đa 1 lần.

Quan trọng: dù dùng path nào, việc capture evidence là **deterministic**. Wrapper `withDynamicEvidenceCapture()` ghi nhận raw finding từ sanitizer tool — LLM chỉ quyết định tool nào gọi, không quyết định finding nào được ghi.

Stage C — Synthesize (deterministic):

Giai đoạn này không có LLM. `reconcileDynamicEvidence()` fold mỗi dynamic finding vào bundle tương ứng tốt nhất (dựa trên correlation rank: cùng function > cùng file > cùng allocation type). `computeDynamicCoverage()` gán explicit coverage status cho mỗi bundle: `exercised_leak`, `exercised_clean`, hoặc `not_exercised`.

Stage D — Hybrid judge:

Giai đoạn cuối cùng. Mỗi bundle nhận heuristic verdict trước (`heuristicVerdict()`). Các bundle borderline — defined là `likely_leak/uncertain` HOẶC confidence nằm trong khoảng `[0.35, 0.7]` — được escalate lên LLM judge hoặc consensus judge.

3.5.2. LLM judge và escalation logic

`judgeBundleWithLlm()` trong `llmJudge.ts` gửi cho model: (a) system prompt dài khoảng 500 từ với rubric chi tiết, (b) source snippet hàm chứa allocation (comments stripped để không leak benchmark labels), (c) tóm tắt static context (ownership, alloc→free pairs, feasible leak paths), và (d) runtime evidence nếu có.

Model bắt buộc trả JSON: `{verdict, confidence, explanation, evidence}`. Response được parse bằng Zod, sau đó enrich bởi `enrichLeakVerdict()` — đảm bảo mọi leak verdict đều có root cause và repair diff.

Logic `shouldEscalate()` quyết định bundle nào cần LLM second opinion:


```
function shouldEscalate(bundle: LeakBundle): boolean {
  const v = bundle.verdict;
  if (!v) return true;
  // Borderline verdict
  if (v.verdict === 'likely_leak' || v.verdict === 'uncertain') return
true;
  // Borderline confidence
  if (v.confidence >= 0.35 && v.confidence <= 0.7) return true;
  // Static↔verdict contradiction
  if (hasStaticEvidence(bundle) && v.verdict === 'false_positive') return
true;
  return false;
}
```

3.5.3. Consensus judge: K-sample voting

Khi `cfg.consensus.n > 1`, Stage D chuyển sang chế độ consensus — gọi LLM judge N lần (mỗi lần `temperature > 0` để tạo diversity) rồi kết hợp bằng `combineVerdicts()`.

Ba rule kết hợp:

- **majority**: Đơn giản nhất — hơn nửa samples đồng ý flag thì flag.
- **weighted**: Mỗi vote có `weight = confidence`. Vote contradict dynamic evidence bị giảm `weight × 0.3` (flagging cleared site hoặc clearing confirmed leak).
- **unanimous-to-flag**: Bảo thủ nhất — tất cả samples phải đồng ý mới flag.

Sau khi combine, heuristic precision-override kiểm tra: nếu heuristic có strong exculpation (`confidence ≥ 0.75`, verdict `likely_false_positive` hoặc `false_positive`) VÀ dynamic không confirm leak → consensus flag bị veto. Override chỉ xóa flag, không bao giờ thêm — recall của LLM được bảo toàn.

3.6. Cài đặt common library (@cleak/common)

3.6.1. Heuristic judge: scoring function

`heuristic-judge.ts` là module quan trọng nhất của `@cleak/common` — nó tạo verdict cho **mọi** bundle, dù ở chế độ `no_llm` hay `llm_assisted`. Scoring function tính điểm trên thang `[0, 1]` dựa trên nhiều tín hiệu:

Tín hiệu	Điểm	Điều kiện
Runtime leak correlated	+0.50	definitely_lost/asan_leak, linked to candidate
Runtime leak correlated (possibly lost)	+0.20	linked, nhưng leak kind yếu
Unpaired alloc→free	+0.25	Biến allocation không có free nào
Conditional alloc→free	+0.15	Free trên một số path, không

Tín hiệu	Điểm	Điều kiện
		free trên path khác
Feasible leak path reachable	+0.20	Exit path reachable, unreconciled allocation
Path-sensitive leak	+0.15	Conditional + reachable exit leak
Allocator role, no ownership out	+0.15	Allocator trả về local, không transfer
Early return skip cleanup	+0.10	Hàm có nhiều return, allocation có thể bị skip
Clang scan-build corroboration	+0.15	scan-build báo leak cho candidate này
Structural high	+0.50	Phân tích cấu trúc xác định missing free
Ownership transferred	-0.25	Ownership trả caller/sink (không có runtime leak)

Ngưỡng phán xét: `confirmed` ≥ 0.7 , `likely` ≥ 0.4 . Đây là frozen benchmark defaults — thực nghiệm luôn dùng giá trị này để đảm bảo công bằng.

Hai precision gate ngăn FP:

Gate 1 — Dynamic cleared: Sanitizer chạy và không báo leak cho site này, static signals yếu $\rightarrow$ `likely_false_positive` (confidence 0.8). Gate này không kích hoạt cho real leak vì Juliet * `_bad` hoặc leak ở runtime (correlated) hoặc không free ở đâu (structural high).

Gate 2 — Strong signal required: Muốn flag leak, phải có ít nhất một tín hiệu mạnh (correlated runtime leak, structural high, unpaired alloc $\rightarrow$ free, path-sensitive leak, hoặc ownership issue). Một đồng heuristic lexical yếu không đủ.

3.6.2. Consensus judge: combineVerdicts

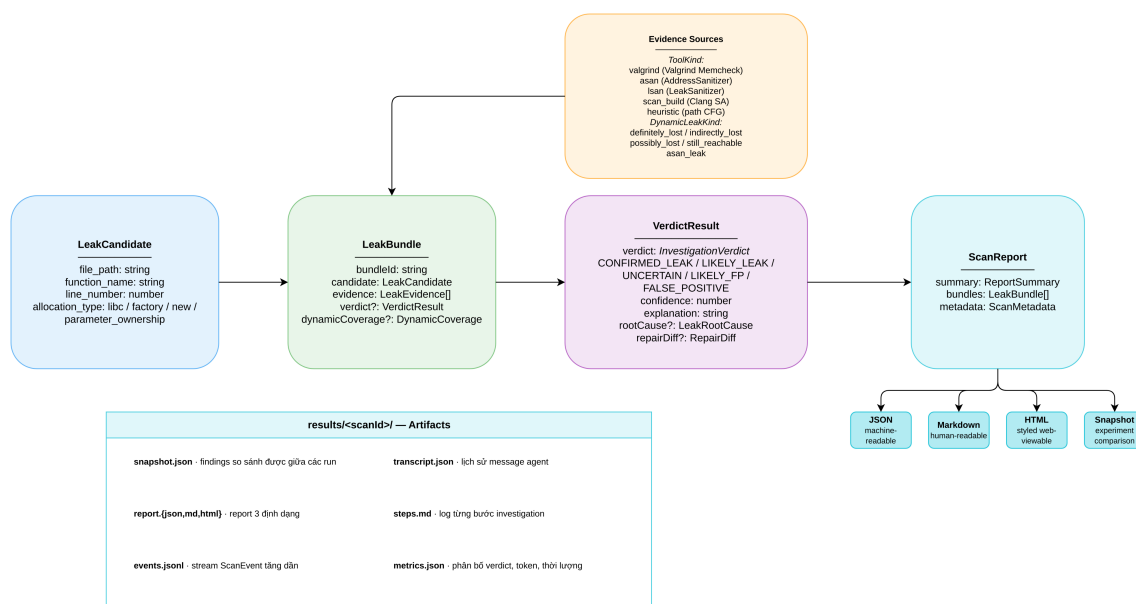
Module `consensus-judge.ts` là đóng góp cốt lõi C1 của luận văn. Hàm `combineVerdicts()` là pure function (không I/O), hoàn toàn unit-testable.

Trước khi vote, hàm `deriveFusion()` tóm tắt bằng chứng thành hai trục:

- **Static:** `'leak'` (unpaired alloc hoặc reachable leak path), `'clean'` (ownership handed out), hoặc `'ambiguous'`.
- **Dynamic:** `'confirmed'` (runtime leak correlated), `'cleared'` (sanitizer chạy clean), hoặc `'none'`.

Vote weight trong rule weighted bị ảnh hưởng bởi fusion: nếu dynamic `'cleared'` mà vote vẫn flag $\rightarrow$ weight giảm $\times 0.3$. Ngược lại, dynamic `'confirmed'` mà vote clear $\rightarrow$ cũng giảm $\times 0.3$. Cơ chế này tạo "giọng nói" cho

bằng chứng runtime — sanitizer đã chạy sạch thì LLM khó mà flag bừa. (xem Hình 3.4)



Hình 3.4: Mô hình dữ liệu LeakBundle và các cấu trúc liên quan

3.6.3. Report renderers

@cleak/common cung cấp 4 renderer: JSON (machine-readable), Markdown (human-readable), HTML (styled web-viewable), và snapshot (experiment comparison format kèm metadata). Mọi renderer đều nhận cùng một ScanReport object — đảm bảo tính nhất quán giữa các format.

3.7. Cài đặt tầng LLM-generalization (POLICY)

Nguyên tắc cốt lõi xuyên suốt hệ thống: **LLM sở hữu POLICY**, engine sở hữu **MECHANISM**. Phần này trình bày ba module nơi LLM đề xuất quyết định theo-từng-project, còn engine thực thi quyết định đó một cách tắt định.

3.7.1. Allocator profiler

allocatorProfiler.ts giải quyết bài toán: làm sao biết dự án nào dùng allocator nào mà không hardcoded?

Quy trình gồm 4 bước:

1. **Gather project text:** Duyệt file C/C++ trong dự án, ưu tiên header files (chứa public API) và file mang tên dự án (ví dụ cJSON.h, cJSON.c). Budget mặc

định 40,000 ký tự (~13k tokens). Source files bị cap 6,000 ký tự/file để một file lớn không chiếm hết budget.

2. **LLM inference:** Gửi project text cho model, yêu cầu liệt kê allocators, deallocators, reallocators, và ownership notes. Response được parse bằng Zod schema:

```
const AllocatorProfileSchema = z.object({
  allocators: z.array(z.string()).default([]),
  deallocators: z.array(z.string()).default([]),
  reallocators: z.array(z.string()).default([]),
  ownershipNotes: z.array(z.string()).default([]),
});
```

3. **Grep-verify:** Mỗi tên LLM đề xuất được verify bằng cách grep trong source — nếu tên không xuất hiện, nó bị loại. Bước này ngăn hallucination (LLM "bịa" ra tên hàm không tồn tại).

4. **Cache:** Kết quả được lưu vào `.cleak/allocator-profile.json` trong thư mục dự án. Lần scan sau không cần gọi LLM lại.

Trong thực nghiệm trên cJSON (mimo/mimo-v2.5-pro, temperature 0), allocator profiling đạt: Allocator Recall 85%, Deallocator Recall 100%. LLM phát hiện nhiều allocator hơn list hardcoded — cụ thể `cJSON_Parse()` và `cJSON_Print()` trả về owned memory mà hardcoded set bỏ sót.

3.7.2. Strategist

`strategist.ts` implement adaptive planning — LLM đọc metadata dự án và quyết định chiến lược phân tích. Bước đầu, `gatherRepoMetadata()` thu thập thông tin deterministic (không cần LLM): số file, tỉ lệ C++, build system nào (`CMakeLists.txt`, `Makefile`, `meson.build`...), và mật độ smart pointer (`unique_ptr`, `shared_ptr`, `g_object_ref`...).

Metadata + allocator profile được gửi cho model, yêu cầu trả về:

```
const StrategyPlanSchema = z.object({
  runDynamic: z.boolean(), // có chạy sanitizer không
  judge: z.enum(['single', 'consensus']), // single LLM hay consensus
  staticDepth: z.enum(['shallow', 'full']), // phân tích tĩnh nông hay sâu
});
```

Đây là module OPT-IN (`--strategy auto`). Trong benchmark, `strategist` bị bypass — config explicit được dùng để đảm bảo reproducibility.

3.7.3. Judge tuner

`judgeTuner.ts` cho phép LLM đề xuất ngưỡng phán xét phù hợp với dự án. Ví dụ: dự án dùng nhiều RAII/smart pointer → nâng ngưỡng `confirmed` lên (giảm FP); dự án `manual malloc/free` lộn xộn → hạ ngưỡng xuống (tăng recall).

Điểm mấu chốt: ngưỡng bị **clamp** cứng — `confirmed` nằm trong `[0.55, 0.85]`, `likely` trong `[0.25, 0.6]`, và `confirmed` luôn $>$ `likely`. LLM không thể làm judge quá nhạy (ngưỡng thấp) hay quá bảo thủ (ngưỡng cao). Trong benchmark, tuner cũng bị bypass — frozen defaults luôn được dùng.

3.8. Triển khai và vận hành

3.8.1. Docker Compose

Hai analyzer service được deploy qua Docker Compose:

```
services:
  static-analyzer:
    ports: ["127.0.0.1:50061:50061"]
    volumes: [".:/demo:/workspace/demo", "./targets:/workspace/targets"]
    networks: [mcpvul-net]

  dynamic-analyzer:
    ports: ["127.0.0.1:50062:50062"]
    volumes: ["runs:/runs", ".:/demo:/workspace/demo",
      "./targets:/workspace/targets"]
    networks: [mcpvul-net]
```

Port chỉ bind `127.0.0.1` — đây là dịch vụ nội bộ, không authenticated, không nên expose ra LAN. Dynamic analyzer có thêm named volume `runs` để persist analysis artifacts giữa các container restart.

Mỗi container đọc env từ `apps/<svc>/ .env` (optional — boots với defaults nếu file không tồn tại). Docker image bake sẵn `clang` và `clang-tools-extra` (cho `scan-build`) vào static-analyzer image.

3.8.2. LLM gateway

Thực nghiệm sử dụng local LLM gateway tại `http://localhost:20128/v1` với model `mimo/mimo-v2.5-pro`. Gateway tuân thủ OpenAI Chat Completions API — nên hệ thống dùng provider `local` (thực chất là OpenAI-compatible endpoint). Temperature mặc định: 0 (judge single), 0.7 (consensus sample).

3.8.3. Configuration hierarchy

Hệ thống hỗ trợ 4 nguồn config, theo thứ tự ưu tiên giảm dần:

1. **CLI flags:** `--provider`, `--max-turns`, `--consensus-n`, v.v.
2. **Environment variables:** `PROVIDER`, `MAX_TURNS`, `STATIC_URL`, v.v.
3. **Config file:** `~/ .config/cleak/config.json` — Zod-validated, chmod 600 (vì chứa API key).
4. **Built-in defaults:** Xem `configTemplate()` — ví dụ `maxTurns: 15`, `staticConcurrency: 3`, `staticGroupSize: 4`.

Config file được validate bằng Zod lenient parse — mỗi key được validate độc lập, key invalid bị bỏ qua với warning trên stderr thay vì reject toàn bộ file.

3.8.4. Corpus pipeline

Quản lý corpus đánh giá là một pipeline riêng, gồm 4 bước:

1. **Ingest:** `corpus/juliet/ingest.ts` copy file từ NIST Juliet Test Suite v1.3, group C++ multi-file cases, derive label từ convention `*bad*/*good*` trong tên file. `corpus/lamed/ingest.ts` clone repo từ Zenodo tại bug commit cụ thể.
2. **Validate:** `validate-corpus.ts` chạy 5 gates:
 - Gate 1: Zod schema validation (manifest format đúng)
 - Gate 2: Structural check (file tồn tại, không rỗng)
 - Gate 3: Compile check (`clang -fsyntax-only` pass)
 - Gate 4: Label overlap detection (không có file vừa bad vừa good)
 - Gate 5: Content-hash integrity
3. **Lock:** Tạo `<corpus>.lock.json` với source hash, ingest commit, clang version — đảm bảo reproducibility.
4. **Eval gate:** Chạy evaluation chỉ trên corpus đã validated + locked.

3.9. Tổng kết chương

Chương này đã trình bày chi tiết quá trình hiện thực hoá hệ thống từ thiết kế sang code. Một số điểm đáng chú ý:

Thứ nhất, quyết định viết toàn bộ hệ thống bằng TypeScript — thay vì C++ hay Python — xuất phát từ ràng buộc thực tế: MCP SDK chỉ có bản TypeScript, và orchestrator không cần hiệu năng tính toán cao (phần nặng nhất — parsing AST — đã do tree-sitter C library xử lý).

Thứ hai, cơ chế threading `extraAllocators/extraDeallocators` xuyên suốt 7 static tools là thay đổi có impact lớn nhất trong quá trình phát triển. Trước khi có cơ chế này, hệ thống hoàn toàn blind với non-libc allocator — dẫn đến 0% recall trên LAMeD benchmark (dự án thực không dùng malloc trực tiếp). Sau khi threading, recall trên LAMeD tăng từ 0/41 lên 12/41.

Thứ ba, deterministic evidence capture (Stage B wrapper) là yếu tố then chốt cho reproducibility. Bằng cách loại bỏ LLM discretion khỏi quá trình ghi nhận finding, hệ thống đảm bảo rằng cùng một sanitizer run luôn produce cùng một evidence — dù LLM quyết định tool nào gọi.

Phần tiếp theo (Chương 4) sẽ đánh giá hệ thống đã cài đặt này trên ba corpus (Juliet 1658 ca, LAMeD 41 ca và MemHint 19 ca), so sánh với Clang Static Analyzer, và phân tích đóng góp của từng thành phần qua ablation study.

Chương 4: Đánh giá kết quả

Mọi thiết kế đều là giả thuyết cho đến khi được kiểm chứng bằng thực nghiệm. Chương này trình bày kết quả đánh giá trên ba corpus: Juliet CWE-401 (corpus synthetic, 1658 ca), LAMeD benchmark (corpus dự án thực, 41 ca) và MemHint (corpus dự án thực tự tái lập, 19 ca), với các ablation study phân rã đóng góp từng thành phần. Số liệu được báo cáo trung thực, kể cả khi chúng không như kỳ vọng.

4.1. Thiết kế thực nghiệm

4.1.1. Corpus

Juliet CWE-401 validated: 1658 test cases, lockfile `f578c3ee`, 0 quarantined. Đây là phiên bản re-ingest từ NIST v1.3, đã fix lỗi C++ multi-file variant (422/1984 ca cũ không build được) và label drift (1171 ca cũ bị mislabel). Mỗi ca có ground truth: `flaws[]` (hàm leak) và `clean[]` (hàm sạch).

LAMeD benchmark: 41 leak từ 7 dự án thực (curl 14, libtiff 6, libsolv 6, cJSON 6, libxml2 4, libssh2 3, rabbitmq-c 2). Positive-only — không có label sạch nên chỉ đánh giá recall, FP count, FP/KLOC.

4.1.2. Scoring model

Function-mode cho Juliet: hàm bao là site. Bất kỳ finding nào trong hàm `bad` → positive prediction; trong hàm `good` → negative. Nhiều finding cùng hàm collapse thành một sample — không inflate TP/FP.

Verdict được tính là "flagged" (positive prediction) khi `verdict ∈ {confirmed_leak, likely_leak}`.

4.1.3. Metrics

- **Precision (P):** $TP / (TP + FP)$
- **Recall (R):** $TP / (TP + FN)$
- **F1:** harmonic mean of P and R
- **ECE (Expected Calibration Error):** đo calibration của confidence
- **FP/KLOC:** false positive per thousand lines of code

4.1.4. Sampling

--stratify chọn mẫu evenly qua deterministic round-robin theo functionalVariant. Điều này đảm bảo ngay cả sample nhỏ (n=50) cũng bao phủ tất cả 10 families (char, int, malloc, new, strdup, struct, twoIntsStruct, wchar, destructor, virtual). Top-N sẽ bị skewed: 50 ca đầu tiên là 100% family char.

4.1.5. Fairness rules

Baseline positive-only (Clang) chỉ enumerate leak finding → TN=0. Vì vậy bỏ specificity/MCC/accuracy khỏi bảng so sánh, chỉ so P/R/F1/FP count/FP/KLOC. Mọi hệ thống dùng cùng scoreCase trên cùng corpus.

4.1.6. LLM configuration

Các run chốt của luận văn dùng model deepseek-v4-flash qua gateway OpenAI-compatible (openai-compat) nội bộ tại port 20128. Cụ thể: sweep full-corpus 1658 ca (mục 4.2, commit 5eec8b1), bộ run LAMeD 2026-08-20 (mục 4.5), ablation consensus n=50 (mục 4.6) và validation allocator-profile (mục 4.9). Temperature 0 cho judge single, 0.7 cho consensus sampling. Idle-timeout: 75 giây.

Không phải mọi thí nghiệm đều dùng một model, và chúng tôi ghi rõ model của từng run thay vì gộp chung. Bảng n=50 stratified ở mục 4.2.3 là thí nghiệm đầu (2026-06), đo trên mimo/mimo-v2.5-pro, sau đó re-measure trên corpus đã validate. Hai sweep phụ cấu hình B6a trên full corpus dùng model khác để kiểm tra tính tổng quát: mimo-v2.5 cho F1 0.737, glm-5.2 (z-ai) cho F1 0.774 (run 1/2 hoàn tất, số derive từ per-case rows). Cùng cấu hình B6a trên cùng corpus f578c3ee, F1 dao động từ 0.737 đến 0.863 theo model: cấu hình không được "tune" theo một model duy nhất, nhưng mức F1 tuyệt đối thì phụ thuộc model. Chi phí giữa các provider không so sánh trực tiếp được với nhau.

4.2. Ablation 9-baseline capability (headline)

4.2.1. Thiết kế

Năm trục độc lập: [static, dynamic, planner, tool_selector, fusion]. Chín baseline B1–B7 khai báo bằng YAML, resolver ánh xạ flags thành engine knobs.

4.2.2. Kết quả full-corpus 1658 ca (headline)

Sweep 9 baseline trên toàn bộ corpus 1658 ca, commit 5eec8b1, model deepseek-v4-flash, chạy trên WSL2 với `--concurrency 16`. Các baseline dùng LLM (B4 đến B7) chạy 3 lần để đo mean $\pm$ std; hai run riêng lẻ (B6b/run-2 và B7/run-3) từng bị nhiễm lỗi fallback-judge im lặng, được phát hiện và chạy lại trước khi tính vào bảng. Bảng sau là kết quả gộp:

ID	Baseline	TP	FP	FN	TN	P	R	F1	tok/case	Chi phí
B1	Static only	1433	674	1145	2785	0.680	0.556	0.612	0	—
B2	Dynamic only	735	0	2283	0	1.000	0.244	0.392	0	—
B3	Rule-based ensemble	1686	674	892	2785	0.714	0.654	0.683	0	—
B4	LLM + static	2035	470	543	2989	0.812	0.789	0.801 $\pm$ 0.001	9.947	\$9.69
B5	LLM + dynamic	735	0	2282	0	1.000	0.244	0.392 $\pm$ 0.005	303	\$0.26
B6	LLM + all (no planner/sel)	2009	74	569	3390	0.964	0.779	0.862 $\pm$ 0.001	5.513	\$5.99
B6a	+ planner only	2010	72	568	3392	0.965	0.780	0.863 $\pm$ 0.001	6.155	\$6.63
B6b	+ tool_selector only	2017	104	561	3360	0.951	0.782	0.858 $\pm$ 0.003	30.590	\$26.07
B7	Full adaptive	2004	101	574	3363	0.952	0.777	0.856 $\pm$ 0.002	32.004	\$27.14

Tổng chi phí sweep: \$75.78 (B1, B2, B3 là no_llm, không định giá). MCC của B6a là 0.790, lấy làm mean của 3 run từ `variance.json`. Thứ hạng F1: B6a 0.863 $\approx$ B6 0.862 $>$ B6b 0.858 $\approx$ B7 0.856 $>$ B4 0.801 $>$ B3 0.683 $>$ B1 0.612 $>$ B5 0.392 $\approx$ B2 0.392.

Ba điều đáng đọc từ bảng. Thứ nhất, cùng sweep, cùng commit: B6a 0.863 so với B1 static-only 0.612, tức trên full corpus judge LLM thực sự giúp. Thứ hai, chi phí: B7 agentic tốn \$27.14, khoảng 4 lần B6a (\$6.63), để đạt F1 thấp hơn; tính trên token thì 32k/case so với 6.2k/case, khoảng 5 lần. Thứ ba, dynamic evidence tiếp tục là "FP killer" ở quy mô lớn: B4 (LLM + static, không dynamic) tạo 470 FP, thêm dynamic (B6) giảm còn 74 FP với F1 cao hơn. Kết luận rút ra ở n=50 vì thể tái lập trên corpus lớn gấp 33 lần, với model khác.

4.2.3. Kết quả n=50 stratified (thí nghiệm đầu, giữ làm phụ)

Bảng dưới là thí nghiệm đầu của chuỗi ablation (2026-06), chạy trên model mimo/mimo-v2.5-pro, stratified n=50, single run (mean 3 runs của B6a: F1

0.938±0.015, xem mục 4.8.2). Chúng tôi giữ nguyên làm dữ liệu phụ; câu chuyện đầy đủ về hai bộ số nằm ở mục 4.2.5.

ID	Baseline	TP	FP	FN	TN	P	R	F1	ECE	Token
B1	Static only	42	11	11	146	0.792	0.792	0.792	0.548	0
B2	Dynamic only	35	0	24	0	1.000	0.593	0.745	0.041	0
B3	Rule-based ensemble	48	11	5	146	0.814	0.906	0.857	0.161	0
B4	LLM + static	50	18	3	139	0.732	0.943	0.824	0.054	1.310.030
B5	LLM + dynamic	35	0	24	0	1.000	0.588	0.740	0.003	37.529
B6	LLM + all (no planner/sel)	48	1	5	156	0.973	0.899	0.935	0.129	455.434
B6a	+ planner only	48	1	5	156	0.973	0.906	0.938	0.125	463.047
B6b	+ tool_selector only	48	2	5	155	0.960	0.899	0.929	0.128	4.239.560
B7	Full adaptive	48	2	5	155	0.960	0.899	0.929	0.130	4.115.938

Tổng sweep: 10,6 triệu token. Riêng B6b + B7 (agentic) chiếm 8,36 triệu (79%).

4.2.4. Kết quả n=100

Từ cùng đợt thí nghiệm đầu (model mimo/mimo-v2.5-pro), mở rộng n=100, chỉ chạy các cấu hình không agentic:

ID	TP	FP	FN	TN	P	R	F1
B1	78	22	24	289	0.780	0.765	0.772
B3	90	22	12	289	0.804	0.882	0.841
B6	86	0	16	311	1.000	0.843	0.915
B6a	89	0	13	311	1.000	0.873	0.932
B7	88	2	14	309	0.978	0.863	0.917

Ở n=100, B6a đạt P=1.000 hoàn hảo (FP=0). Dynamic + LLM judge hoàn toàn loại bỏ false positive.

4.2.5. Hòa giải: stratified 0.938 so với full-corpus 0.863

Cùng cấu hình B6a, cùng corpus hash f578c3ee, mà F1 chênh 0.075 (0.938±0.015 ở n=50 stratified so với 0.863±0.001 ở full corpus). Khác ở đâu? Không phải ở model hay cấu hình, mà ở cách lấy mẫu. Mẫu stratified round-robin bốc đều 10 families, trong khi corpus thật bị lệch hẳn về họ C++ new: 588/1658 ca (35%), gấp đôi họ lớn kế tiếp. Kết quả full-corpus vì thế chịu ảnh hưởng của họ khó nhất nhiều hơn tỷ trọng mà mẫu cân bằng phản ánh.

Phân rã theo family trong sweep full-corpus cho thấy khoảng cách này là thật, không phải nhiễu. Với B6a, theo từng run (byFunctionalVariant): family

`malloc` đạt F1 0.991 ở run tốt nhất (mean 3 run: 0.975), còn `strdup` chỉ đạt 0.584 (mean: 0.576). Một cấu hình có thể gần như hoàn hảo trên một family và kém hơn 0.4 điểm F1 trên family khác; mỗi sample nhỏ 50 ca đều qua round-robin chỉ chứa ~5 ca mỗi family, nên hiệu ứng family bị pha loãng. Std ± 0.015 của B6a ở $n=50$ so với ± 0.001 ở full corpus cũng là tín hiệu cùng chuyện: biến động đến từ thành phần mẫu, không từ run LLM.

Cả hai con số đều đúng, chúng chỉ trả lời hai câu hỏi khác nhau. 0.863 là hiệu năng trên toàn corpus, con số nên dùng khi nói về hệ thống nói chung. 0.938 là ablation cấu phần trên mẫu cân bằng family, hợp lý khi so sánh tương đối giữa các cấu hình. Bài học phương pháp luận: sample stratified nhỏ làm kết quả LLM-judge trông tốt hơn trên corpus lệch family, và mọi so sánh cần ghi rõ cách lấy mẫu.

4.3. Ma trận 2×2 (LLM orchestration $\times$ Dynamic evidence)

Hai trục phân rã: LLM (`no_llm` vs `llm_assisted`) $\times$ Dynamic (`off` vs `on`). Kết quả trên 30 ca đầu:

	Static (<code>--dynamic off</code>)	+ Dynamic
no_llm	TP29 FP7 FN3 · P0.806 R0.906	TP29-30 FP7 FN2-3 · R0.906-0.938
llm_assisted	TP29 FP7 FN3 · P0.806 R0.906	TP29-30 FP7 FN2-3 · R0.906-0.938

Hai phát hiện quan trọng:

LLM không cải thiện Juliet. `no_llm static` $\equiv$ `llm_assisted static` — cả hai cho TP29 FP7 FN3. Lý do: Juliet produce "non-borderline" bundles, heuristic finalize hết, LLM judge không bao giờ engage.

Dynamic thêm recall, FP ổn định. FN 3 $\rightarrow$ 2 khi leak path thực sự chạy. FP=7 ở mọi cell — bằng chứng động xác nhận, không tạo FP mới.

Điều này không có nghĩa LLM vô dụng. Trên Juliet, điểm khác biệt nằm ở tầng enrichment cộng judge: cấu hình LLM tốt nhất (B6a 0.863 full corpus) hơn hẳn static-only B1 (0.612) trong cùng sweep. Còn LLM judge có lật được verdict trên dự án thực hay không, mục 4.5 trả lời bằng số liệu đo được, không phải kỳ vọng.

4.4. Ablation static evidence tools

Câu hỏi: 11 tool tĩnh, tool nào thật sự cần? Câu trả lời: không phải "cạnh tranh" mà là "bổ sung."

Static tools	TP	FP	FN	P	R	F1	ECE
none (candidateScan only)	42	11	11	0.792	0.792	0.792	0.548
+ functionSummary	42	11	11	0.792	0.792	0.792	0.492
+ pathConstraints	42	11	11	0.792	0.792	0.792	0.503
+ both (default)	50	13	3	0.794	0.943	0.862	0.555

Đây là phát hiện đáng chú ý nhất trong ablation: hai tool có tính **synergistic**. Mỗi tool riêng lẻ không cải thiện confusion matrix — nhưng kết hợp lại, recall tăng từ 0.792 lên 0.943 (+8 TP, FN 11→3). Lý do: path-sensitive heuristic cần CẢ function summary (alloc→free pairing scope) lẫn path constraints (guard reconciliation) để fire.

4.5. Đánh giá trên LAMeD — dự án thực

4.5.1. Kết quả trên denominator 50 site (bộ run 2026-08-20)

Số liệu dưới đây từ bộ run 2026-08-20, sau khi sửa một lỗi đếm site: `computeBundleId` cũ dùng pseudo-hash cắt cụt, gộp các candidate khác nhau thành một, làm thiếu cả recall lẫn denominator. Sau fix, corpus LAMeD cho 50 site chấm được (trước đây 44: libsolv 6→11 site, cJSON 6→7, 5 dự án còn lại không đổi).

Cấu hình	TP	FP	FN	Recall	Precision
Clang Static Analyzer (43 site)	0	0	43	0.000	—
no_llm mặc định (50 site)	15	0	35	0.300	1.000
no_llm + interproceduralFlow	15	0	35	0.300	1.000
llm_assisted (mean 3 runs, std = 0)	15	0	35	0.300	1.000

15 leak bắt được, FP=0, Clang bắt 0. Ba cấu hình của hệ thống cho confusion matrix giống hệt nhau, và điều đó nói lên hai điều. Một, `interproceduralFlow` không thêm giá trị đo được trên benchmark này: run mặc định và run `+interproceduralFlow` byte-identical trên cả 41 ca. Nguyên nhân đã được root-cause bằng lời gọi cô lập trực tiếp tới tool trên ca `merge_patch`: nó đếm được hai call site `cJSON_Delete(target)` reachable từ hàm và kết luận allocation đã được free, mà không kiểm tra hai free đó nằm trên hai nhánh loại trừ lẫn nhau (bug thật nằm ở nhánh lỗi thứ ba, không free nhánh nào). Đây là lỗ hổng path-insensitivity trong chính phép đếm alloc/free của tool, cùng lớp giới hạn với guard-subset reconciliation của judge nhưng nằm sớm hơn một tầng. Hai, `llm_assisted` cho kết quả y hệt

no_llm: judge LLM (deepseek-v4-flash, temp 0) thực sự được gọi từ 47 đến 149 lần mỗi run trên các bundle heuristic đánh dấu borderline, nhưng không lật verdict nào. Telemetry judge-path xác nhận LLM có chạy thật, không phải bị bỏ qua. Phù hợp với phát hiện ở Juliet: đòn bẩy chính của lớp leak này là dynamic evidence, không phải judging.

Về denominator 43 so với 50: hai hàng không dùng chung mẫu số, và đó là thiết kế của scorer chứ không phải lỗi. scoreCase suy ra tập site chấm được từ chính finding granularity của tool đang được chấm, nên site count là thuộc tính của từng tool, không phải ground truth ngoài chung cho mọi tool. Chi tiết lập luận nằm ở results/lamed-correction-2026-08-20-README.md, mục 4.

4.5.2. Case study: cJSON merge_patch

cJSON_merge_patch nhận tham số target, giải phóng trên đường thành công nhưng không trên đường lỗi. Đây là archetype của lớp leak path-sensitive + parameter-ownership mà heuristic judge nhắm tới: guard-subset reconciliation được thiết kế để phát hiện một free chỉ cover exit path khi tập guard condition của free đó là tập con của guard set trên đường exit. Trong bộ run 2026-08-20, ca này hiện là FN: như phân tích ở trên, interproceduralFlow kết luận "balanced" vì đếm đủ hai call site free mà không soi tính loại trừ của nhánh. Một phép đo trước đó trên denominator 44 site (trước fix computeBundleId) từng ghi +1 TP cho interproceduralFlow trên chính ca này (recall 0.250 → 0.273); phát hiện đó đã bị thu hồi sau khi verify cô lập như trên. Case study này vì thế giữ giá trị ở vai trò minh hoạ giới hạn: điểm yếu của tầng phân tích cross-function hiện tại không phải là "chưa gọi tool" mà là tool đếm alloc/free không path-sensitive.

4.5.3. Phân tích 6 leak classes cJSON

Đọc trực tiếp từ 6 fix-commit:

1. **Deallocator-semantics:** cJSON_Duplicate buffer cJSON_strdup gắn cờ cJSON_StringIsConst → cJSON_Delete bỏ qua không free. Cần mô hình hoá ngữ nghĩa deallocator.

2. **Deallocator-semantics:** cJSON_ReplaceItemInObject tương tự.

3. **Path-sensitive:** `merge_patch` thiếu `cJSON_Delete(target)` trên đường lỗi. Lớp DUY NHẤT có hi vọng bắt bằng path analysis.
4. **Path-sensitive:** `FindPointer` thiếu `cJSON_free(full_pointer)` trên đường lỗi.
5. **Control-flow:** `suffix_object` reorder + null-guard trước alloc — tinh vi.
6. **File-level:** leak trong setup/teardown test.

Sáu ca gộp thành bốn lớp cấu trúc khác nhau: deallocator semantics, path-sensitivity, control-flow reordering, và code file/test-only. 35 FN còn lại trên 50 site không quy về một thành phần yếu duy nhất, mà là thiếu chiều phân tích (deallocator semantics, interprocedural dataflow alias-aware). Cùng kết luận mà LAMeD tự báo: ngay cả các tool tiên tiến nhất cũng chỉ bắt 5-10/43 leak thật trong đánh giá của họ.

4.6. Ablation consensus judge

4.6.1. Thí nghiệm đầu: n=30

Chúng tôi kể theo đúng trình tự thí nghiệm đã chạy, kể cả khi kết quả cuối đi ngược kết quả đầu. Đợt đo đầu tiên dùng 30 ca đầu của corpus, tức 100% một family duy nhất (`char`, xem mục 4.2.5), với hai nhánh single-LLM ($K=1$) và consensus ($K=3$), hai lần chạy mỗi nhánh, lặp lại hai campaign A/B:

Judge arm	Campaign	Case stability	Flip rate	Modal agreement
single-LLM ($K=1$)	A	73.3%	26.7% (8/30)	86.7%
single-LLM ($K=1$)	B	86.7%	13.3% (4/30)	93.3%
consensus ($K=3$)	A	93.3%	6.7% (2/30)	96.7%
consensus ($K=3$)	B	93.3%	6.7% (2/30)	96.7%

Tại thời điểm đó, kết quả trông rất rõ: consensus arm lặp lại y hệt qua hai campaign (6.7% / 96.7% cả hai lần), single-LLM flip rate dao động 26.7% rồi 13.3%. McNemar trên 77 site của campaign B cũng nghiêng về consensus: consensus acc 83.1% / F1 0.822 so với single 79.2% / 0.784, 5 nghiêng consensus so với 2 nghiêng single, $\chi^2=0.57$, $p=0.45$. Chưa có ý nghĩa thống kê ở $n=30$, nhưng xu hướng thì một chiều.

4.6.2. Đo lại trên mẫu stratified n=50: kết quả đảo ngược

Bài học từ mục 4.2.5 khiến chúng tôi nghi ngờ chính mẫu n=30. Đo lại trên n=50 stratified round-robin (2 runs mỗi nhánh, model `deepseek-v4-flash`, commit `f0d371c`), 205 site chấm được. Hai campaign cho cùng một chiều kết quả, flip rate lặp lại y hệt:

Judge arm	Flip rate	F1
single-LLM (K=1)	2.0%	0.852
consensus (K=3)	8.0%	0.793

Mọi thứ đảo chiều so với n=30. Single-LLM nay là nhánh ổn định hơn (flip 2.0% so với 8.0%) và chính xác hơn (F1 0.852 so với 0.793), nhất quán ở cả hai campaign. McNemar paired trên 205 site: trong các site bất đồng, đa số nghiêng về single-LLM; $\chi^2=3.13$, $p=0.077$, xu hướng lean single nhưng chưa đạt ngưỡng ý nghĩa thống kê ở quy mô này.

4.6.3. Vì sao kết quả đầu sai, và kết luận

Hai phép đo đó không mâu thuẫn nhau; chúng đo hai mẫu khác nhau. Mẫu n=30 đầu (100% family `char`) là corpus dễ, nơi consensus sampling ở temp 0.7 có lợi thế; khi mức khó được phân bố đều qua 10 families, các ca khó thật (như family `strdup` với F1 0.584 ở mục 4.2.5) phơi bày điểm yếu của consensus: phiếu LLM trên ca khó không độc lập, và đa số phiếu có thể bầu cho sai một cách có hệ thống. Flip rate của single-LLM ở n=30 (từ 13.3% đến 26.7%) cũng không tái lập ở n=50 stratified (2.0%): phần lớn "bất ổn" trước đó là hiệu ứng family-dễ, không phải tính chất của judge.

Kết luận chính thức của luận văn: **consensus không được khuyến nghị làm mặc định**. Trên bằng chứng tốt nhất hiện có (mẫu đại diện phân bố khó, n=50, paired test 205 site), consensus vừa kém ổn định hơn vừa kém chính xác hơn single-LLM, chênh lệch chưa đạt ý nghĩa thống kê ($p=0.077$). Cơ chế vẫn giữ lại như tùy chọn opt-in (`consensus.n > 1`) chờ nghiên cứu multi-seed trên quy mô lớn hơn. Đây là kết quả âm tính có chủ đích: cơ chế nằm trong tên đề tài, được đánh giá nghiêm túc, và bị từ chối bởi chính giao thức đánh giá của luận văn.

4.7. So sánh với baseline bên ngoài

4.7.1. Juliet n=30 (live re-run, cùng scorer)

Bảng so sánh trực tiếp trên 30 ca đầu (thí nghiệm sớm, cùng một mẫu đơn-family như mục 4.6.1), chạy lại cả hai hệ trên cùng scorer:

Hệ thống	Sites	TP	FP	FN	TN	P	R	F1
no_llm (heuristic)	77	29	7	3	38	0.806	0.906	0.853
clang-analyzer	44	27	12	5	0	0.692	0.844	0.761

Chú thích về cột "Sites": đây là số site cấp phát chấm được trên 30 ca đó (mỗi ca có thể chứa nhiều site), không phải một số ca thứ hai. Output của Clang Static Analyzer (chỉ warning, không có clearance per-site tường minh) tạo ra một tập site nhỏ hơn và không trùng; TN=0 của hàng Clang là cấu trúc đặc trưng của công cụ positive-only. Trên mẫu này, câu hình heuristic thắng Clang về cả F1 lẫn mật độ FP. So sánh ở quy mô full corpus nằm ở bảng mục 4.2.2.

4.7.2. LAMeD

Hệ thống	Sites	TP	FP	Recall	Precision
Hệ thống (no_llm = llm_assisted)	50	15	0	0.300	1.000
clang-analyzer	43	0	0	0.000	—

Recall 30.0% với FP=0 nằm trong dải LAMeD tự báo cho các công cụ có annotation (5-10/43 leak). Clang raw = 0 vì `unix.Malloc` không mô hình hoá factory allocator. Chênh lệch denominator 43 so với 50 là by-design của scorer (mỗi hệ được chấm trên tập site do chính finding granularity của nó suy ra), đã phân tích ở mục 4.5.1 và chi tiết tại `results/lamed-correction-2026-08-20-README.md`, mục 4.

4.7.3. So sánh với các hệ thống LLM khác

Bảng sau so leak-count với các hệ thống dùng LLM cho leak C/C++. Lưu ý: corpus và giao thức chấm khác nhau, nên so sánh chỉ mang tính định hướng, không dùng để kết luận hệ thống nào "tốt hơn."

Hệ thống	Corpus	Kết quả	Phương pháp	Peer-review
MemHint [20]	7 dự án thực, 3.4M+ SLOC	52–54 leak tìm thấy (49 confirmed/fixed, 4 CVE)	LLM + Z3 + CodeQL/Infer	Không

Hệ thống	Corpus	Kết quả	Phương pháp	Peer-review
LAMeD [21] (annotation)	cJSON, 152 hàm	P 0.933 / R 0.583 (28 TP / 2 FP / 20 FN)	LLM sinh AllocSource/FreeSink annotation	Có
LAMeD [21] (Cooddy)	6 dự án thực, 43 leak	5→10 phát hiện	Cooddy tiêu thụ annotation	Có
LAMeD [21] (CodeQL)	6 dự án thực, 43 leak	5→10 phát hiện	CodeQL tiêu thụ annotation	Có
Hệ thống luận văn	LAMeD, 41 ca (50 site)	15 TP / 0 FP	no_llm và llm_assisted cho kết quả như nhau	—
Hệ thống luận văn	Juliet, full corpus 1658 ca	2010 TP / 72 FP (B6a)	LLM orchestration + static + dynamic	—

MemHint [20] đạt leak-count cao nhất (52–54, 49 confirmed/fixed) nhưng trên corpus lớn hơn nhiều (7 dự án, 3.4M+ SLOC) và chưa qua peer-review. Bảng dùng số tự báo bản v3 của paper (7 dự án, 3.4M+ SLOC, 52–54 leak, 49 confirmed — kiểm chứng trong `researchs/04`); bộ số "8 dự án / 3.6M LOC / 54 leak, 53 confirmed" thuộc bản arXiv đầu và đã bị bác bỏ. Điểm chung đáng nói: cả MemHint và hệ thống này đều cần LLM khám phá allocator, MemHint dùng LLM phân loại hàm, còn luận văn dùng LLM profiler với `grep-verify` (độ chính xác đo ở mục 4.9).

Với LAMeD [21] trên cJSON: Cooddy + annotation đạt $P=0.933$ / $R=0.583$. Khoảng cách với kết quả của hệ thống trên benchmark LAMeD chủ yếu do Cooddy có annotation function-level chi tiết hơn (AllocSource/FreeSink) so với allocator set đơn giản hơn của hệ thống. Đây chính là động lực cho tầng LLM allocator profiler, và là lý do cJSON được chọn làm scope validation của tầng đó.

4.8. Xác minh tái lập

4.8.1. Tier-1

Hai lần chạy no_llm (thư mục tách biệt, cùng cấu hình): TP29 FP7 FN3 TN38 y hệt. Gate `determinism-gate.sh` PASS.

4.8.2. Tier-2

Sweep full-corpus 1658 ca (mục 4.2.2, commit 5eec8b1, model `deepseek-v4-flash`), B6a chạy 3 lần: F1 mean 0.863 ± 0.001 , P mean 0.965 ± 0.002 , R mean 0.780 ± 0.001 , MCC mean 0.790 ± 0.002 . B6b ± 0.003 và B7 ± 0.002 tương tự nhỏ. Trên mẫu stratified $n=50$ (thí nghiệm đầu, model khác), B6a 3 runs cho F1 mean 0.938 ± 0.015 .

Variance nhỏ nhưng không phải không: ± 0.001 trên full corpus đủ chặt để kết luận thứ hạng giữa các cụm cấu hình (B6/B6a so với B6b/B7) tái lập được, trong khi ± 0.015 của mẫu $n=50$ nhắc rằng biến động thành phần mẫu lớn hơn biến động run LLM. LLM judge đủ ổn định cho practical use, nhưng không đủ cho bitwise reproducibility claim.

4.9. Độ chính xác LLM allocator profiling

Chạy `validate-allocator-profile.ts` trên toàn bộ 7 dự án LAMeD, model `deepseek-v4-flash temp 0` (2026-08-19). Trên cJSON, scope mà paper LAMeD cite, profiler đạt:

- **Allocator: P 35% / R 92%, F1 0.51.**
- **Deallocator: P 67% / R 100%, F1 0.80.**

Recall cao, precision thấp. Phần "false positive" khi soi thủ công thực ra là allocator thật mà list hardcoded bỏ sót, ví dụ `cJSON_Parse()`, `cJSON_Print()` trả owned memory. Nghĩa là LLM đầy đủ hơn list người viết tay; cái sai chủ yếu là hướng ngược lại (gắn nhãn allocator cho hàm không cấp phát). Ownership notes cũng chính xác: ví dụ `cJSONUtils_FindPointerFromObjectTo` trả chuỗi freed bằng `cJSON_free`.

Tuy nhiên, gộp trên cả 7 dự án thì số liệu thấp hơn hẳn: allocator P 24% / R 21% (F1 0.22), deallocator P 16% / R 19% (F1 0.17). cJSON là dự án nhỏ với naming convention đồng nhất (`cJSON_*`); các dự án còn lại dùng wrapper allocator đa hình hơn nhiều. Đây là giới hạn thật của tầng profiler trên dự án thực, và nó giải thích một phần vì sao recall trên LAMeD dừng ở 30% thay vì cao hơn: tầng phát hiện allocator, đầu vào của mọi tầng sau, chưa bám được các convention này.

4.10. Đánh giá trên corpus MemHint

MemHint [20] là baseline rõ ràng bộ nhớ C/C++ trực tiếp nhất trong related work: phân tích tĩnh neuro-symbolic kết hợp Z3 và bước xác nhận LLM, đo trên tập dự án thực. Danh sách leak từng ca của paper không được công bố, nên ground truth của họ không thể tái dùng trực tiếp. Chúng tôi vì thế tự dựng lại corpus: 19 ca rò rỉ từ 6 dự án thực (tmux, curl, openssl, redis, vim, freerdp), trùng 6 trong 7 dự án mục tiêu mà [20] liệt kê (thiếu FFmpeg). Mỗi ca gắn `github_url` trỏ đúng commit sửa lỗi, lưu trong

demo/memhint/memhint_bugs.json để kiểm chứng độc lập. Corpus là positive-only như LAMeD: 26 site rò trên 19 ca, không có nhãn site sạch, vì thế recall và số FP là hai đại lượng chấm được (mục 4.1.5). Toàn bộ chạy trên WSL2 qua driver scripts/memhint-eval-driver.sh, commit 30e04cb1c, lockfile 442de35d, môi trường clang 14.0.0 và valgrind 3.18.1; artifacts nằm tại results/memhint-no_llm-2026-08-28/ và results/memhint-llm_assisted-2026-08-28/.

4.10.1. Kết quả

Hai cấu hình được đo: no_llm --enrich (1 run) và llm_assisted (3 runs, model deepseek-v4-flash qua openai-compatible, temp 0). Hệ chạy ở cấu hình đề xuất llm_assisted (nhóm fusion-B6 trong ablation, mục 4.2): planner và tool-selector là hai trục chỉ được bật trên Juliet, còn lệnh trên corpus này dùng --strategy off và --no-tool-select.

Cấu hình	TP	FP	FN	Recall	Precision
no_llm --enrich (1 run)	12	0	14	0.462	1.000
llm_assisted (mean 3 runs, std = 0)	12	0	14	0.462	1.000

Hệ bắt 12 trên 26 site rò, không tạo FP nào; ba run llm_assisted cho confusion matrix giống hệt nhau $\{(12, 0, 14)\}$, tức $P\ 1.000 \pm 0.000 / R\ 0.462 \pm 0.000 / F1\ 0.632 \pm 0.000$. Sự đồng nhất này có nguyên nhân đo đếm được chứ không phải trùng hợp. Judge hybrid chỉ gọi LLM trên các bundle heuristic đánh dấu borderline, mà judge heuristic tự quyết khoảng 17,1 nghìn flagged verdict mỗi run; LLM judge chỉ can thiệp trên 2 site trong đúng 1/3 run (run 2, ca freerdp_9fc23ad2, judge paths {heuristic: 17154, llm: 2}), hai run còn lại đi thuần nhánh heuristic. Không một verdict nào bị lật. llm_assisted vì thế $\equiv$ no_llm trên corpus này, cùng mẫu null với LAMeD (mục 4.5.1): LLM judging chỉ thêm giá trị khi bundle borderline thực sự tồn tại. Token vẫn tiêu, khoảng 44,3 triệu cho ba run llm (vào khoảng 26,5 triệu, ra khoảng 17,8 triệu), nhưng đổ vào allocator profiling và static fan-out chứ không tác động tới verdict.

4.10.2. Đối chiếu với số liệu MemHint tự báo

Trên tập dự án đầy đủ của mình, MemHint [20] tự báo 52-54 leak trên 7 dự án thực (3.4M+ SLOC), 49 ca confirmed/fixed, 4 CVE; cùng thiết lập đó, CodeQL tìm 19 và Infer 3. Corpus ở mục này chỉ là tập con 19 ca tự tái lập trên 6/7 dự án đó, chấm theo site với ground truth tự dựng, nên hai tập số không nằm trên cùng một mẫu số. So sánh vì thế chỉ mang tính định hướng: điểm dừng được là hệ đạt FP=0 với recall 46.2% trên lớp dự án thực tương tự, chứ không phải xác nhận hay bác bỏ con số của [20]. Paper này cũng chưa qua peer-review tại thời điểm viết.

4.10.3. Vì sao không chạy đủ 9 baseline trên corpus này

Protocol trên corpus MemHint chỉ gồm `no_llm` và `llm_assisted`, không lặp ablation 9 baseline như trên Juliet. Quyết định này có ba căn cứ, xếp theo sức mạnh tăng dần.

Thứ nhất, sức mạnh thống kê. Corpus positive-only 19 ca với 26 bug: TN=0 nên MCC không định nghĩa, và hiệu ứng có thể đo được giữa hai cấu hình là chênh 1-2 TP, tương đương 3.8-7.7 điểm recall trên mẫu số 26 site, nằm trong vùng nhiễu. Ablation 9 cấu hình chỉ có ý nghĩa trên corpus có nhóm đối chứng đủ lớn; Juliet 1658 ca với hơn 3.000 TN là nơi duy nhất trong luận văn thoả điều kiện đó.

Thứ hai, nhất quán protocol với LAMeD. Benchmark dự án thực trước đó (50 site, mục 4.5) cũng chỉ chạy hai công đoạn: `no_llm` mặc định, một biến thể `interproceduralFlow`, và `llm_assisted` ×3; 9 baseline không hề được chạy trên đó. MemHint theo đúng giao thức này.

Thứ ba, chi phí cộng precedent null. Dynamic stage trên repo thật chạy serialize với concurrency 1 (ca redis gồm 2235 files), mỗi cấu hình thêm khoảng 0.5-1 ngày build và sanitizer run, đủ 9 cấu hình tốn cỡ một tuần máy. LAMeD đã chỉ ra `llm_assisted` $\equiv$ `no_llm` trên dự án thực, nên các biến thể LLM còn lại (B4-B7) gần như chắc chắn tái tạo null đó. Run MemHint này vừa xác nhận thêm bằng chứng: LLM judge chỉ can thiệp 2 site trên khoảng 17,1 nghìn quyết định verdict, không lật verdict nào.

4.10.4. Threats to validity

Áp lực bộ nhớ là giới hạn vận hành lớn nhất. Ca redis đẩy static analyzer vào dải OOM (khoảng 14GB RSS), nên driver phải thiết kế 8 lần retry với `--resume`; một lần VM reboot giữa run làm mất attempt đầu tiên, run chốt hoàn tất ở attempt 3/8 và artifacts được sha256-verify byte-identical qua reboot. Lần chạy đầu (2026-08-28) vì thế chỉ chấm được 11 ca, để lại anchor recall 42.3% (11/26) với 8 ca mất vì OOM; run chốt khôi phục đủ 19/19 và recall lên 46.2% (12/26). Vì `no_llm` bitwise deterministic, 11 ca đã chấm trước đó phải cho kết quả y hệt, nên +1 TP chắc chắn đến từ các ca recovered chứ không phải thay đổi code; thư mục kết quả vẫn giữ tên mốc 2026-08-28 của lần chạy đầu.

Phân bố FN cũng lệch: khối lớn nhất nằm ở vim, 3 ca với 2 FN mỗi ca trên tổng 14 FN. Cuối cùng, ground truth là tập tự tái lập 19 ca, không phủ toàn bộ tập leak của 7 dự án mà [20] nhầm tới; kết quả ở mục này đo hệ thống trên lớp dự án thực tương tự, không phải tái lập đánh giá của paper.

4.11. Tổng hợp chương

Bảng tổng hợp toàn bộ kết quả

Corpus	Cấu hình tốt nhất	F1	P	R	Ghi chú
Juliet full 1658 ca	B6a	0.863	0.965	0.780	MCC 0.790, \$6.63; sweep 9 baseline
Juliet n=50 stratified (thí nghiệm đầu, model cũ)	B6a	0.938	0.973	0.906	Phụ: ablation cấu phần, mẫu cân bằng family
LAMeD (50 site, positive-only)	no_llm = llm_assisted	—	1.000	0.300	TP15/FP0; Clang 0/43 site
MemHint (26 site, positive-only)	no_llm = llm_assisted	0.632	1.000	0.462	TP12/FP0; 19 ca tự tái lập, 6 dự án thực
Consensus K=3 (n=50 stratified)	không khuyến nghị	0.793	—	—	Thua single (F1 0.852, flip 2.0% vs 8.0%), p=0.077

Hai kết luận nổi lên từ bảng. Dynamic evidence là "FP killer" ở mọi quy mô: thêm dynamic giảm FP từ 470 xuống 74 trên full corpus (B4→B6), hiệu ứng này tái lập đúng mẫu 18→1 FP ở n=50. Ngược lại, consensus là kết quả âm tính có phương

pháp luận: cơ chế chỉ thắng trên mẫu đơn-family lệch dễ, và thua trên mẫu đại diện, một phát hiện về hiệu ứng sampling mà nghiên cứu LLM-judge trước đó ít để ý.

Hai corpus dự án thực nói cùng một điều và nói theo cùng cách: trên LAMeD lẫn MemHint, ba cấu hình của hệ hội tụ về đúng kết quả no_llm, LLM judge được gọi nhưng không lật verdict nào, và recall dừng ở mức tầng allocator profiling cho phép (30.0% so với 46.2%). Null result lặp lại trên hai corpus độc lập là dữ kiện mạnh hơn một null result đơn lẻ: giá trị của LLM orchestration trước hết nằm ở tầng discovery và enrichment, còn judging chỉ hoạt động khi bundle borderline thật sự tồn tại, như trên Juliet full corpus.

Điều kiện nào LLM orchestration có lợi?

Từ kết quả trên ba corpus, có thể rút ra ba điều kiện:

Corpus khó, bundle borderline, nhưng còn phụ thuộc lớp allocator profiling.

Trên Juliet, heuristic finalize phần lớn bundle, nên judge LLM làm việc chủ yếu ở tầng đánh giá; cấu hình LLM tốt nhất (B6a 0.863) hơn static-only (B1 0.612) nhờ enrichment cộng judge đúng chỗ. Trên LAMeD, mọi cấu hình LLM hội tụ về đúng kết quả no_llm (LLM được gọi từ 47 đến 149 lần mỗi run, không lật verdict nào): khi tầng discovery đã bỏ sót site (recall profiling thấp trên dự án thực), judge không thể cứu những gì không bao giờ thành bundle. LLM orchestration có lợi trước hết khi candidate đúng đã vào pipeline.

Cần cross-function reasoning, và tầng đó phải path-sensitive.

interproceduralFlow hiện tại $\Delta=0$ trên cả Juliet lẫn LAMeD: tool đếm alloc/free không phân biệt nhánh, nên kết luận sai trên ca merge_patch. Bài học rộng hơn con số: orchestration chỉ hữu ích khi tool trong pipeline mạnh hơn heuristic nó bổ trợ; gọi một tool path-insensitive để xử lý leak path-sensitive chỉ tốn token.

Chi phí phải được kiểm soát, và agentic phải trả giá bằng kết quả. B6a là điểm ngọt: \$6.63 cho F1 0.863. B7 agentic tốn \$27.14 (khoảng 4 lần) cho F1 thấp hơn, B6b tương tự, trên cả hai quy mô mẫu và hai model. Cấu hình sản xuất là B6a; agentic tool-selection chỉ đáng cân nhắc khi exploration thật sự cần thiết, và bằng chứng hiện tại chưa chỉ ra corpus nào thuộc nhóm đó trong phạm vi luận văn.

Chương 5: Kết luận và hướng phát triển trong tương lai

Câu hỏi khởi nguồn của luận văn nghe rất đơn giản: liệu một LLM điều phối công cụ phân tích tĩnh và động có phát hiện memory leak tốt hơn việc dùng từng công cụ một? Trả lời câu đó đòi hỏi bốn chương: bối cảnh và các hướng tiếp cận hiện có (Chương 1), kiến trúc HYBRID (Chương 2), hiện thực hoá hệ thống (Chương 3), đánh giá thực nghiệm trên hai loại corpus (Chương 4). Chương cuối này chốt lại câu trả lời cho bốn câu hỏi nghiên cứu trên số liệu đã chốt, khái quát bốn đóng góp, rồi bàn về những gì hệ thống chưa làm được cùng hướng đi tiếp.

5.1. Trả lời câu hỏi nghiên cứu

RQ1: LLM orchestration có cải thiện hiệu suất phát hiện leak không?

Có, trên toàn bộ corpus, với điều kiện LLM đứng đúng vị trí. So sánh quyết nằm trong cùng một sweep 9 baseline, cùng commit `5eec8b1`, trên đủ 1658 ca Juliet CWE-401 (mục 4.2): B6a, cấu hình đề xuất gồm planner, recipe động tắt định và LLM judge, đạt F1 0.863 với precision 0.965, recall 0.780, MCC 0.790. Baseline tĩnh thuần B1 trong cùng sweep chỉ đạt F1 0.612. Một run `no_llm postfix4` độc lập (commit `c07bee5`) cho đúng bộ số precision 68.0%, recall 55.6%, F1 0.612, xác nhận chéo cho mốc baseline này. Chênh lệch 0.251 F1 vì thế đo được trên cùng corpus, cùng scorer, cùng thể hệ code.

Xếp hạng đầy đủ của sweep: B6a 0.863, B6 0.862, B6b 0.858, B7 0.856, B4 0.801, B3 0.683, B1 0.612, rồi B5 và B2 cùng 0.392. Bốn cấu hình đứng đầu đều có LLM, nhưng hai cấu hình cuối cùng lại tụt dưới cả baseline tĩnh. Sự phân tán đó chính là thông điệp: thêm LLM vào không tự động làm hệ tốt hơn. Giá trị đến từ cách bố trí, planner gate luồng chạy, recipe ghim cứng phần chạy động, judge chỉ chốt verdict cuối.

Chi phí đi kèm cũng phân hoá theo thiết kế. B7, nhánh agentic đầy đủ, tốn 27.14 USD, khoảng 4 lần B6a (6.63 USD), đổi lại F1 thấp hơn (0.856 so với 0.863). Trên corpus synthetic như Juliet, việc thăm dò công cụ theo kiểu agentic phần lớn gọi lại những công cụ mà bằng chứng tĩnh đã đủ để kết luận.

Câu trả lời RQ1 vì thế là có, nhưng có điều kiện: nhánh judge LLM trong cấu hình đề xuất nâng F1 từ 0.612 lên 0.863 trên toàn corpus; các cách dùng LLM khác, tiêu biểu là tool selection agentic, có thể không chỉ đắt hơn mà còn kém hơn.

RQ2: Kết quả có tái lập được không?

Mỗi chế độ chạy được báo cáo theo đúng bản chất tất định của nó, tạo thành giao thức hai tầng (mục 4.8). Tầng 1 áp dụng cho no_llm, tầng 2 cho llm_assisted; hai tầng không ép về một chuẩn chung.

Tier 1 phải tất định tuyệt đối, và thực sự như vậy. Hai lần chạy no_llm tách thư mục, cùng cấu hình, cho bằng nhằm lẫn trùng từng ca: TP29, FP7, FN3, TN38. Gate determinism-gate.sh chứng nhận điều này và đồng thời từ chối hai kiểu đầu giả đã gặp thật trong quá trình phát triển: so thư mục với chính nó vì trùng dấu thời gian, và một run degenerate (mất kết nối analyzer nên mọi ca lỗi) dễ bị nhầm là "tất định". Trên toàn corpus, run no_llm postfix4 chốt mốc tầng 1: precision 68.0%, recall 55.6%, F1 0.612, MCC 0.375.

Tier 2 không thể đồng nhất bit-for-bit vì judge LLM mang sampling phía provider, nên số liệu được báo cáo dạng phân phối. Sweep chính chạy 3 lần cho mỗi cấu hình LLM: B6a cho $F1\ 0.863 \pm 0.001$. Trên LAMeD, ba run llm_assisted cho std=0 (mục 4.5). Sweep stratified n=50 ghi B6a 0.938 ± 0.015 (mục 4.2). Kèm mean±std, tỉ lệ lật verdict từng ca được đo riêng qua verdict-stability.ts, vì tổng hợp có thể trùng nhau do may trong khi verdict từng ca vẫn dao động.

Một run llm_assisted thuộc thế hệ commit cũ hơn (9db30f0, sinh ngày 2026-08-11, trước chuỗi fix reference-out-param, virtual-dispatch và RAII của 2026-08-12) được giữ lại chỉ cho dữ liệu chi phí và đường judge: 66.4% verdict bị flag do heuristic chốt, 33.6% do LLM chốt. Run này không tham gia bất kỳ so sánh hiệu suất nào vì lệch thế hệ code so với sweep chính.

RQ3: Consensus voting có giảm dao động verdict không?

Thí nghiệm đầu nói có. Trên mẫu n=30, nhánh single-LLM lật verdict 13.3 đến 26.7% giữa các lần chạy, nhánh consensus K=3 lặp lại y hệt 6.7% qua hai đợt; McNemar cho p=0.45, xu hướng nghiêng consensus nhưng chưa đạt ý nghĩa thống kê. Khi đó, kết luận "consensus ổn định hơn 2 đến 4 lần" có vẻ vững.

Mẫu $n=30$ đó lại vô tình gồm 100% family char, do manifest Juliet xếp theo family mà lấy đầu danh sách không stratify. Lặp đúng protocol trên mẫu $n=50$ stratified đủ 10 family (commit f0d371c, mục 4.6) đảo chiều hoàn toàn kết quả:

Nhánh judge	Tỉ lệ lật verdict	F1
Single-LLM ($k=1$)	2.0%	0.852
Consensus ($k=3$)	8.0%	0.793

Hai đợt chạy độc lập cho cùng số trên. McNemar trên 205 site ghép cặp: 7/8 cặp bất đồng nghiêng về single, $\chi^2=3.13$, $p=0.077$.

Điều thú vị là bảng này không nói consensus tệ, mà nói kết quả đầu tiên là sản phẩm của cách lấy mẫu. Khi ba mẫu độc lập gần như luôn đồng ý, bỏ phiếu đang "ổn định hoá" thứ chưa từng bất ổn. Trên mẫu đại diện đủ family, chính single-LLM ổn định hơn và chính xác hơn; mức $p=0.077$ vẫn dưới ngưỡng ý nghĩa nên luận văn dừng ở "xu hướng nghiêng single", không khẳng định mạnh hơn dữ liệu cho phép.

Kết luận RQ3: consensus không được khuyến nghị bất mặc định. Cơ chế vẫn là code thật, vẫn opt-in qua `consensus.n > 1`, và luận văn báo cáo cả hai kết quả thay vì chỉ giữ số thuận lợi. Với nghiên cứu LLM-judge nói chung, đây là một ví dụ cụ thể cho việc hiệu ứng sampling có thể lật chiều một ablation.

RQ4: Hệ thống hoạt động ra sao trên dự án thực?

Trên LAMeD, 41 ca từ 7 dự án C thật với 50 site rò sau khi chỉnh lại denominator (mục 4.5), hệ thống bắt TP15, FP0, FN35, recall 30.0%. Ba lần chạy `llm_assisted` cho `std=0` và trùng khớp từng ca với `no_llm`. LLM judge thực tế được gọi từ 47 đến 149 lần mỗi run nhưng không lật verdict nào; thêm `interproceduralFlow` vào đường evidence tĩnh cũng cho kết quả byte-identical ($\Delta=0$). Clang Static Analyzer trên corpus này bắt 0 trên 43 site; hai mẫu số 43 và 50 khác nhau do scorer chấm mỗi hệ trên granularity finding của chính nó, không phải lỗi đo (chi tiết ở mục 4.7).

Ca dự án thực đầu tiên hệ bắt được có giá trị minh hoạ riêng: leak tham số target của `cJSON_merge_patch`, rò trên đúng một đường lỗi, được phát hiện nhờ scoring parameter-ownership kết hợp guard-subset reconciliation (kể chi tiết ở mục 4.5.2). Ba mươi lăm site còn sót thuộc các lớp khó: hai ca cần mô hình hoá deallocator semantics, hai ca path-sensitive, phần còn lại là ownership interprocedural phức tạp. Chính LAMeD cũng chỉ báo 5-10 trên 43 site cho các công cụ có annotation.

Corpus thực thứ hai là MemHint: 19 ca trên 6 dự án lớn (vim, redis, openssl...), ground truth tự tái lập vào file JSON vì danh sách bug của paper gốc không công bố. Giao thức gồm một run `no_llm` kèm enrich và ba run `llm_assisted`, điều phối bởi `scripts/memhint-eval-driver.sh` với resume tự động qua từng lần lỗi OOM. Tại thời điểm viết chương này, hai nhóm run MemHint chưa hoàn tất; số liệu [PENDING, bổ sung vào mục tương ứng của chương 4 khi run xong] và không còn số nào được suy đoán cho corpus đó.

Cả hai corpus thực dùng quy ước positive-only: recall và số FP là hai đại lượng so sánh được, còn precision và MCC không định nghĩa vì $TN=0$ theo cách dựng corpus (mục 4.1.2). $FP=0$ trên 50 site vì thế là kết quả mạnh, nhưng phải đọc kèm recall 30%.

5.2. Đóng góp của luận văn

Bốn đóng góp dưới đây xếp theo thứ tự trọng tâm sau khi mọi kết quả đã chốt: pipeline tắt định-trừ-judge là novelty trung tâm, còn consensus được trình bày như một kỹ thuật đã điều tra kỹ với kết quả âm trung thực.

C1: Pipeline hybrid tắt định-trừ-judge. Tầng dynamic được tắt định hoá triệt để: recipe build cộng sanitizer run ghim cứng (`buildTarget` $\rightarrow$ `lsanRun`), không LLM trong vòng chạy; mọi finding được capture tắt định vào store; planner LLM chỉ gate luồng, không tự ghi evidence. Judge trở thành nơi duy nhất còn sampling, và cấu hình đạt kết quả full-corpus mạnh nhất chính là cấu hình đó: B6a với F1 0.863, MCC 0.790. Trên corpus thực, pipeline chạy ba lần cho `std=0`.

C2: Giao thức tái lập hai tầng. Tier 1 (`no_llm`) tắt định bit-for-bit, được gate chứng nhận và biết từ chối hai kiểu đầu giả. Tier 2 (`llm_assisted`) báo cáo phân phối: `mean $\pm$ std` qua nhiều lần chạy, cộng tỉ lệ lật verdict từng ca. Mỗi chế độ được đo theo bản chất của nó; một tổng số trùng khớp không còn có thể che giấu dao động cấp ca.

C3: Làm giàu bằng chứng cho judge. Mỗi bundle mang ownership của biến, cặp `alloc $\rightarrow$ free`, `feasible-leak-path` và `correlationMethod` phân biệt tương quan chặt (khớp file, dòng, hàm) với tương quan yếu (chỉ file). Mọi verdict truy vết được về bằng chứng trong `snapshot.json` và hiển thị trong báo cáo cùng TUI. Allocator

annotation theo dự án, do LLM profiler đọc source rồi grep-verify, mở rộng discovery sang factory allocator kiểu `cJSON_Duplicate` mà danh sách hardcoded bỏ sót.

C4: Consensus judge với kết quả âm trung thực. Cơ chế k mẫu độc lập, precision-override veto và escalation theo bất đồng static-dynamic là code thật, đã tối ưu chi phí bằng batch judging. Trên mẫu đại diện $n=50$, nó thua cả về ổn định lẫn F1 so với single-LLM; luận văn báo cáo cả hai kết quả, không khuyến nghị mặc định, và rút ra bài học phương pháp luận về sampling. Kết quả âm được kể đủ là một đóng góp, không phải điểm yếu cần giấu.

5.3. Bàn luận

Hai cách lấy mẫu, hai con số

Sweep stratified $n=50$ cho B6a 0.938 ± 0.015 trong khi sweep toàn corpus cho 0.863 ± 0.001 (mục 4.2). Hai con số không mâu thuẫn; chúng trả lời hai câu hỏi khác nhau, một về ablation component trên mẫu cân bằng family, một về hiệu năng toàn corpus. Family variance đo trên chính các run B6a của sweep toàn corpus giải thích khoảng cách: F1 0.991 trên family malloc nhưng chỉ 0.584 trên strdup. Bài học lấy mẫu này chảy thẳng vào kết quả consensus ở RQ3, nơi cùng một cơ chế đổi chiều chỉ vì mẫu đổi từ ngẫu nhiên thành stratified.

Chi phí của agentic

Sweep 9 baseline tốn 75.78 USD tổng cộng, trong đó B7 agentic chiếm 27.14 USD, khoảng 4 lần B6a, để đạt F1 thấp hơn. Điểm ngọt nằm ở B6a: planner gate luồng, recipe tắt định chạy dynamic, LLM chỉ chốt verdict ở cuối. Agentic tool selection trên corpus dễ phần lớn gọi lại những công cụ mà bằng chứng tĩnh đã đủ để kết luận.

Bản chất non-deterministic của LLM judge

Ngay cả temperature=0, provider không bảo đảm bit-for-bit do batching phía hệ thống. Giao thức hai tầng là câu trả lời thiết kế cho giới hạn đó: tầng nào cần tắt định thì tách LLM ra, tầng nào có LLM thì báo phân phối. Consensus, như RQ3 cho thấy, không phải cách khắc phục; nó chỉ trộn thêm sampling vào sampling.

Path-sensitive: heuristic CFG chưa đủ chính xác

Bật static enrichment dựa trên CFG heuristic trên fixture Juliet trước remediation làm FP vọt từ 7 lên 44 (số đo ghi trong tài liệu đóng góp của hệ thống). Prototype Z3 từng cắt ngược về 8 nhờ phát hiện đường rò bất khả thi, nhưng bị loại khỏi kiến trúc: runtime WASM chặn heap 2 GiB, abort không bắt được, analyzer treo. In-process SMT vì thế được kết luận bất khả thi trong phạm vi này; hướng thay thế là kết hợp static với dynamic cộng judge, còn enrichment giữ opt-in.

5.4. Hạn chế của nghiên cứu

Corpus synthetic là chính. Juliet có pattern công thức: hàm `bad()` luôn leak, hàm `good()` không bao giờ. Kết quả toàn corpus bị kéo xuống bởi các family khó (C++ `new/delete`, `malloc`) mà mẫu cân bằng family che đi; không có corpus nào trong nghiên cứu này đủ lớn để khẳng định thứ bậc tuyệt đối giữa các hệ.

Một mô hình chính. Các sweep formal dùng `deepseek-v4-flash`. Hai sweep phụ B6a-mimo (F1 0.737) và B6a-zai (F1 0.774 từ run 1 hoàn chỉnh) cho thấy xu hướng giữ đúng hướng trên mô hình khác, nhưng trực model chưa bao giờ được thiết kế thành ablation; so sánh đầy đủ giữa các mô hình nằm ngoài phạm vi.

Baseline mỏng. Chỉ LAMeD (EASE 2025) là peer-reviewed đầy đủ cho leak C/C++; MemHint là preprint. Baseline so sánh trực tiếp cùng corpus, cùng scorer chỉ có Clang Static Analyzer.

Corpus thực positive-only và nhỏ. LAMeD có 41 ca, MemHint 19 ca; TN=0 nên precision và MCC không định nghĩa trên hai corpus này. FP=0 đọc phải kèm recall.

Recall dự án thực 30%. 35 trên 50 site bị bỏ sót, phần lớn thuộc các lớp cần deallocator semantics, path-sensitive feasibility hoặc ownership interprocedural.

Parse OOM trên repo khổng lồ. Tree-sitter với cache giới hạn byte vẫn có thể cạn bộ nhớ trên repo rất lớn. Đây là giới hạn scale được ghi nhận, không sửa trong phạm vi luận văn.

MemHint chưa hoàn tất khi viết chương này. Nếu các run không kết thúc được, corpus đó sẽ được báo cáo như threats-to-validity thay vì kết quả.

5.5. Hướng phát triển trong tương lai

Che phủ các family Juliet còn lại. Các flow-variant 44, 45, 63-68 (C++ reference shapes), 81-82 (virtual dispatch FN) và 83-84 (RAII FN) chưa được xử lý. Đây là nguồn FN còn lại trên corpus synthetic và là đích mở rộng tự nhiên của heuristic judge.

Dataflow interprocedural nhận diện alias. interproceduralFlow hiện theo dõi biến theo tên. Pointer analysis, chẳng hạn qua LLVM, sẽ cho theo dõi con trỏ qua biên hàm chính xác hơn; hiện tại một biến được gán lại qua tham số dễ làm mất dấu.

Mô hình hoá deallocator semantics. `cJSON_Delete` bỏ qua buffer có cờ const, một ngữ nghĩa mà code thuần không mã hoá nổi. Hướng đã chuẩn bị sẵn: LLM đọc implementation của deallocator, suy luận rule, grep-verify rồi cache theo dự án.

Path feasibility ngoài tiến trình. SMT in-process đã bị loại vì tràn heap WASM. Chạy Z3 thành tiến trình riêng, hoặc giới hạn query trên đường lỗi ngắn, là hướng còn mở.

Ablation theo trục model. Hiện chỉ có bằng chứng phụ từ hai sweep B6a-mimo và B6a-zai. Một sweep thiết kế đầy đủ nhiều mô hình, cùng corpus cùng scorer, mới trả lời được câu hỏi độ nhạy của kết quả với model.

Mở rộng scale parsing. Cắt incremental hoặc streaming cho tree-sitter trên repo không lồ để loại rủi ro OOM, kết hợp song song hoá worker parse.

Corpus khó hơn và negative samples. Mở rộng LAMeD, dựng corpus từ CVE database (DiverseVul [10], CVEfixes) và bổ sung negative samples để đo precision đầy đủ.

MCP ecosystem. Kiến trúc MCP cho phép tool mới từ cộng đồng plug vào orchestrator không cần sửa. Tool phân tích Rust ownership hoặc CVE lookup là hai ví dụ gần nhất.

Điểm neo xuyên suốt bốn chương là một lựa chọn thiết kế: những gì có thể tất định thì tất định, chỉ phần không thể (judge LLM) mới được chấp nhận dao động và được đo lường hoàng. Số liệu toàn corpus khẳng định lựa chọn đó: 0.863 so với 0.612 trong cùng sweep, std=0 trên dự án thực, và một negative result về consensus được kể trung thực. Luận văn để lại cho người sau một giao thức tái lập hai tầng, một bảng

đóng băng kết quả truy vết được, và một câu hỏi mở đáng giá về hiệu ứng sampling trong đánh giá LLM judge.

Tài liệu tham khảo

Mọi reference đã kiểm chứng qua arXiv/DOI/publisher page.

Format: IEEE style. Số [N] dùng thống nhất trong TOÀN BỘ luận văn.

A. Công cụ phân tích tĩnh

- [1] A. Bugs, "Clang Static Analyzer," LLVM Project. [Online]. Available: <https://clang-analyzer.llvm.org/>
- [2] S. Calcagno and D. Distefano, "Infer: An Interprocedural Memory Safety Analyzer for Large-Scale C Programs," Facebook Research, 2015. [Online]. Available: <https://fbinfer.com/>
- [3] GitHub, "CodeQL: Semantic Code Analysis Engine." [Online]. Available: <https://codeql.github.com/>
- [4] M. Brunsfeld, "Tree-sitter: An Incremental Parsing System for Programming Tools," 2018. [Online]. Available: <https://tree-sitter.github.io/tree-sitter/>
- [5] C. Wang, Y. Gao, W. Zhang, X. Liu, J. Guo, M. Zheng, Q. Shi, and X. Zhang, "NESA: Relational Neuro-Symbolic Static Program Analysis," in Proc. ASPLOS, 2025. arXiv:2412.14399.
- [6] G. Horvath, R. Kovacs, and Z. Porkolab, "Scaling Symbolic Execution to Large Software Systems," arXiv:2408.01909, 2024.

B. Công cụ phân tích động

- [7] N. Nethercote and J. Seward, "Valgrind: A Framework for Heavyweight Dynamic Binary Instrumentation," in Proc. PLDI, 2007. DOI: 10.1145/1250734.1250746.
- [8] S. Serebryany, D. Bruening, A. Potapenko, and D. Vyukov, "AddressSanitizer: A Fast Address Sanity Checker," in Proc. USENIX ATC, 2012.
- [9] F. Gorter and C. Giuffrida, "RangeSanitizer: Detecting Memory Errors with Efficient Range Checks," in Proc. USENIX Security, 2025.
- [10] M. Marini, D. E. Daniele Cono, M. Payer, and L. Querzoni, "QMSan: Efficiently Detecting Uninitialized Memory Errors During Fuzzing," in Proc. NDSS, 2025.

- [11] M. Marini, F. Gortler, D. C. D'Elia, and C. Giuffrida, "CombiSan: Unifying Software Sanitizers for Comprehensive Fuzzing," preprint, 2025–2026.
- [12] Q. Sang, Y. Wang, Y. Liu, X. Jia, T. Bao, and P. Su, "AirTaint: Making Dynamic Taint Analysis Faster and Easier," in Proc. IEEE S&P, 2024.
- [13] K. Hassler, P. Goerz, and S. Lipp, "A Comparative Study of Fuzzers and Static Analysis Tools for Finding Memory Unsafety in C and C++," arXiv:2505.22052, 2025.
- [14] A. Murali, M. Alfadel, and M. Nagappan, "AddressWatcher: Sanitizer-Based Localization of Memory Leak Fixes," IEEE Trans. Softw. Eng., 2024.

C. LLM cho phát hiện lỗi

- [15] A. Khare, S. Dutta, Z. Li, A. Solko-Breslin, R. Alur, and M. Naik, "Understanding the Effectiveness of Large Language Models in Detecting Security Vulnerabilities," arXiv:2311.16169, 2023.
- [16] Z. Li, S. Dutta, and M. Naik, "IRIS: LLM-Assisted Static Analysis for Detecting Security Vulnerabilities," arXiv:2405.17238, 2024.
- [17] I. Ceka, F. Qiao, A. Dey, A. Valecha, G. Kaiser, and B. Ray, "Can LLM Prompting Serve as a Proxy for Static Analysis in Vulnerability Detection," arXiv:2412.12039, 2024.
- [18] Y. Nie, H. Li, C. Guo, R. Jiang, Z. Wang, B. Li, D. Song, and W. Guo, "VulnLLM-R: Specialized Reasoning LLM with Agent Scaffold for Vulnerability Detection," arXiv:2512.07533, 2025.
- [19] J. Ghebremichael, S. Vasan, S. Ullah, G. Tystahl, D. Adei, C. Kruegel, G. Vigna, W. Enck, and A. Kapravelos, "SemTaint: Multi-Agent Taint Specification Extraction for Vulnerability Detection," arXiv:2601.10865, 2026.

D. Leak C/C++ trực tiếp (baseline chính)

- [20] H. Huang, J. Shi, B. Wang, Z. Yang, and D. Lo, "MemHint: Finding Memory Leaks in C/C++ Programs via Neuro-Symbolic Augmented Static Analysis," arXiv:2603.27224, 2026.

- [21] E. Shemetova, I. Shenbin, I. Smirnov, A. Alekseev, A. Rukhovich, S. Nikolenko, V. Lomshakov, and I. Piontkovskaya, "LAMEd: LLM-generated Annotations for Memory Leak Detection," arXiv:2505.02376, 2025.
- [22] D. Liu, Z. Lu, S. Ji, K. Lu, J. Chen, and Z. Liu et al., "Detecting Kernel Memory Bugs Through Inconsistent Memory Management Intention Inferences," in Proc. USENIX Security, 2024.

E. Agentic / Multi-agent cho SE

- [23] Y. Hou, H. Wang, M. Lyu, M. Momeu, E. Nguyen, T. Yang, K. Sen, D. Song, and D. Wagner, "Revelio: Cost-Efficient Agentic Memory Safety Vulnerability Detection For Repository-Scale Codebases," arXiv:2606.22263, 2026.
- [24] M. Shafiuzzaman, A. Desai, W. Guo, and T. Bultan, "SAILOR: Guiding Symbolic Execution with Static Analysis and LLMs for Vulnerability Discovery," arXiv:2604.06506, 2026.
- [25] J. Guo, C. Wang, X. Xu, Z. Su, and X. Zhang, "RepoAudit: An Autonomous LLM-Agent for Repository-Level Code Auditing," arXiv:2501.18160, 2025.
- [26] Z. Sheng, Z. Chen, Q. Xu, K. Zhu, and J. Huang, "FuzzingBrain V2: A Multi-Agent LLM System for Automated Vulnerability Discovery and Reproduction," arXiv:2605.21779, 2026.
- [27] C. Zhang, Y. Park, F. Fleischer, Y.-F. Fu, J. Kim et al., "SoK: DARPA's AI Cyber Challenge (AIXCC): Competition Design, Architectures, and Lessons Learned," arXiv, 2026.

F. LLM Foundations

- [28] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing Reasoning and Acting in Language Models," in Proc. ICLR, 2023. arXiv:2210.03629.
- [29] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou, "Self-Consistency Improves Chain of Thought Reasoning in Language Models," in Proc. ICLR, 2023. arXiv:2203.11171.

- [30] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language Agents with Verbal Reinforcement Learning," in Proc. NeurIPS, 2023. arXiv:2303.11366.
- [31] S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan, "Tree of Thoughts: Deliberate Problem Solving with Large Language Models," in Proc. NeurIPS, 2023. arXiv:2305.10601.
- [32] O. Khattab et al., "DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines," in Proc. ICLR, 2024. arXiv:2310.04618.
- [33] T. Guo, X. Chen, Y. Wang, R. Chang, S. Pei, N. V. Chawla, O. Wiest, and X. Zhang, "Large Language Model based Multi-Agents: A Survey of Progress and Challenges," arXiv:2402.01680, 2024.
- [34] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering," arXiv:2405.15793, 2024.
- [35] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang, "Agentless: Demystifying LLM-based Software Engineering Agents," arXiv:2407.01489, 2024.
- [36] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena," in Proc. NeurIPS Datasets and Benchmarks Track, 2023. arXiv:2306.05685.

G. Benchmarks & Datasets

- [37] NIST, "Juliet Test Suite v1.3 for C/C++," SAMATE/SARD, 2017. [Online]. Available: <https://samate.nist.gov/SARD/testsuite.php>
- [38] LAMeD Benchmark, "memleak_benchmark.json," Zenodo record 15089703, 2025. DOI: 10.5281/zenodo.15089703.
- [39] Y. Chen, Z. Ding, L. Alowain, X. Chen, and D. Wagner, "DiverseVul: A New Vulnerable Source Code Dataset for Deep Learning Based Vulnerability Detection," in Proc. RAID, 2023. arXiv:2304.00409.

- [40] SV-COMP, "Software Verification Competition — Memsafety Category," 2024. [Online]. Available: <https://sv-comp.sosy-lab.org/2024/>
- [41] Magma Benchmark, "A Ground-Truth Fuzzing Benchmark." [Online]. Available: <https://magma-benchmark.github.io/>

H. Giao thức & Tiêu chuẩn

- [42] Anthropic, "Model Context Protocol (MCP) Specification," 2024. [Online]. Available: <https://modelcontextprotocol.io/>
- [43] D. Svoboda, W. Klieber, L. Flynn, R. Martins, and J. Hoskinson, "A Pointer-Ownership Model for C Inspired by Rust," in Proc. LCTES (ACM SIGPLAN), 2026. DOI: 10.1145/3814943.3816182.

I. Kiến trúc & hiệu năng phân tích tĩnh quy mô lớn

- [44] r2c, Inc., "Semgrep: Lightweight Static Analysis for Many Languages." [Online]. Available: <https://semgrep.dev>
- [45] C. Sadowski, J. van Gogh, C. Jaspan, E. Söderberg, and C. Winter, "Tricorder: Building a Program Analysis Ecosystem," in Proc. 37th Int. Conf. on Software Engineering (ICSE), 2015, pp. 598–608. DOI: 10.1109/ICSE.2015.76.
- [46] C. Calcagno, D. Distefano, J. Dubreil, D. Gabi, P. Hooimeijer, M. Luca, P. W. O'Hearn, I. Papakonstantinou, J. Purbrick, and D. Rodriguez, "Moving Fast with Software Verification," in NASA Formal Methods (LNCS vol. 9058), Springer, 2015, pp. 3–11.
- [47] T. Szabó, "Incrementalizing Production CodeQL Analyses," in Proc. 31st ACM Joint European Software Engineering Conf. and Symp. on the Foundations of Software Engineering (ESEC/FSE), 2023. DOI: 10.1145/3611643.3613860. arXiv:2308.09660.

Phụ lục A: Bảng kết quả chi tiết

A.1. Full confusion matrices — 9-baseline ablation (Juliet n=50, stratified)

ID	Baseline	TP	FP	FN	TN	P	R	F1	Acc	MCC	ECE	FP/KLOC	Token
B1	Static only	42	11	11	146	0.792	0.792	0.792	0.934	0.800	0.548	0.672	0
B2	Dynamic only	35	0	24	0	1.000	0.593	0.745	—	—	0.041	0.000	0
B3	Rule-based ensemble	48	11	5	146	0.814	0.906	0.857	0.924	0.827	0.161	0.672	0
B4	LLM + static	50	18	3	139	0.735	0.943	0.826	0.898	0.750	0.054	1.121	1,310,030
B5	LLM + dynamic	35	0	24	0	1.000	0.593	0.745	—	—	0.003	0.000	37,529
B6	LLM + all (no planner/sel)	48	1	5	156	0.980	0.906	0.941	0.971	0.923	0.129	0.062	455,434
B6a	+ planner only	48	1	5	156	0.980	0.906	0.941	0.971	0.923	0.125	0.062	463,047
B6b	+ tool_selector only	48	2	5	155	0.960	0.906	0.932	0.967	0.911	0.128	0.123	4,239,560
B7	Full adaptive	48	2	5	155	0.960	0.906	0.932	0.967	0.911	0.130	0.123	4,115,938

Lưu ý: B2 và B5 có TN=0 vì dynamic-only không enumerate site sạch (positive-only, giống Clang). Acc/MCC không tính được.

A.2. Full confusion matrices — n=100 stratified

ID	Baseline	TP	FP	FN	TN	P	R	F1	ECE
B1	Static only	78	22	24	289	0.780	0.765	0.772	0.542
B2	Dynamic only	68	0	46	0	1.000	0.596	0.747	0.060
B3	Rule-based ensemble	90	22	12	289	0.804	0.882	0.841	0.139
B4	LLM + static	97	41	5	270	0.703	0.951	0.808	0.064
B5	LLM + dynamic	67	0	45	0	1.000	0.598	0.749	0.004
B6	LLM + all (no planner/sel)	86	0	16	311	1.000	0.843	0.915	0.112
B6a	+ planner only	89	0	13	311	1.000	0.873	0.932	0.120
B6b	+ tool_selector only	88	2	14	309	0.978	0.863	0.917	0.123
B7	Full adaptive	88	2	14	309	0.978	0.863	0.917	0.122

A.3. 2×2 ablation (Juliet n=30)

	TP	FP	FN	TN	P	R	F1
no_llm, dynamic off	29	7	3	38	0.806	0.906	0.853
no_llm, dynamic on	29–30	7	2–3	38	0.806–0.811	0.906–0.938	0.853–0.865
llm assisted,	29	7	3	38	0.806	0.906	0.853

	TP	FP	FN	TN	P	R	F1
dynamic off							
llm_assisted, dynamic on	29–30	7	2–3	38	0.806–0.811	0.906–0.938	0.853–0.865

TP dao động 29↔30 do flow-variant 12 (globalReturnsTrueOrFalse()) leak trên ~50% run.

A.4. Static evidence-tool ablation (B1, n=50)

Static tools	TP	FP	FN	TN	P	R	F1	ECE
none (candidateScan only)	42	11	11	146	0.792	0.792	0.792	0.548
+ functionSummary	42	11	11	146	0.792	0.792	0.792	0.492
+ pathConstraints	42	11	11	146	0.792	0.792	0.792	0.503
+ both (default)	50	13	3	146	0.794	0.943	0.862	0.555
+ interproceduralFlow	50	13	3	146	0.794	0.943	0.862	0.554
+ scanBuild	50	13	3	146	0.794	0.943	0.862	0.547

A.5. LAMeD — bộ run 2026-08-20 trên mẫu đã sửa (50 site)

Bộ bốn run chốt của luận văn trên corpus LAMeD (41 ca / 7 dự án), chấm trên mẫu 50 site đã sửa (số 44 site cũ sai do lỗi computeBundleId gộp nhầm các candidate, đã fix ở commit 41bf1ec: libsolvr 6→11 site, cJSON 6→7). Điểm số theo quy ước positive-only: recall và số FP là hai số liệu so sánh được, precision chỉ mang tính thông tin (TN = 0 theo cấu trúc corpus nên specificity và MCC không định nghĩa được).

Cấu hình	Sites	TP	FP	FN	Recall	Precision
no_llm mặc định	50	15	0	35	0.300	1.000
no_llm + interproceduralFlow	50	15	0	35	0.300	1.000
llm_assisted (mean 3 runs, std = 0)	50	15	0	35	0.300	1.000
Clang SA baseline	43	0	0	43	0.000	—

Ghi chú:

- Hai run no_llm (mặc định so với +interproceduralFlow) cho kết quả byte-identical trên cả 41 ca: thêm interproceduralFlow không đổi gì ($\Delta = 0$). Chi tiết phân tích nguyên nhân ở results/lamed-correction-2026-08-20-README.md mục 2.
- llm_assisted chạy 3 run độc lập (std = 0), cho kết quả trùng khớp no_llm: judge LLM được gọi thật 47–149 lần mỗi run nhưng không lật verdict nào trên corpus này.

- Clang SA chấm trên 43 site riêng của nó (TP0/FN43). Số site 43 so với 50 không cùng một mẫu số, đây là thiết kế của scorer (mỗi tool bị chấm theo chính tập finding của nó), không phải lỗi chưa fix: xem `results/lamed-correction-2026-08-20-README.md` mục 4.

A.6. Consensus ablation — verdict stability

A.6.1. Kết quả chốt: n=50 stratified (kết quả đảo chiều)

Đo lại trên mẫu stratified n=50 round-robin (2 runs mỗi nhánh, model `deepseek-v4-flash`, commit `f0d371c`, 2026-08-19), 205 site chấm được. Kết quả đảo chiều so với thí nghiệm n=30 ban đầu: single-LLM ổn định hơn và chính xác hơn consensus.

Judge arm	Flip rate	F1
single-LLM (K=1)	2.0%	0.852
consensus (K=3)	8.0%	0.793

McNemar paired trên 205 site: trong các site bất đồng, đa số nghiêng về single-LLM; $\chi^2 = 3.13$, $p = 0.077$, xu hướng lean single nhưng chưa đạt ngưỡng ý nghĩa thống kê ở quy mô này. Kết luận chính thức: consensus không được khuyến nghị làm mặc định (kết quả âm tính có chủ đích, xem mục 4.6).

A.6.2. Thí nghiệm ban đầu n=30 (giữ làm dữ liệu lịch sử có chú thích)

Thí nghiệm consensus đầu tiên chạy trên n=30 (100% family char), 2 campaign độc lập A/B, 2 runs mỗi nhánh. Là thí nghiệm sớm trên mẫu đơn-family lệch dễ: kết quả bên dưới không dùng làm số liệu chốt của luận văn, giữ lại để kể trung thực trình tự thí nghiệm (phân tích nguyên nhân đảo chiều ở mục 4.6.3).

Judge arm	Campaign	Case stability	Flip rate	Modal agreement
single-LLM (K=1)	A	22/30 (73.3%)	8/30 (26.7%)	26/30 (86.7%)
single-LLM (K=1)	B	26/30 (86.7%)	4/30 (13.3%)	28/30 (93.3%)
consensus (K=3)	A	28/30 (93.3%)	2/30 (6.7%)	29/30 (96.7%)
consensus (K=3)	B	28/30 (93.3%)	2/30 (6.7%)	29/30 (96.7%)

A.7. LLM variance (multi-run, 3 runs)

Config	Metric	Mean	Std	Min	Max
B6a	Precision	0.973	0.031	0.940	1.000
B6a	Recall	0.899	0.011	0.887	0.906
B6a	F1	0.935	0.020	0.913	0.950

Config	Metric	Mean	Std	Min	Max
B6a	Accuracy	0.968	0.010	0.957	0.976
B6a	MCC	0.915	0.027	0.885	0.937
B6a	ECE	0.129	0.004	0.126	0.133
B7	Precision	0.973	0.012	0.959	0.980
B7	Recall	0.906	0.019	0.887	0.925
B7	F1	0.938	0.015	0.922	0.951
B7	ECE	0.125	0.004	0.121	0.127

A.8. MemHint (19 ca / 6 dự án thực)

Bộ run MemHint trên corpus 19 ca tự tái lập (6 dự án thực), hai cấu hình theo protocol hai-công-đoạn giống LAMeD: `no_llm --enrich ×1` và `llm_assisted ×3`. Chấm theo quy ước positive-only (recall + FP; TN = 0 nên precision/specificity không định nghĩa được). Corpus gồm 6 dự án: `tmux`, `curl`, `openssl`, `redis`, `vim`, `freerdp`, tức 6 trong 7 dự án mục tiêu của MemHint (arXiv:2603.27224, bản v3), tái lập độc lập theo `demo/memhint/memhint_bugs.json`. Run chốt 2026-09-05, commit 30e04cb1c, hash corpus 442de35d6410bdd03d30b665b5f0f912; artifacts tại `results/memhint-no_llm-2026-08-28/` + `results/memhint-llm_assisted-2026-08-28/`.

Cấu hình	Sites	TP	FP	FN	Recall	Precision	F1
<code>no_llm --enrich ×1</code>	26	12	0	14	0.462	1.000	0.632
<code>llm_assisted</code> (mean 3 runs, std = 0)	26	12	0	14	0.462	1.000	0.632

Phân bố TP/FN theo dự án (giống hệt nhau ở mọi run):

Dự án	TP	FN	Sites
<code>tmux</code>	3	1	4
<code>curl</code>	1	2	3
<code>openssl</code>	1	2	3
<code>redis</code>	3	3	6
<code>vim</code>	0	6	6
<code>freerdp</code>	4	0	4
Tổng	12	14	26

`vim` là khối FN tập trung nhất (0/6, mọi site đều bỏ sót); `freerdp` thì ngược lại, toàn TP (4/4).

Ghi chú:

- `llm_assisted` chạy 3 run độc lập (model `deepseek-v4-flash`, temp 0): cả 3 run cho cùng một confusion matrix (12, 0, 14), std = 0 trên cả ba số P 1.000±0.000 / R 0.462±0.000 / F1 0.632±0.000.

- Judge heuristic quyết định ~17.1k flagged verdict mỗi run; judge LLM chỉ được gọi ở 1 trong 3 run, trên đúng 2 site (run 2: 2 flagged verdict trong `freerdp_9fc23ad2`, judge paths `{heuristic:17154, llm:2}`; run 1/3: chỉ heuristic, `{heuristic:17157}` / `{heuristic:17106}`). Không site nào borderline đủ để làm đổi confusion matrix nên theo thiết kế judge lại (LLM chỉ chạy trên bundle BORDERLINE), `llm_assisted` cho kết quả trùng khớp `no_llm`. Công đoạn LLM vẫn chạy ở giai đoạn thu thập bằng chứng tĩnh, song không lật verdict nào.
- $FP = 0$ và $P = 1.000$ là tính chất của cách chấm positive-only trên corpus chỉ gồm site dương ($TN = 0$), không phải kết quả của một phép phân loại đầy đủ; recall và số FP mới là số liệu so sánh được, precision chỉ mang tính thông tin.
- Đối chiếu với anchor từng phần 2026-08-28 ($R\ 42.3\% = 11/26$, 8 ca mất do static-analyzer OOM): số chốt $R\ 46.2\%$ ($12/26$) chênh đúng +1 TP, đến từ các ca được phục hồi. Vì `no_llm` bitwise-deterministic, 11 ca đã chấm trước đó phải cho kết quả cũ nên TP cộng thêm chỉ có thể nằm trong 8 ca chưa chấm (chi tiết ở `docs/CURRENT_STATUS.md §Pending #2`).
- Run hoàn tất ở lần thử thứ 3/8 của driver; lần thử 1 mất do VM reboot giữa run và được khôi phục bằng cơ chế `--resume`, artifacts sha256-verified byte-identical qua lần reboot.

Phụ lục B: Prompt templates (verbatim)

Mọi prompt dưới đây trích xuất trực tiếp từ source code. File gốc ghi rõ ở mỗi mục.

B.1. Static sub-agent — system prompt

File: apps/leak-inspector-tui/src/domain/subAgentPrompts.ts:39-54

You are a STATIC-ANALYSIS evidence-gathering sub-agent for C/C++ memory leaks.

You do NOT record verdicts — your only job is to RUN the static tools so the system can collect structured evidence for each candidate, then call ``done_static``.

For EACH candidate in your list, gather evidence:

- ``functionSummary`` (filePath, functionName) — alloc/free balance + leaky exit paths.
- ``pathConstraints`` (filePath, lineNumber of the allocation) — feasible leaking paths.
- ``astScan`` (filePath) — structural patterns + early returns.
- ``ownershipConventions`` (filePath) — ownership-transfer / missing-free conventions.
- ``read_file`` to inspect the source and, for interprocedural cases (a function returning an allocation), follow the caller.

Efficiency: you MAY call several of these read-only tools in a SINGLE turn — they run in parallel. The repository root is `${repoPath}`.

When you have run the static tools for EVERY candidate in your list, call ``done_static``. Do NOT reply with prose — only tool calls advance the work.

B.2. Static sub-agent — user message

File: subAgentPrompts.ts:56-63

Gather static evidence for these `${N}` candidate allocation site(s):

- `${bundleId}` — `${function}()` at `${file}:${line}` (`${allocation_type}`)
- ...

Run the static tools for each, then call `done_static`.

B.3. Dynamic worker — system prompt

File: subAgentPrompts.ts:67-78

You are a DYNAMIC-ANALYSIS sub-agent for C/C++ memory leaks. Build the project ONCE with a sanitizer, run it under a sanitizer, then call ``done_dynamic``.

1. ``read_file`` the Makefile / CMakeLists.txt / build script under `${repoPath}` to learn how it builds. A hint build command was provided: ``${buildCommand}``.

2. ``buildTarget`` (projectPath=\${repoPath}, buildCommand = a clang command with sanitizer flags). Prefer LeakSanitizer (``-fsanitize=leak -g -O0``) – it reports at exit and never aborts mid-run.
3. Run the binary with ``lsanRun`` (or ``asanRun`` / ``valgrindMemcheck``).

The system CAPTURES every finding from your sanitizer runs AUTOMATICALLY and attaches it to the matching candidate – you do NOT record evidence yourself. Your only job is to get a successful sanitizer run. Build at most ONCE and run each dynamic tool at most once. If a build or sanitizer fails twice, stop and call ``done_dynamic``. When a sanitizer has run, call ``done_dynamic``. Do NOT reply with prose.

B.4. LLM Judge — system prompt

File: `apps/leak-inspector-tui/src/domain/llmJudge.ts`

(SYSTEM_PROMPT)

You are an expert C/C++ memory-leak analyst. Decide whether ONE allocation is a real leak, using the code, static context, and any runtime evidence provided.

Respond with a JSON object ONLY (no prose), in this exact shape:

```
{
  "verdict": "confirmed_leak | likely_leak | uncertain |
likely_false_positive | false_positive",
  "confidence": 0.0-1.0,
  "explanation": "...",
  "evidence": ["..."]
}
```

Calibrate using the EVIDENCE, in this priority order:

- A runtime leak (sanitizer/valgrind) whose allocation site is LINKED to this candidate is decisive → `confirmed_leak` (confidence ≥ 0.9). Weight by leak kind: `definitely_lost` / `asan_leak` ⇒ decisive; `possibly_lost` ⇒ weak corroboration; `still_reachable` ⇒ usually benign, lean `false_positive`.
- A runtime finding in the SAME FILE but a DIFFERENT site (not linked) is weak – do not treat it as proof for this allocation. `still_reached` with no other evidence → `false_positive`.
- A CLEAN sanitizer/valgrind run that EXERCISED this allocation and reported NO leak here is strong evidence this is NOT a leak → lean `false_positive` / `likely_false_positive` (unless a runtime leak is LINKED to this very allocation).
- Ownership is decisive for false positives: if the allocation is RETURNED to the caller or its pointer is HANDED OFF to a sink/callback/another function (ownership transferred), freeing it is NOT this function's job. When ownership is transferred AND no runtime leak is linked to THIS allocation, answer `likely_false_positive` or `false_positive` – do NOT flag it just because you cannot see the free inside this snippet. An UNPAIRED `alloc`→`free` with a reachable leak path and NO ownership transfer → `confirmed_leak` (≥ 0.85).
- PATH-SENSITIVE leak: an allocation freed on the main/success path but NOT on an error or early-return path (e.g. ``if (err) return NULL;`` or ``goto fail;`` before the free) IS a leak – `confirmed_leak` – EVEN IF the value is returned or added to a structure on the success path. If the static context lists the allocation as freed "on some paths only" (conditional) or names it on a reachable un-freed exit path, treat that as decisive.
- PARAMETER-ownership leak (allocation_type 'parameter_ownership'): a function that frees a pointer PARAMETER on some paths (taking ownership) but returns on a reachable branch WITHOUT freeing it leaks that parameter – `confirmed_leak` (e.g. cJSON's ``merge_patch``).
- Freed on all paths / static-global → `false_positive` (high confidence). Use uncertain only when the evidence is genuinely insufficient.
- Control flow is concrete, not hypothetical: a constant or scaffolding global such as ``if(1)``/``if(0)`` or ``globalReturnsTrue()`` does NOT change

between two checks in the SAME function – `if(1)` always runs and `if(0)` is dead code. If the buffer is freed under the same condition it was allocated (or in the `else` of a constant `if`), it IS freed. Do NOT call a leak just because the `free()` sits in a different block, behind a constant condition, or after a `break`/in a second loop – trace whether it actually executes.

B.5. LLM Judge — user message template

File: `llmJudge.ts:146-161`

```
ALLOCATION SITE: ${function}() at ${file}:${line} (${allocation_type})

CODE (context around the allocation):
\`\`\`c
${sourceSnippet}
\`\`\`

STATIC ANALYSIS CONTEXT:
${summarizeStatic}

DYNAMIC EVIDENCE (${N}):
${summarizeEvidence}

PROJECT OWNERSHIP CONVENTIONS:
- ${ownershipNotes}

Return your JSON verdict.
```

B.6. Allocator profiler — system prompt (tóm tắt)

File: `apps/leak-inspector-tui/src/domain/allocatorProfiler.ts`

LLM đọc header + source → liệt kê API cấp phát/giải phóng custom (factory *_new/*_create/*_dup, parser/printer trả owned, macro #define ALLOC, smart-ptr make_unique) + ownershipNotes (quy ước sở hữu: transfer/borrow/refcount/pool). Output JSON {allocators, deallocators, reallocators, ownershipNotes, confidence, explanation}.

B.7. Completion nudges

Static completion nudge (`workflowInvestigation.ts:192-197`):

You stopped, but \${N} candidate(s) have NO static evidence yet: \${ids}. Run functionSummary/pathConstraints/astScan/ownershipConventions for them, then call done_static. Only tool calls advance the work.

Dynamic completion nudge (`workflowInvestigation.ts:243-246`):

No successful sanitizer run yet. buildTarget (with a sanitizer flag), then run lsanRun/asanRun/valgrindMemcheck, then call done_dynamic. Only tool calls advance the work.

Phụ lục C: Cài đặt và hướng dẫn chạy

C.1. Yêu cầu hệ thống

Thành phần	Yêu cầu
OS	Linux (dynamic analysis); macOS/Linux (static only)
Docker	≥ 20.10 (chạy analyzer)
Node.js	≥ 22 (runtime chính)
pnpm	≥ 11 (package manager, qua corepack)
Clang	≥ 14 (compile test case Juliet, scan-build)
Valgrind	≥ 3.18 (chỉ Linux, optional)
LLM Gateway	OpenAI-compatible endpoint (mặc định: localhost:20128)

C.2. Cài đặt

```
# Clone repo
git clone <repo-url> && cd leak-investigator

# Install dependencies
pnpm install

# Build all packages + apps
pnpm run build

# Start analyzers (Docker)
docker compose up --build -d

# Verify analyzers
curl -s http://localhost:50061/mcp # static-analyzer
curl -s http://localhost:50062/mcp # dynamic-analyzer
```

C.3. Cấu hình LLM

Tạo file `.env` ở root hoặc `apps/leak-inspector-tui/.env`:

```
# openai-compat gateway – cấu hình của các run chốt trong FREEZE
(deepseek-v4-flash, temp 0)
LLM_PROVIDER=openai-compat
LLM_BASE_URL=https://api.deepseek.com/v1
LLM_API_KEY=sk-...
LLM_MODEL=deepseek-v4-flash

# Local gateway (thí nghiệm đầu 2026-06, mục 4.2.3; model mimo/mimo-v2.5-
pro – cấu hình cũ,
# không phải cấu hình của các run chốt)
# LLM_PROVIDER=local
# LLM_BASE_URL=http://localhost:20128/v1
# LLM_API_KEY=not-needed
# LLM_MODEL=mimo/mimo-v2.5-pro

# Hoặc OpenAI
# LLM_PROVIDER=openai
# LLM_API_KEY=sk-...
```

```
# Hoặc Anthropic
# LLM_PROVIDER=anthropic
# LLM_API_KEY=sk-ant-...
```

C.4. Chạy đánh giá

```
# Single run - deterministic (no_llm)
pnpm exec tsx scripts/evaluate-corpus.ts no_llm --corpus
demo/juliet_cwe401 --dynamic selective

# LLM-assisted, 5 runs → mean ± std
pnpm exec tsx scripts/evaluate-corpus.ts llm_assisted --corpus
demo/juliet_cwe401 --runs 5

# 9-baseline sweep (stratified n=50)
pnpm exec tsx scripts/run-baselines.ts --corpus demo/juliet_cwe401 --
limit 50 --stratify

# So sánh với Clang SA
pnpm exec tsx scripts/compare-baselines.ts --corpus demo/juliet_cwe401 --
limit 30

# Determinism gate
bash scripts/determinism-gate.sh

# Consensus ablation
K=3 LIMIT=30 bash scripts/consensus-ablation.sh
```

C.5. Cấu hình nâng cao

Mọi setting đều có thể cấu hình qua (thứ tự ưu tiên giảm dần):

1. CLI flags (--provider, --model, --dynamic, --consensus-n, v.v.)
2. Env vars (LLM_PROVIDER, CONSENSUS_N, AGENT_MAX_TURNS, v.v.)
3. Config file ~/.config/cleak/config.json
4. Default values trong apps/leak-inspector-tui/src/config.ts

Xem thêm: `cleak config` (hiển thị cấu hình hiện tại) hoặc lệnh `/config` trong TUI.

C.6. Cấu trúc output

Mỗi lần scan tạo `results/<scanId>/` chứa:

File	Nội dung
snapshot.json	Findings format chuẩn (so sánh được giữa các run)
report.json	Full report với verdict, evidence, explanation
report.md	Báo cáo Markdown
report.html	Báo cáo styled HTML
events.jsonl	Stream events (tăng dần)
transcript.json	Lịch sử message agent
steps.md	Log từng bước
metrics.json	Phân bố verdict, token, thời lượng

Phụ lục D: Bảng tham chiếu tool MCP

D.1. Static analyzer — 11 tools

#	Tool	Chức năng	Content-capable	Phoi cho TUI
1	indexFiles	Quét đệ quy tất cả file C/C++ từ root path	✗	✗
2	candidateScan	Lexical scan tìm allocation site (malloc, new, factory)	✓	✓
3	astScan	Phân tích cấu trúc AST cho memory leak pattern	✓	✓
4	callGraph	Trích xuất đồ thị gọi hàm	✗	✗
5	functionSummary	Tóm tắt hàm: alloc/free balance, exit path	✓	✓
6	pathConstraints	Phân tích ràng buộc đường đi quanh allocation	✓	✓
7	interproceduralFlow	Truy vết alloc/free liên hàm	✗	✗
8	ownershipSummary	Tóm tắt quy ước ownership qua nhiều file	✗	✗
9	ownershipConventions	Phát hiện quy ước transfer ownership trong file	✓	✓
10	scanBuildRun	Chạy Clang Static Analyzer (scan-build)	✗	✗
11	scanBuildGetReport	Lấy kết quả scan-build	✗	✗

Content-capable = nhận content inline (không cần mount filesystem). TUI chỉ expose 5 content-capable tools cho sub-agent tĩnh.

D.2. Dynamic analyzer — 9 tools

#	Tool	Chức năng	Serial/Heavy
1	buildTarget	Build dự án với sanitizer flags	✓
2	valgrindMemcheck	Chạy Valgrind	✓

#	Tool	Chức năng	Serial/Heavy
		Memcheck	
3	valgrindGetReport	Lấy normalized Valgrind report	✗
4	valgrindListFindings	Query findings với filter	✗
5	valgrindCompareRuns	So sánh hai lần chạy Valgrind	✗
6	asanRun	Chạy binary với AddressSanitizer	✓
7	lsanRun	Chạy binary với LeakSanitizer	✓
8	runBinary	Chạy binary không instrumentation	✗
9	listRuns	Liệt kê các lần chạy đã lưu	✗

Serial/Heavy = chạy tuần tự, timeout 300s, cần phê duyệt (ask).

D.3. Tool domain

Tool	Chức năng	File
read_file	Đọc source file từ repository (tối đa 16000 chars)	domainTools.ts
done_static	Kết thúc vòng lặp static sub-agent	workflowInvestigation.ts:183
done_dynamic	Kết thúc vòng lặp dynamic worker	workflowInvestigation.ts:233

D.4. Tool execution policy

Chính sách	Áp dụng	Hành vi
CONCURRENCY_SAFE	Tool read-only nhẹ	Chạy song song, timeout 30s
SERIAL_HEAVY	build/sanitizer/scan-build	Chạy tuần tự, timeout 300s, cần ask
Content-capable	5 static tools	Nhận content inline, không cần filesystem mount
Non-content	6 static tools + dynamic	Cần path map (EVAL_STATIC_PATH_MAP trong Docker)

D.5. MCP protocol

- **Transport:** Streamable-HTTP, POST /mcp (JSON-RPC 2.0, stateless JSON mode)
- **Tool schema:** Zod inputSchema → MCP SDK tự convert sang JSON Schema
- **Client:** @modelcontextprotocol/sdk
StreamableHTTPClientTransport
- **Ports:** static-analyzer : 50061, dynamic-analyzer : 50062